

C++

Dr. Justin Furuness

- In this course we'll be going over C++ and Object Oriented Design!
- We'll also cover various tools that are critical for low-level developers (such as C++ devs):
 - The terminal, Git/GitHub, Virtual Environments, Testing (with PyTest), Vim/Emacs, Tmux, etc.
 - All of which are very critical for jobs these days
 - It would be very hard to get hired in my opinion if you don't know the basics of these tools
- Also, please just raise your hand or interrupt me with questions as we go, for this lecture and all others

- Laurent filmed an earlier version of this course
- Definitely was a HUGE inspiration as to what the course became today
- His videos are great! But they are in a different order as to what we'll be covering
- Additionally, I likely won't be making videos this term, as my time will be devoted towards building the material
- So make sure you show up to class!

Why Learn C++?

- C++ is one of the more powerful programming languages
- It allows for extremely fine-grained control of memory, performance, and design
- These qualities make it the go-to language for critical software systems.

Applications of C++

- Operating Systems, Compilers
- Games and Graphics
- Finance (HFT, financial systems) and Databases
- Large-scale Simulations, CUDA
- It's in most places where performance really matters

C++ vs Other Languages

- Python
 - Way higher level than C++, as we'll see
 - 10-100x slower for the same code in most cases
 - Generally considered faster to develop in
 - Errors easier to debug
- Java
 - Falling in popularity in industry
 - Worst of both worlds in comparison to Python/C++
 - Not as high level as Python, not as low level as C++
- Go
 - Google made this
 - It never really took off in the larger industry like these other languages
 - IMO industry is moving away from Go, preferring Rust

C++ vs Other Languages

- C

- C++ is mostly a superset of C, i.e. C plus classes!
- C lacks a lot of the abstractions that C++ provides
- Still used where performance is extremely critical and/or code must be very lightweight, like with microcontrollers

- Rust

- Mostly a subset of C++, with some new features for memory safety
- Forces you to code in a non-object-oriented way (no classes)
- Voted most popular language nine years in a row on stack overflow
- A huge portion of industry is moving to this for a significant portion of the stack
- Rust is hard to understand without first knowing C++!

C++ vs Other Languages

- Objective-C
 - formerly dominant in Apple ecosystem
 - largely replaced by Swift for app development.
- C#
 - Microsoft-backed, used in some Windows applications and Unity game engine.
 - These days, even Microsoft typically uses C++ instead of C#
 - Never was even close to as popular as C++
- So C++ really is a great language to learn!

- This course is designed to help you on the career path towards a C++ Software Engineer
 - If you can master C++ and write good performant code, you'll be golden
- This is very different from other jobs
- Python developer
 - Generally the back-end, requiring learning vast amounts of libraries:
 - FastAPI, Django, Numpy, Pandas, Docker, etc.
- The same is true for front-end, infrastructure, etc.
 - An large breadth of knowledge is needed to be competitive

- Salaries: typically much higher for C++ jobs
 - Other languages are simply easier to learn and more people know them
- C++ is also more resilient to AI than other languages
 - The faster code becomes, the less clean it gets in general and the harder it is for AI to follow
 - There's also a lot less code examples online compared to something like Python
- See <https://levels.fyi/2025> for salary data. Something to strive for!

Prerequisites

- You're supposed to have CSE3100, CSE2050, CSE1010, etc.
- That being said, students told me last semester that they nothing really stuck from these courses, especially when it comes to knowing C
- Additionally there is the break between semesters as well so plenty is forgotten
- I try to teach the course assuming little to no pre-requisite knowledge
 - I may even try to remove some of these prerequisites in the future
 - Be warned though - the course picks up speed rather quickly

C++ as a First Language

- For those coming from non-CS majors:
- I don't recommend learning C++ as your first language
- Other languages like Python or even JavaScript (ugh) are much easier
- C++ can get pretty in depth, and this class will dive deep on it for sure
- That being said, some people like learning lower level languages like C++ first
 - To each their own

- I also wanted to touch on the difficulty for this course
- I try to align my expectations of you with what is going to be expected when you go out and try to get a job
- For most, that means this will be a very hard class
 - To me it's worthwhile and you'll learn a lot of skills that are necessary for getting a job
 - But that does involve a considerable amount of hard work

- This is also a 3000 level course
- Expectations are going to be higher than other lower level courses
- If you fall behind, the expectation is that you show up to office hours and catch up
- For most, if you don't show up to class, or use AI on all the homework assignments, you'll likely fail the class
 - I know that's not the case with other classes, I'm just trying to set expectations early so that it's not a surprise

- AI is amazing and there are some amazing AI tools out there
- Boris Cherny (created Claude Code at Anthropic) claims he hasn't typed code by hand in two months. 259 PRs in 30 days
- Ryan Dahl, The creator of Node.js and Deno likewise said "the era of writing code is over"
- Charlie Marsh (Astral, Ruff, UV Ty etc): "I think it's been over a month since I made a code change that didn't go through Claude Code"
- Every other major AI startup (XAI with Grok, Google with Gemini, Openai with ChatGPT) all are using claude-code as well

- But to get hired - you must know more than the AI! Or else what's the point of hiring you instead of just prompting an AI?
- You must be able to direct AI based on your programming knowledge.
- Additionally, industry is not using ChatGPT like students are
 - They are using agentic models and tools like claude-code and/or cursor, generally in fairly specific ways
- Using ChatGPT isn't gaining you any employable skills imo, not even AI

- I'm trying to teach you the building blocks of programming
- This way, when you encounter problems that AI can not solve, you can succeed
- Additionally, many problems AI can solve with direction, and you'll be able to give this direction
- You'll see this in the course project at the end of this course
 - Last semester many students used AI and they had to rewrite large portions manually
 - Good programmers think AI can do everything
 - bad programmers using AI just produces garbage and can't get it to work
- Learn this course well and when you add AI afterwards you'll be golden!

- Therefore my recommendation is that you not use AI to complete assignments unless it's otherwise specified
- When you need help, personally I recommend using ChatGPT study mode
- Ask the AI to generate problems on the subject and walk you through them
- Then complete a problem from the AI without help
- Then try to complete the original problem
- This isn't a steadfast rule however, use your own judgment, it's just my recommendation
- I won't penalize people for AI, but imo you're just hurting yourself in the long run if you use it
 - Last semester, people got 100s on hws and straight zeroes on exams because they used AI as a crutch excessively
- Also don't forget, most interviews don't allow AI

Course Structure & Grading

- This will be the overall structure of the class in terms of grading/teaching:
- Lectures
- Homeworks (7%)
- Labs (7%)
- Quizzes (10%)
- Exams (60%)
- Course Project (16%)
- If you get an F on either the quizzes, the exams, or the course project, you fail the class.

- Lectures are twice a week, totally 2.5hrs/wk
- You can always follow along with the slides but I strongly recommend coming to class
 - Last semester the people who showed up did very well
 - Most who followed along on their own struggled
 - Generally you'll for the first little bit, giving a false sense of confidence...
 - Then we'll move too quickly and you won't be able to catch up
- When you miss class or want to improve your understanding, come to office hours of myself or TAs
 - These will be posted when I have them from TAs

Homeworks

- Homeworks will be worth 7%
 - This likely will equate to about half a point per homework
 - This however could change depending on how many we get to
- They'll be released on Fridays and due the following Friday. There will be a one week grace period.

- Labs will be worth 7%
 - This likely will equate to about half a point per lab
 - This however could change depending on how many we get to
- They'll be released on Mondays and due on Friday (the same time as the homeworks). There will also be a one week grace period for labs.
- Labs are designed to be short enough to complete in lab
- Lab participation is not required, but there will be quizzes and potentially exams in labs

Labs and Homeworks: Extensions

- Last semester I ended up spending often multiple hours a day dealing with one-off requests, extensions, etc
- Now I'm teaching multiple sections, this is infeasible and is a waste of time
- To solve this problem, I've given a one week grace period for every single assignment
 - This means you can submit a week late and receive no penalty
- This also fixes the problem of people signing up two weeks in
- These assignments were completable within a single week last term
- There will be no extensions beyond this already one week extension for any reason whatsoever
- Complete the assignments when they come out
- Get help on the assignments during office hours
- If you're late, it's a zero. No exceptions.

- Three exams
- 20% each
- Short answer and coding based
- These will likely be a combination of some short answer problems
- This will be followed by a few coding problems, open-ended
 - You will actually program for this
- They'll be taken with the lockdown browser
- If you get an F average on the exams, you fail the class
- 2/12, 3/12, and the final exam are the exam dates

Quizzes

- Likely one each week
- 10% total
- Open-ended coding problems
 - You will actually program for this
- They'll be taken with the lockdown browser
- If you get an F average on the quizzes, you fail the class
- If you get a 90+% on an exam, all quizzes will automatically be a 100% until the next exam
- I will drop your lowest quarter of quizzes - this is specifically meant for absences

Course Project

- Worth 16% of grade
- This will take place during the latter half of the semester
- You'll be building a large codebase to simulate the internet with C++
- AI will be allowed - but it will likely not help too much
- Relatively low amount of starter code - you'll code this from start to finish
- If you get an F on the course project you fail the class

- Discord for all communication
- Username: first.last
- Invite: <https://discord.gg/9PkX6GjtvC>
 - Check huskyct for the code
- Office hours: TBD
- Note that ALL announcements will come through Discord. If you don't sign up, you will miss them.

Cheating

- The department is cracking down on cheating now
- If you cheat on anything:
 - You will get a zero for the course
 - I will report you for it
- In this class, you're really only cheating yourself
- How well you learn this material will likely directly impact how much money you make as a software engineer

Use the Resources Available

- Ask for help often
- Lecture, Office hours, TAs
- Peer Tutoring Center
- Discord
- ChatGPT Study Mode

- Everyone is expected to have a laptop in all classes and labs
- Everyone is also expected to have a charger brought everywhere
- We'll get into how to configure your laptop later on, but for now just make sure you have one
- Additionally, if you are on a windows machine, before next class, you must have installed WSL2 (Windows subsystem for Linux 2)

Who Am I?

- Justin Furuness
 - Just call me Justin
- PhD in CS, BGP Security
- Undergrad, masters, PhD, all at UConn
- Worked at NeuralFrame and try to make this class applicable for industry
- Developed popular Python packages
- Code used worldwide and in many publications (NIST, others)
- Experience with PyBind11/C++ extensions
- Taught this class last term!

Questions?

- By the way, please just interrupt me any time there is a question

- There's a lot of glue that we'll add throughout this course in terms of tooling
 - The terminal, Git/GitHub, Virtual Environments, PyTest, and a few others
- This will allow us to be able to compile, run, and test our code
- But first, let's go over some C++ basics!

Hello World

- Go ahead to `clientide.com` and select C++
- We're going to start off with writing a hello world program in C++
 - AKA a program that is going to print hello world
 - A program is a list of instructions that the computer follows from top to bottom
- Go ahead and start by writing the code below

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- You can go ahead and run this code
- Congrats, you've completed your first C++ program!
- There's a lot going on here, but we don't know enough to be able to dig into most of it
- The most important part is that we can modify the lines inside of the curly braces to print and do different things!
- We're going to start there, and come back to this once we have a better understanding

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- We'll just go over this code at a very high level
- `#include<iostream>` allows us to use `cout` and `endl`
- The `main` function is where every program starts
 - Everything inside the curly braces gets run
 - Indentation is optional, curly braces are what really matters

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Hello World

- Everything to the right of `std::cout` is what gets printed
- `std::endl` adds a new line (i.e. goes to a newline when printing)
- Any text that gets printed needs to be in quotes (single or double)
 - Caveat: Numbers don't need quotes

Hello World Example

```
#include <iostream>

int main() {
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Questions?

- First things first, in C++ you can create comments!
- Comments are a nice way to let readers know what your code is doing
- They're just notes to yourself or future readers, they affect nothing
- Remember code is read far more often than it is written, so make sure to comment your code when it makes sense!
 - Typically comments are only necessary when they help clarify something that isn't clear
 - Even though a more experienced C++ developer probably wouldn't need them in this case, we'll add them anyways just to showcase them

Comments

- In C++ There are two ways to create comments: a double slash, or `/* */` for multiline comments
- Generally, comments go above what they are describing
- Rarely they can be on the side, if the comment is very short and it would be less clear above the line.

Hello World Example

```
/*  
This is a multiline comment  
About this piece of code  
*/  
  
#include <iostream>  
  
int main() {  
    // This line prints out Hello World!  
    std::cout << "Hello world!" << std::endl;  
    return 0; // success!  
}
```


Variable Declaration and Assignment

- In C++ we can also declare (aka create) a variable
- Variables are basically stored data that has a name
- C++ is strongly typed, so all variables must have types
- Let's take a look at an example for how to create a variable holding the number 100 and print it

Variable Example

```
#include <iostream>

int main() {
    int num_students; // Declaring the variable
    num_students = 100; // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- First we declare the variable `num_students`
- Any time that we declare a variable, we must include what type the variable is
- `num_students` has the type `int`, which stands for integer, which means this is a number
 - We'll dive deeper into types later on

Variable Example

```
#include <iostream>

int main() {
    int num_students; // Declaring the variable
    num_students = 100; // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- Directly below our declaration, we assign 100 to our variable
- Here the equals is the assignment operator, NOT equality
- The assignment operator is taking the value 100, and storing it in the variable named num_students
- The assignment operator *always* goes right to left

Variable Example

```
#include <iostream>

int main() {
    int num_students; // Declaring the variable
    num_students = 100; // Assigning to the variable
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- When we declare a variable, C++ gets space from our computer to store the variable
- The space it takes from your computer is in the computers memory, or RAM (Random Access Memory)
- The amount of memory that the computer requests is exactly the size of the variable it is storing
- If we declare an integer variable, C++ gets enough memory to store an integer
- Later C++ can set the integer in that space in memory to be whatever you wish

Variable Declaration and Assignment



Integer Space

```
int num_students;
```

C++ has reserved space in memory for an integer, but no value has been stored yet.

Variable Declaration and Assignment



```
num_students = 100;
```

The value 100 is now stored inside the memory space associated with `num_students`.

Variable Declaration and Assignment

- We can also both declare a variable, and assign a value to it, on the same line
- There is no difference between declaring and assigning a value to a variable on the same line versus separate lines
- Typically in C++ people want concise code, so they like to do both at once whenever possible

Variable Example

```
#include <iostream>

int main() {
    // Declaring a variable and assigning to it
    int num_students = 100;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

- Notice how at the end of each line there is a semicolon
- That's because this C++ program is comprised of many statements
- A statement in C++ is a command that instructs the program to perform a specific action
- We need to let the C++ compiler that runs your program know when a statement ends
- A statement doesn't end on a new line, it ends with a semicolon!

Semicolons

- Because statements end with a semicolon and not a new line...
- If we wanted, we could write many statements on one line!
 - Although that would certainly be a lot less clear

Variable Example

```
#include <iostream>
int main() {
    int num_students=100; std::cout<<"Students="<<
        num_students<<std::endl; return 0;
}
```

Variable Declaration and Assignment

- In C++ variables can change as your program runs
- In other words, variables can "vary"
- When we are changing a variable, we can't change its type, only the value of the variable
 - So an int variable must remain an int variable
 - Later on we'll go over special ways to convert between types, but let's leave this to the side for now
- Take a look at this program now, and you can see that it outputs 5, not 100!

Variable Declaration and Assignment

- Here we set the variable to be something different

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    num_students = 5;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Declaration and Assignment

- We can even make variables equal to other variables!

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    int other_students = 200;
    num_students = other_students;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Variable Naming Conventions

- In terms of what to call your variables in C++
- Variable names can contain letters, digits, and underscores ONLY
 - No whitespace or special characters
 - Digits are also an odd choice to put in variables
- Names must begin with a letter or an underscore
- Certain keywords that the compiler already has (such as `int`, `class`, etc) can't be used as variable names
- Generally, snake case works well
 - This is lowercase with underscores to separate words
 - Such as with `num_students`

- When you declare a variable, you must also declare the type of the variable
- There are LOTS of types in C++, but here are some of the more common ones:
 - int (for whole numbers)
 - double (for decimals)
 - bool (for true or false)
 - char (for characters)
- We'll dive deeper into all of these types later on
- For now, let's just practice creating some of them!

Variable Example

```
#include <iostream>

int main() {
    int num_students = 100;
    double grade_average = 98.6;
    // A character is a single character, "Hello" is many
    // characters
    char first_initial = 'j'; // Chars must use single
    // quotes!
    std::cout << "Average=" << grade_average << std::endl;
    return 0;
}
```

Types

- Boolean values can either be true or false, or 1 or 0
- C++ will automatically convert the 1 or 0 to a true/false
- In fact, any non-zero number converts to true
- They are always printed as 1 or 0 to be backwards compatible with C

Variable Example

```
#include <iostream>
int main() {
    bool pass_the_class = true;
    bool fail_the_class = false;
    bool you_guys_are_the_best = 1;
    fail_the_class = 0;
    std::cout << "Pass=" << pass_the_class << std::endl;
    return 0;
}
```


Questions?

Arithmetic Expressions

- We can also do all the things your calculator can do with numbers!
- We can add, subtract, multiply, divide, use parenthesis, use exponents, and more!

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 100;
    int total_python_students = 200;
    int total_students = total_cpp_students +
        total_python_students;
    std::cout << "Students=" << total_students << std::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also do all the things your calculator can do with numbers!
- We can add, subtract, multiply, divide, use parenthesis, and more!

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 100;
    int total_python_students = 200;
    int total_students = total_cpp_students +
        total_python_students;
    std::cout << "Students=" << total_students << std::endl;
    return 0;
}
```

Arithmetic Expressions

- Here's an example for how to subtract

Variable Example

```
#include <iostream>
int main() {
    int total_cpp_students = 300 - 200;
    std::cout << "Students=" << total_cpp_students << std::
        endl;
    return 0;
}
```

Arithmetic Expressions

- We can also multiply

Variable Example

```
#include <iostream>
int main() {
    int assignments_per_student = 5;
    int num_students = 100;
    int total_assignments = num_students *
        assignments_per_student;
    std::cout << "Assignments=" << total_assignments << std
        ::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also divide

Variable Example

```
#include <iostream>
int main() {
    int total_assignments = 500;
    int num_students = 100;
    int assignments_per_student = total_assignments /
        num_students;
    std::cout << "Assignments/Student=" <<
        assignments_per_student << std::endl;
    return 0;
}
```

Arithmetic Expressions

- We can also make more complicated math statements with parenthesis

Variable Example

```
#include <iostream>
int main() {
    int value = (5 * 4) / (2 * 5);
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

- The modulo operator is very useful
- This leaves us only with the remainder

Variable Example

```
#include <iostream>
int main() {
    int value = 10 % 4;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```


- One common use case for modulo is to determine if a number is even
- For example if a number divided by 2 has no remainder, it is even

Variable Example

```
#include <iostream>
int main() {
    int value = 10 % 2;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Questions?

Type Conversions

- Let's take a look at something slightly more complex
- What if you have two integers that don't divide evenly?

Variable Example

```
#include <iostream>
int main() {
    int numerator = 10;
    int denominator = 4;
    int value = numerator / denominator;
    std::cout << "Value=" << value << std::endl;
    return 0;
}
```

Type Conversions

- Here you have what's called integer division!
- Normally this would be 2.5
- But the value can't be 2.5 since that is not an integer!
- Instead, C++ gets rid of everything after the decimal
 - Often referred to as "truncating the decimal"
- Integer division occurs when both numerator and denominator are integers, or the final result is an integer

Variable Example

```
#include <iostream>
int main() {
    int number = 10;
    int denom = 4;
    int val = number / denom;
    std::cout << "Value=" << val << std::endl;
    return 0;
}
```

Type Conversions

- To fix this, we can convert *either* type to a double
- Our final result, the value, *must* also be a double

Variable Example

```
#include <iostream>
int main() {
    double n = 10;
    int d = 4;
    double v = n / d;
    std::cout << "Value=" << v << std::endl;
    return 0;
}
```

Variable Shortening

- You may also notice that I've been shortening the variable names as we went
- This is a very common practice in programming in general
 - But *definitely* in lower level languages like C++ or C
- Generally you want your variable names to be descriptive enough so that others can understand
 - n, d, v may be a bit too short in this case!
 - The program loses a bit of readability
 - However, in some places, single letter variables may improve readability!

Variable Shortening

Variable Example

```
#include <iostream>
int main() {
    double n = 10;
    int d = 4;
    double v = n / d;
    std::cout << "Value=" << v << std::endl;
    return 0;
}
```

Questions?

Assignment operators

- We've already been practicing with one assignment operator, the "=" sign
- C++ Gives us several others that we can use for shortcuts
- For example, let's take this very common pattern

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_signed_up = 20;
    // Student enrolls here!
    num_students = num_students + just_signed_up;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- Instead, we can shorten this with the "+=" assignment operator

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_signed_up = 20;
    // Student enrolls here!
    // num_students = num_students + just_signed_up;
    num_students += just_signed_up;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- We can do the same with -=

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int just_dropped = 20;
    // Student drops the class here!
    // num_students = num_students - just_dropped;
    num_students -= just_dropped;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- And `*=`

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int classes_merged = 2;
    // Classes merge here
    // num_students = num_students * classes_merged;
    num_students *= classes_merged;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Assignment operators

- And /=

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    int split_class = 2;
    // Classes split in half here
    // num_students = num_students / split_class;
    num_students /= split_class;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Questions?

Increment/Decrement

- Often times we are just adding or subtracting one from a given number
- C++ gives us a really easy way to do this
 - Even shorter than the "+=" and "-="

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    // Student enrolls here!
    // num_students = num_students + 1;
    // num_students += 1;
    num_students++;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Increment/Decrement

- And of course C++ provides the decrement shortcut for subtraction

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    // Student drops here!
    // num_students = num_students - 1;
    // num_students -= 1;
    num_students--;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```


Increment/Decrement

- Increment and decreemnt can be done inline
- If it is before, increment runs before the statement executes.
- Otherwise increment/decrement runs after the statement executes.
- This is an extremely common way to reduce lines of code.

Variable Example

```
#include <iostream>
int main() {
    int num_students = 100;
    std::cout << "Students=" << num_students++ << std::endl;
    std::cout << "Students=" << num_students << std::endl;
    num_students = 100;
    std::cout << "Students=" << ++num_students << std::endl;
    return 0;
}
```

Common Errors

- Forgotten semicolons
- Misspelling variable names
- Using variables before declaring them

Variable Example

```
#include <iostream>
int main() {
    num_students = 100
    // Student drops here!
    num_student--;
    std::cout << "Students=" << num_students << std::endl;
    return 0;
}
```

Questions?

In class exercises

- Write a hello world program

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

In class exercises

- Set a variable for total students equal to 100
- Set a variable for new students equal to 20
- Add the new students to the total students using an assignment operator
- Decrement total students by 1
- print the remainder of the total students when divided by 2
 - This will indicate if the total number of students are divisible by 2

In Class Exercises

```
#include <iostream>
int main() {
    int num_students = 100;
    int new_students = 20;
    num_students += new_students;
    num_students -= 1;
    std::cout << num_students % 2 << std::endl;
    return 0;
}
```

In class exercises

- Get the average grade given two scores of 90 and 93
 - To get the average: Add the scores together, then divide by the total number of scores

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    double average = (90 + 93) / 2.0;
    std::cout << average << std::endl;
    return 0;
}
```

In class exercises

- Store the first letter of your name as a character and print it

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    char first_initial = 'j';
    std::cout << first_initial << std::endl;
    return 0;
}
```

In class exercises

- Store store a boolean value and print it

In Class Exercises

In Class Exercises

```
#include <iostream>
int main() {
    bool passing = true;
    std::cout << passing << std::endl;
    return 0;
}
```

Expanding Our Memory Model

- Especially for C++ code (which often must be very fast), it's very important to understand why and how memory works in C++
- This will allow us in the future to optimize our code
 - Both for speed
 - And for memory

Expanding Our Memory Model

- Remember that your computer is merely a bunch of transistors
 - If you don't know what a transistor is, that's okay
- These transistors have current (electricity) flowing through them
- When the voltage on a transistor is low, it's represented as a zero
- When the voltage on a transistor is high, it's represented as a one
- From a high level, you can kind of think of your computer as a bunch of switches that are either on (represented as 1) or off (represented as 0)

Expanding Our Memory Model

- When you put a bunch of these ones and zeroes together, you get what's called binary code
- In other words, lines of code that are ones and zeroes, simply telling your computer which switches to turn on or off
- This is how all coding works under the hood
- When we click this run button here, our C++ code is being converted to ones and zeroes before being run

Expanding Our Memory Model

- So then how does memory work on a computer?
- Well, every variable gets stored as zeroes and ones
- The term for this is binary
- For example, numbers can be stored in binary as a binary number, as zeroes and ones
- So how do binary numbers work?

Expanding Our Memory Model

- Instead of counting in base 10 like we normally do, we can count in base 2
- Normally for base 10, every digit represents 10 to the next power
- 257 represents $10^2 * 2 + 10^1 * 5 + 10^0 * 7$ in base 10
 - Base10 is what we normally count in
- We can do the same but in binary

Expanding Our Memory Model

- Let's start with a type of `uint8_t`
- This is a type of C++ that sets aside 8 of these "switches"
- Each switch represents what we call a "bit" on our computer
- And your computer groups these bits together in groups of 8 called a "byte"
- A "byte" is just a group of 8 bits
- So for `uint8_t`, this is simply a number that takes up one byte, or 8 bits of space

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number 0, which in binary might be represented as 00000000

Bit Flipped	0	0	0	0	0	0	0	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	0	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 1, which in binary might be represented as 00000001

Bit Flipped	0	0	0	0	0	0	0	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	0	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 2, which in binary might be represented as 00000010

Bit Flipped	0	0	0	0	0	0	1	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	2	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 3, which in binary might be represented as 00000011

Bit Flipped	0	0	0	0	0	0	1	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	0	0	2	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

End of day 1

In class exercise

- How do you store 10 in binary as a `uint8_t`?
- How do you store 21 in binary as a `uint8_t`?

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 10, which in binary might be represented as 00001010

Bit Flipped	0	0	0	0	1	0	1	0
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	0	8	0	2	0

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Binary Bits and Their Values

- Computers store numbers using bits (0s and 1s)
- Each bit position is worth twice the one to its right
- For example, take the number, 21, which in binary might be represented as 00010101

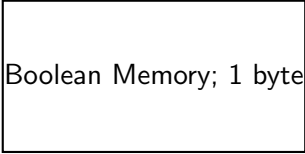
Bit Flipped	0	0	0	1	0	1	0	1
Bit Position	128	64	32	16	8	4	2	1
Bit Position	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bit Value	0	0	0	16	0	4	0	1

- Each position doubles as you move left
- Add the values of bits set to 1 to get the number

Expanding Our Memory Model

- Your computer has a limited amount of these bits
- Remember that these bits are grouped into 8 called bytes
- For example, your iphone might have 16 gigabytes
 - That's it! No more space!
- All of the types that we've discussed thus far are also represented in binary (ones and zeroes)
- For example, take a boolean type
- A boolean type for example, only needs a single bit, since it's either true (1) or false (0)
- However, because your computer always groups 8 bits together, you have a minimum of one byte of space for every type
- So one boolean gets one byte of space reserved for it

Expanding Our Memory Model



Boolean Memory; 1 byte

```
bool passing;
```

When this line of code runs, C++ reserves space in memory for exactly one byte for this boolean value, but no value has been stored yet. While the boolean value needs only a single bit, it must occupy the entire byte of space

Expanding Our Memory Model

- Likewise, letters, stored as characters (the `char` type), are also represented in binary
- These have one byte associated to them
- Each binary number gets converted to a letter
- This conversion system is known as ASCII
- For example, a capital 'A' is stored as the number 65
- A lowercase 'a' is stored as the number 97

Expanding Our Memory Model

- Integers can also be stored in this way
- Integers are represented in binary (1s and 0s)
- In C++ integers are 32 bits, (32 1s and 0s)
 - How many bytes is that?
- So how do we go from 32 bits to an actual number?

Expanding Our Memory Model

- 32 bits are simply four bytes back to back
- The way that memory works is that you **MUST** have these bytes placed next to each other
- You can skip memory blocks
- This is called "contiguous memory"

Expanding Our Memory Model

- Intuitively, you may be thinking that 32 bits for the number one can be stored as:
 - |0000 0000 |0000 0000 |0000 0000 |0000 0001 |
 - (here each pipe is representing a block of space)
- And it can... on big endian systems
- But typically most computers actually arrange the bytes in reverse order
- This is called little endian
- So the number one would be stored as:
 - |0000 0001 |0000 0000 |0000 0000 |0000 0000 |
- While I cover it here just so we are aware of how it works, most of the time you don't need to think about endianness unless you're working with raw bytes, files, or networks.

Binary Bits and Their Values

- Of course, for a 32 bit number, the maximum size that the number can be is $2^{32} - 1$
- This is when all the "switches" are turned on
- This is when all the bits are set to 1
- Every type has a maximum and a minimum size
 - We'll dig into the implications of this in a moment

Binary Bits and Their Values

- If we say that the integer is "signed", that means that the first bit represents the sign
- 0 For positive
- 1 for negative
- This can get a bit complicated as to how these binary numbers are represented, so we won't go into the binary for this since it's not really necessary to know
- It's just important to know that for signed numbers, the maximum size has one bit less
- So for an integer, the maximum size for a "signed" integer (the default) is $2^{31} - 1$
 - This is in contrast to an unsigned integer where the default max size is $2^{32} - 1$

Questions?

Expanding Our Memory Model

- Integers take up 4 bytes, aka 32 bits, on most machines
- That means when you declare and create an integer, your computer is reserving 4 bytes or 32 bits of space for that integer
- In C++ since an integer is a "signed" type, the first bit is reserved for indicating whether a number is positive or negative (for example, negative 4, or positive 4)
- That means the maximum size of that integer is actually the number when the remaining 31 bits are set to 1
- In other words, the maximum size of a signed integer (keyword: `int`) is 2,147,483,647
 - 2,147,483,647 in binary is 01111111111111111111111111111111

Expanding Our Memory Model

```
int num_students (4 bytes) = 2147483647
```

```
int num_students=2147483647;
```

When this line of code runs, C++ reserves **4 bytes (32 bits)** of memory to store the integer `num_students`, and we set the value to be 2147483647 (the maximum integer value).

Overflow Error

- So then what happens when we add one to the maximum value an integer can be?
- The number can not grow to 33 bits...
- So it "overflows" and actually wraps around!
- Think: Just like what happens when you add one hour to a twelve hour clock!
 - Adding an hour to 6PM, no problem. 7PM, no problem. All the way to 11PM, no problem. But if you try to add one hour at 12PM, you wrap back around to 1AM!
 - The same is true for numbers in C++!
- When we add 1 to the most positive integer, 2147483647:
 - It wraps around to the most negative integer, -2147483648!
- This is called an overflow error

Overflow Error

- In C++ however, overflows aren't always that nice
- Sometimes, overflow errors result in undefined behavior
- That means your program can start doing wacky stuff that doesn't make any sense
- It's also undefined, meaning different runs it may do different things
- It can be very hard to debug
- On most machines you'll see wraparound, but the standard does not guarantee this
 - The compiler is allowed to do anything

Overflow Error

- Let's take a look at this in our code!
- We can cause an overflow error to happen!
- Now the printed value is actually wrapping around and is the most negative integer

Example

```
#include <iostream>
int main() {
    int num = 2147483647;
    std::cout << num + 2 << std::endl;
    return 0;
}
```

Overflow Error

- To avoid this, it's really important to ensure that all numbers fit within their specified range for that specific type
- There are many numeric types just for these situations
- Also see these unsigned types, which assume that the number is always positive
 - Then all bits are used for the number itself (there are no sign bits)

Type	Size	Minimum Value	Maximum Value
bool	1 byte	0	1
char	1 byte	-128	127
unsigned char	1 byte	0	255
short	2 bytes	-32,768	32,767
unsigned short	2 bytes	0	65,535
int	4 bytes	-2,147,483,648	2,147,483,647
unsigned int	4 bytes	0	4,294,967,295
long long	8 bytes	-9.22×10^{18}	9.22×10^{18}
unsigned long long	8 bytes	0	1.84×10^{19}

- Here's a similar table for dealing with what's called **floating point** numbers
 - In other words, numbers with a decimal point
- Floating point values can also overflow or underflow
- These types sacrifice precision for a larger range
- They may also suffer from **rounding errors**...
 - We'll come back to this later

Type	Size	Minimum Value	Maximum Value
float	4 bytes	$\approx 1.18 \times 10^{-38}$	$\approx 3.40 \times 10^{38}$
double	8 bytes	$\approx 2.23 \times 10^{-308}$	$\approx 1.80 \times 10^{308}$
long double	16 bytes	$\approx 3.36 \times 10^{-4932}$	$\approx 1.18 \times 10^{4932}$

Overflow Error

- Absolutely no need to memorize these tables
- Just make sure to understand these concepts:
 - Overflow
 - Signed vs unsigned
 - Code is converted to binary which tells your computer what to do
 - Numbers are stored in binary and take up specific sizes in bytes
 - C++ does not reserve less size than a byte of space in memory
 - So even a boolean (1 bit) takes up an entire byte of memory
 - Each byte is 8 bits
 - Each bit represents a single switch on your computer

Questions?

Arrays

- What about when you want to be able to store many items at once?
- In other words, what if you want to have 20 student scores or 1000?
- It's a bit silly if we have to store separate variables
- Instead, we can just put them in what's called an array
- An array is a collection of values of the same type
- All of the values are stored right next to each other in memory
- Each value in the array can be accessed using a number called an index

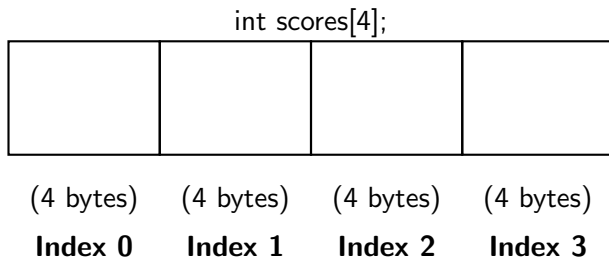
Arrays

- You can create an array like so:
- Instead of `int scores` -> `int scores[4]`
- That tells C++ to grab four blocks of memory for integers
- Remember each integer is typically 4 bytes
- In total that means this array will take 16 bytes of space

Example

```
#include <iostream>
int main() {
    int scores[4];
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Arrays



- When we want to set an item at a specific block in this array, we can specify the index and do so with the following:

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Arrays

```
int scores[4];  
scores[0] = 90;  
scores[1] = 80;
```

90	80		
-----------	-----------	--	--

(4 bytes)

(4 bytes)

(4 bytes)

(4 bytes)

Index 0

Index 1

Index 2

Index 3

Arrays

- We can also get the value at an index by using the same bracket syntax
- Example below

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    int second_value = scores[1];
    std::cout << scores[0] << std::endl;
    return 0;
}
```

Arrays

- But what happens if we print one of the array slots that has not been filled yet?
- C++ will just read whatever 1s and 0s the computer already has there and try to turn it into an integer!
- This will result in undefined behavior and errors!

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[0] = 90;
    scores[1] = 80;
    std::cout << scores[2] << std::endl;
    return 0;
}
```

Example

```
#include <iostream>
int main() {
    int scores[4];
    scores[5] = 80;
    std::cout << scores[4] << std::endl;
    return 0;
}
```

- And what if we store a value at index 5? Even though there is no index 5 in the array?
- We will write to memory we do not own!
- This will write memory to outside of the array, clobbering other parts of the program while the program is running
 - It could potentially overwrite something very important!
 - Resulting in memory corruption, potentially leading to undefined behavior!
 - Not good!
- Unlike other programs like Python, C++ will often let this happen!
 - You must catch these bugs (errors) yourself! Be careful!

Arrays

- You can also set an array using curly braces at the time the array is created
- You can't declare the array, and later on set it using curly braces - it all must be done at once

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    std::cout << scores[1] << std::endl;
    return 0;
}
```

- When you use curly braces to initialize your array, whatever isn't specified is defaulted to zero

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80}; // All other indexes are now
        zeroes!
    int other_scores[5] = {}; // Everything is zero!
    std::cout << scores[1] << std::endl;
    return 0;
}
```


Arrays

- Arrays also have a fixed size
- They are not dynamic
- The code must know this size in advance, it can't be a number that can change
- Additionally, once you've made the array, you can not later change it's size

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    std::cout << scores[1] << std::endl;
    return 0;
}
```

Arrays

- Another quirk of arrays: They do not keep track of their size!
- You as the programmer must keep track
- One "hack" to do this: Take the size in memory of the array, and divide it by the size in memory for a single block of that array
- You'll often see this used by C++/C programmers

Example

```
#include <iostream>
int main() {
    int scores[5] = {90, 80, 70, 60, 100};
    int size = sizeof(scores) / sizeof(scores[0]);
    int also_size = sizeof(scores) / sizeof(int); // Less
        dynamic
    std::cout << size << std::endl;
    return 0;
}
```

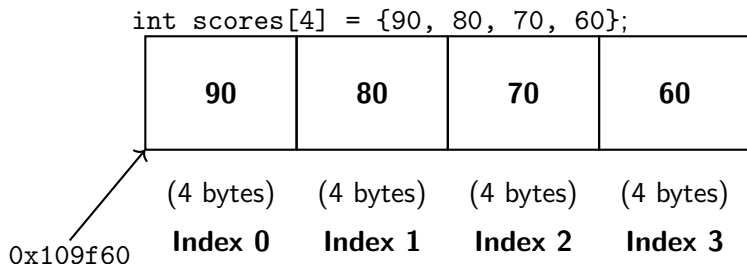
Arrays

- Lastly, you can print out the memory address of an array!
 - In other words, we can print out the exact location in memory where the array begins
- An array's address is the same as the address of its first element
- On most modern systems, memory addresses are 64-bit numbers

Example

```
#include <iostream>
int main() {
    int scores[4] = {90, 80, 70, 60};
    std::cout << scores << std::endl;
    return 0;
}
```

Arrays



- An array of 4 integers reserves **4 contiguous blocks of memory**, with each block taking **4 bytes**.
- We can get the beginning of the array, the address, by printing the array
- We can determine the size of the array with `sizeof(scores) / sizeof(scores[0])`

Arrays

- We can create an array of any type, not sure integers
- Let's create an array of booleans for example

Example

```
#include <iostream>
int main() {
    bool has_drivers_license[5] = {true, false, true, true,
                                   false};
    std::cout << has_drivers_license[1] << std::endl;
    return 0;
}
```

Questions?

In Class Exercise

- Create an array to store grades of 90.5, 89.6, 98.7 on one line
- Change the 89.6 to a 97 after the initial array creation
- Print the first grade in the array

In Class Exercise

- Create an array to store grades of 90.5, 89.6, 98.7 on one line
- Change the 89.6 to a 97 after the initial array creation
- Print the first grade in the array

Example

```
#include <iostream>
int main() {
    double grades[3] = {90.5, 89.6, 98.7};
    grades[1] = 97;
    std::cout << grades[1] << std::endl;
    return 0;
}
```


Character Arrays and C style strings

- Recall our hello world program
- Remember that we can print hello world using double quotes
- This is a C-style string, also in this case it is a string literal
- But what is the type of this "Hello World" C-style string?
 - And how can we store this in a variable?

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

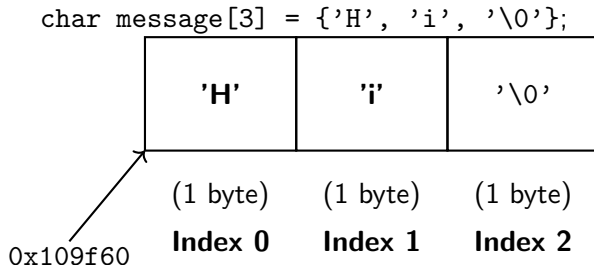
Character Arrays and C style strings

- "Hello World", under the hood, is actually stored as a character array!
- At the end of this character array is what's known as a null byte, i.e. the character `'\0'`
- The null byte lets C++ know:
 - "We've reached the end of our array, stop printing"
- The null byte is represented as a backslash zero, it's a special character
- Let's dive into why we need a null byte and expand on our memory model a bit more

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    std::cout << message << std::endl;
    return 0;
}
```

Arrays



- An array of 3 characters reserved **3 contiguous blocks of memory**, with each block taking **1 byte**.
- We can get the beginning of the array, the address, by printing the array
- We can determine the size of the array with `sizeof(message) / sizeof(message[0])`
- When printing, C++ will start at the beginning address of the array, and proceed to the final null byte

Character Arrays and C style strings

- Recall that C++ doesn't store the size of the array
- C++ also does store the beginning address of the array
- When printing, C++ goes block by block, starting at the beginning, until it reaches the null byte
- The null byte is how C++ knows it's reached the end
- The null byte in the character array is specifically what turns the character array into a C-style string that can be printed
- We can move the null byte further up and the printing will stop there

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', '\0', 'l', 'l', 'o'};
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C style strings

- If we don't have a null byte, this results in undefined behavior!
- C++ may keep reading and printing from memory outside of the array!
- It may continue reading and printing until it finds a random null byte on your computer
 - Or do something else entirely! It's undefined!

Example

```
#include <iostream>
int main() {
    char message[6] = {'H', 'e', 'l', 'l', 'o', 'o'};
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- When we write "Hello", C++ treats it as a **string literal**
- A string literal is stored as a **character array** ending with a null byte (`'\0'`)
- When we assign it to a character array, all characters *including the null byte* are copied

Example

```
#include <iostream>
int main() {
    char message[6] = "Hello";
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- We do **not** need to manually specify the size of a character array
- If we leave the size empty, the compiler will figure it out for us
- The compiler automatically:
 - Counts the characters in the string literal
 - Adds space for the null byte (`'\0'`)
 - Allocates the correct amount of memory

Example

```
#include <iostream>
int main() {
    char message[] = "Hello";
    std::cout << message << std::endl;
    return 0;
}
```

Character Arrays and C-Style Strings

- And of course, just like in a normal array, we can access any index
- We can print any index, or change any index

Example

```
#include <iostream>
int main() {
    char message[] = "Hello";
    message[0] = 'J'; // Recall: characters require single
                       quotes!
    std::cout << message << std::endl;
    return 0;
}
```


Questions?

In Class Exercise

- Store your name in a character array
- Change the first initial of your name to be a T
- Print your name

In Class Exercise

- Store your name in a character array
- Change the first initial of your name to be a T
- Print your name

Example

```
#include <iostream>
int main() {
    char name[] = "Justin Furuness";
    name[0] = 'T';
    std::cout << name << std::endl;
    return 0;
}
```

In Class Exercise

- Create a C-Style string "Hello" using curly braces in a variable called msg
- Print msg

In Class Exercise

- Create a C-Style string "Hello World" using curly braces in a variable called msg
- Print msg

Example

```
#include <iostream>
int main() {
    char msg[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    std::cout << msg << std::endl;
    return 0;
}
```

Output with std::cout

- The next part of our program that we need to understand is `std::cout`
- `cout` stands for **character output**
- It represents a stream of data that gets printed to the terminal
- Anything sent to `std::cout` will appear on the screen

Example

```
#include <iostream>
int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

Output with std::cout

- The << operator is called the **stream insertion operator**
- It sends data into the output stream
- You can think of it as:
 - "Send this value to the screen"
- The data flows from right to left into std::cout

Example

```
std::cout << "Hello World";
```

Output with std::cout

- We can send *multiple* values to std::cout in one line
- Each value is sent using another <<
- The output happens in order, from left to right

Example

```
#include <iostream>
int main() {
    int students = 100;
    std::cout << "Students: " << students << std::endl;
    return 0;
}
```


Output with `std::cout`

- Notice how we can mix text and variables
- Strings must be in quotes
- Variables are written without quotes
- C++ automatically figures out how to print each type

Example

```
std::cout << "Answer = " << 42 << std::endl;
```

Output with `std::cout`

- `std::endl` means **end line**
- It does two things:
 - Moves the cursor to a new line
 - Flushes the output buffer
- For now, just think of it as “go to the next line”

Example

```
std::cout << "Hello" << std::endl;  
std::cout << "World" << std::endl;
```

Output with std::cout

- We can print many different types using std::cout
- Including:
 - Integers
 - Doubles
 - Characters
 - Booleans
 - C-style strings

Example

```
#include <iostream>
int main() {
    int i = 10;
    double d = 3.14;
    char c = 'A';
    bool b = true;

    std::cout << i << " " << d << " " << c << " " << b <<
        std::endl;
    return 0;
}
```

Common std::cout Mistakes

- Forgetting << between values
- Forgetting quotes around text
- Using single quotes for strings
- Misspelling std::cout

Incorrect Example

```
std::cout "Hello World";
```

Questions?

In Class Exercise

- Print the following on one line:
 - The text "Total Students: "
 - The number 250
- Then print a new line

In Class Exercise

Solution

```
#include <iostream>
int main() {
    std::cout << "Total Students: " << 250 << std::endl;
    return 0;
}
```

Functions

- So far, all of our code has lived inside `main`
- As programs grow, this quickly becomes hard to manage
- We need a way to:
 - Break code into smaller pieces
 - Reuse logic
 - Make programs easier to read and reason about
- This is where **functions** come in

Functions

- A function is a named block of code
- It performs a specific task
- We can:
 - Define a function once
 - Call it as many times as we want
- `main` is actually just a function!

- Let's look at the basic structure of a function
- Every function has:
 - A return type
 - A name
 - A list of parameters (possibly empty)
 - A body (inside curly braces)

General Function Form

```
return_type function_name(parameter_type parameter_name) {  
    // function body  
}
```

Functions

- Let's start with a very simple function
- This function takes no inputs and returns nothing
- The keyword `void` means “returns nothing”

Example

```
#include <iostream>

void say_hello() {
    std::cout << "Hello!" << std::endl;
}

int main() {
    say_hello();
    return 0;
}
```

- When we **call** a function:
 - The program jumps to the function
 - Runs the code inside it
 - Then returns back to where it was called
- Functions do *not* run unless they are called

Functions

- Functions can also take inputs
- These inputs are called **parameters**
- Parameters are variables local to the function

Example

```
#include <iostream>

void print_number(int x) {
    std::cout << "!" << x << "!" << std::endl;
}

int main() {
    print_number(42);
    return 0;
}
```

- Here, `x` receives a **copy** of the value passed in
- This is called **pass by value**
- The function gets its own copy
- Changing `x` does *not* change the original value

Functions (Pass by Value)

Example

```
#include <iostream>

void change_value(int x) {
    x = 100;
}

int main() {
    int num = 5;
    change_value(num);
    std::cout << num << std::endl;
    return 0;
}
```

- Output is still 5
- The function only modified its own copy

Functions

- Functions can also return values
- The return type specifies what kind of value is returned
- We use the return keyword to send a value back

Example

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    std::cout << result << std::endl;
    return 0;
}
```

Functions

- When a function returns:
 - The function immediately stops running
 - The returned value replaces the function call
- Think of it as:
 - `add(3, 4)` becomes 7

Functions

- Functions can take multiple parameters
- Each parameter must have a type
- Parameters are separated by commas

Example

```
double average(int a, int b) {  
    return (a + b) / 2.0;  
}
```

Functions

- Functions must be defined **before** they are used
- Otherwise, the compiler doesn't know they exist yet
- (Later we'll learn how to fix this with declarations)

Correct Order

```
int add(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    std::cout << add(2, 3) << std::endl;  
}
```

- Functions help us:
 - Avoid repeating code
 - Give names to ideas
 - Make programs easier to debug
- If you copy and paste code more than once:
 - It probably should be a function

Questions?

Quiz

End of day 2

In Class Exercise

- Write a function called `square`
- It takes an integer as input
- It returns the square of that number
- Call it from `main` and print the result

Example Solution

```
#include <iostream>

int square(int x) {
    return x * x;
}

int main() {
    std::cout << square(5) << std::endl;
    return 0;
}
```

In Class Exercise

- Write a function called `divide`
- It takes two doubles as input
- It returns one number divided by the other
- Call it from `main` and print the result

Example Solution

```
#include <iostream>

double divide(double a, double b) {
    return a / b;
}

int main() {
    std::cout << divide(10, 2) << std::endl;
    return 0;
}
```

- Sometimes, we want a function to have a “default” value for a parameter
- This allows the function to be called with fewer arguments
- These are called **default arguments**

Default Arguments

- A default argument is a value that is used if no argument is provided
- Default arguments are specified in the function definition
- They use the assignment operator =

Example

```
#include <iostream>

void num_students_in_class(int students = 0) {
    std::cout << "There are " << students << "in this class"
    << std::endl;
}

int main() {
    num_students_in_class();
    num_students_in_class(100);
    return 0;
}
```

Default Arguments

- If we call the function without an argument:
 - The default value is used
- If we provide an argument:
 - The default value is ignored
- The function body does not change

Default Arguments

- Default arguments must come **after** non-default arguments
- This is because C++ matches arguments from left to right

Valid

```
int add(int a, int b = 5);
```

Invalid

```
int add(int a = 5, int b); // ERROR
```


Default Arguments

- You can have multiple default arguments
- But once one parameter has a default:
 - All parameters to the right must also have defaults

Example

```
int compute(int a, int b = 10, int c = 20);
```

- Now let's talk about **scope**
- Scope determines:
 - Where a variable exists
 - Where it can be accessed
- When a variable goes out of scope:
 - It no longer exists

Scope

- The most common scope we've seen so far is **function scope**
- Variables declared inside a function:
 - Only exist inside that function

Example

```
#include <iostream>

void my_function() {
    int x = 10;
    std::cout << x << std::endl;
}

int main() {
    my_function();
    // std::cout << x << std::endl; // ERROR
    return 0;
}
```

- Parameters are also scoped to the function
- They only exist inside the function body

Example

```
void print_value(int value) {  
    std::cout << value << std::endl;  
}  
  
// value does NOT exist here
```

- Scope is created by **curly braces** `{}`
- This is not limited to functions
- Any pair of braces creates a new scope

Block Scope

- A block is any code wrapped in {}
- Variables declared inside a block:
 - Only exist inside that block

Example

```
#include <iostream>

int main() {
    {
        int x = 42;
        std::cout << x << std::endl;
    }

    // std::cout << x << std::endl; // ERROR
    return 0;
}
```

Scope Shadowing

- You can declare a variable with the same name in an inner scope
- This is called **shadowing**
- The inner variable temporarily hides the outer one

Example

```
#include <iostream>

int main() {
    int value = 10;

    {
        int value = 20;
        std::cout << value << std::endl;
    }

    std::cout << value << std::endl;
    return 0;
}
```

- Scope helps prevent:
 - Name collisions
 - Accidental misuse of variables
- It also helps the compiler:
 - Know when memory can be reclaimed

Common Scope Errors

- Using a variable outside of its scope
- Assuming variables live forever
- Reusing variable names unintentionally

Questions?

Hello World Program Revisited

- At this point, we know almost all parts of our hello world program
- We understand how functions work, and how the main function works
- We understand character arrays and C-style strings for "Hello World"
- We understand how printing to the character output works and how to add a new line
- We understand scope
- But there's still a few missing pieces here...
 - What is `std::?`
 - What does including `iostream` do?

Local Development

- To be able to successfully answer these questions, we need to learn how to develop locally
- This is extremely critical - unless we can develop on our own computer, we will have a tough time distributing our code
- Additionally, as a C++ developer, we want to be able to write and run code on any server, not just our computer
- To do this, we will not only code on our computer, we will do so in the terminal
 - And we'll edit our code using a terminal editor, just like in those hacker movies
- Lack of command line/terminal knowledge will be a big red flag when you go to get hired, so we're going to do all development in the terminal from here on in to build these skills

Local Development

- To reiterate: using the terminal is a requirement of this course
- Not using one will hurt you when it comes to exams
- The course project will have strange permission issues if you aren't using the terminal
- We will use the terminal in conjunction with git, GitHub, PyTest, etc
 - If you don't use the terminal for these, your grades will suffer
- Some students last semester found wild workarounds for this, like using the pseudo terminal in VS code
 - This is not the terminal
 - They could not complete the later assignments
- If you can't figure this stuff out, you have to come and talk to me or the TAs and get help
- And just to repeat: It'd be tough to get hired without this terminal knowledge

- Your computer consists of a bunch of hardware
 - Your keyboard
 - Your speakers
 - Your screen
 - etc
- At a high level, your operating system is the code that lets you interact with the hardware

- For windows users, Windows is the operating system that is interacting with the hardware
- How many people are currently using windows (non-apple, non-linux)?
- I bring this up just to let you know, that is a bit of a red flag when applying to jobs
 - Why?

- Well, there is another type of operating system that is not windows called Linux
- Macs (from apple), Ubuntu computer, both run Linux-based operating systems
- Here are some other systems that also run Linux-based operating systems:
 - Iphone
 - Android
 - Mac
 - Ubuntu
 - Servers (web, databases, cloud)
 - Cars
 - General consumer electronics
 - And so much more... pretty much all linux
 - Except for Windows

- When you run windows, it's sort of an admission that you aren't routinely working on any of the above, not working on databases, self-hosting websites, etc
 - The skills to run these things on windows are NOT transferable to running them on Linux, they are totally separate
- There are three ways to fix this:

- Option 1 (recommended for most): Use WSL2 (windows subsystem for linux)
 - This is good enough for beginners
 - It "emulates" Linux
 - Still, this will only last you your first few years, since WSL2 can't do everything Linux can do, and is a lot slower
 - Definitely the best place to start for beginners if you're stuck with a windows computer

- Option 2: Get a Mac (from apple) when you buy a new computer instead of Windows
 - macOS is Unix-based and behaves very similarly to Linux for development
 - Used m1 macs are quite fast and typically go for around 300 bucks
 - A brand new mac mini is 500 bucks with the student discount

- Option 3: Alternatively, you can dual-boot your computer to run both Ubuntu and Windows at the same time
 - This is very complicated though, and I don't recommend it anymore
 - Windows is like a virus that really doesn't want you running anything else
 - If you screw up, you "brick" your computer
 - That means your computer is only good for use as a brick because you have destroyed it so badly
 - If you do want to do this, my recommendation would be a Lenovo computer, as their drivers are usually very compatible, but any computer will do
 - I will also say, I don't personally know any serious developers that aren't running a linux distro like Ubuntu.
 - Long term, the people programming every day tend to end up here
 - You want maximum options and freedom
 - You want your computer to exactly match how programs are deployed on servers

- For mac users, if you go to settings, and then privacy and security, you want to enable your terminal to be able to run anything
- This will actually dramatically increase compilation times

The terminal

- I'll go over a few of the basics with you guys now
 - pwd (current dir)
 - ls (print files)
 - cd (change directory)
 - .. (up)
 - - (previous)
 - / (home)
 - mkdir (make directory)
 - rm (delete file)
 - rm -rf (delete dir)
 - cp (copy)
 - mv (move)

- Continued
 - Up arrow (Bring back a prior command)
 - Tab (Complete a command)
 - ctrl + r (searching history)
 - ctrl c (kill whatever is running)
 - clear (clear the screen)

The terminal

- And many more! We don't have time to over them all in this course, but there are great tutorials online. I learned through <https://overthewire.org/wargames/bandit/>
- It will be a bit of a red flag if you want to be a systems programmer, but aren't good with the terminal, so practice!
- And beyond this... ask AI. Doesn't make sense to memorize syntax for anything beyond the basics, and for the average programmer, this is about it.
- To me these are all of the commands I typically use, and anything else I tend to look up using AI.
- There are some other commands I also use commonly, but I don't want to flood you guys all at once so we'll go over part 2 later on

Questions?

- Vim/Neovim and Emacs are the two most dominant terminal editors
- Terminal editors are extremely useful when you need to work on a server (configuring an IDE to work on a server through an SSH tunnel is not easy or practical in many cases)
- Like all the other terminal skills, a terminal editor is a good tool to have in your toolbox, even if you don't use it daily
- Especially when it comes to high performance applications, there are many cases where you'll consistently be working in environments where a terminal editor will be the optimal solution.
- Linus Torvalds (Linux creator), Ken Thompson (co-creator of Unix, Go, B, etc), Richard Stallman (founder of GNU), Dennis Ritchie (co-creator of C, Unix) all used terminal editors
- I won't force you to use one (you can use an IDE like VS code), but you'll be missing out on a valuable skill, and vim motions can be transferred to IDEs as well

- Run `sudo apt-get update && sudo apt-get install vim` to install vim on WSL or Ubuntu
- For a mac, install homebrew (go to brew.sh) and then run: `brew install vim`
- Vim has four modes, we'll just go over the basics of three:
 - normal mode
 - insert mode
 - command mode

Vim: Insert mode

- Pressing the i key puts you in insert mode
- You can type anything you want like a normal editor
- press escape to leave insert mode and go back to normal mode

Vim: Normal mode

- Normal mode is the default mode
- You can move your cursor around with the arrow keys, or hjkl, but not the mouse
- There are vim commands to help you jump around faster
- ex: w jumps forward by a word, and b jumps backwards
- If you want to learn all the shortcuts, there is a good tutorial by ThePrimeagen:
<https://www.youtube.com/watch?v=X6AR2RMB5tE>.
- The shortcuts to move around can also be transferred to any IDE for faster movement
- To start, this is honestly good enough

Vim: Command mode

- If you are in normal mode and press the colon, you enter command mode
- You can run commands here, but really simply:
- `escape + colon + w + q` to save and quit (or omit the `q` to just save)
- That's it for now!

vimrc example

- `~/.vimrc` is a config file that you can add which will make vim more powerful
- You can customize vim pretty much any way you'd like with this config (many examples online)
 - For example, a tab becomes four spaces
 - When you copy a line, it pulls it into your clipboard
 - Turn those annoying bell sounds off
- I've posted a relatively simple one I use personally in huskycat
- Remember, this goes in your `~/.vimrc` location.
- It's important to have a vimrc to convert tabs to spaces.
 - Spaces have the same spacing in every editor whereas tabs display differently
 - Most autoformatter tools convert everything spaces, and many languages even require spaces now
 - With a vimrc, tabs get autoconverted to spaces so you get the best of both worlds

- To clarify, take the posted example:
- Run `vim ~/.vimrc`
- Press `i` to enter insert mode
- Copy paste my config (or another config you find from google/AI) into that file
- Press escape, the colon character, `w`, `q`, enter, to save and quit

Questions?

Compiling and Running our First C++ Program

- Now that we understand the basics of C++, we can compile and run programs locally
- C++ is a **compiled language**
- This means our source code must be translated into machine code *before* it can run
- This translation is done by a **compiler toolchain**

Installing the C++ Toolchain (Linux / WSL / Mac)

- First, make sure your system is up to date for WSL/Ubuntu:
 - `sudo apt-get update`
 - `sudo apt-get upgrade`
- For a mac, instead run: `brew install gcc`
 - This will also update this to the latest version of gcc, which we need.

Installing the C++ Toolchain (Linux / WSL / Mac)

- This installs:
 - The C++ compiler (usually g++)
 - The linker
 - The C++ standard library

- C++ source files typically end in `.cpp`
- Example:
 - `hello_world.cpp`
- Header files (covered later) typically end in:
 - `.h` or `.hpp`

Compiling a C++ Program

Compilation Command

```
c++ hello_world.cpp -o program.out -std=c++23
```

- c++ is a **compiler driver**
- On most Linux systems, this invokes g++
- Some systems may be configured to use clang++

Breaking Down the Compilation Command

- `hello_world.cpp`
 - The source file
- `-o program.out`
 - Names the output executable
 - If omitted, the default output is `a.out`
- `-std=c++23`
 - Specifies the C++ language standard

Running an Executable

Run Command

```
./program.out
```

- . means the current directory
- Linux does **not** search the current directory by default
- The slash specifies the exact file to execute

What Is an Executable?

- An executable is a binary file containing machine instructions
- It is produced by the **linker**
- It must:
 - Contain valid machine code
 - Have execute permissions

Inspecting Files

```
file program.out  
ls -l program.out
```

Compilation Pipeline Overview

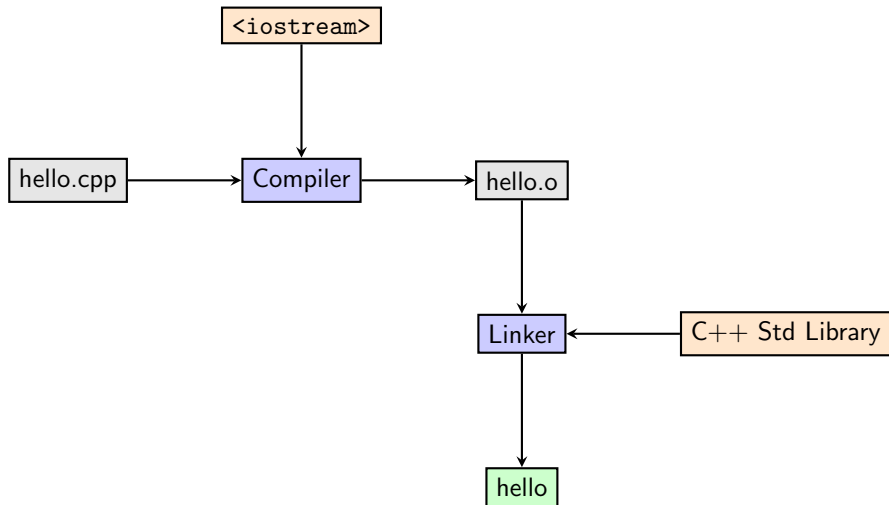
- C++ compilation happens in multiple stages:
 - 1 Preprocessing
 - 2 Compilation
 - 3 Linking
- The `c++` command usually performs all stages automatically

- Happens *before* compilation
- Handles:
 - `#include`
 - `#define`
 - Conditional compilation
- `#include <iostream>` literally copies text
- It is **not** a library import

- The compiler:
 - Checks syntax
 - Checks types
 - Translates code into machine instructions
- Output: an **object file** (.o)
- Object files are *not executable*

- The linker:
 - Combines object files
 - Resolves function and variable references
 - Links the C++ standard library
- Output: the final executable

C++ Compilation Pipeline (Visual)



Separating Compile and Link Steps

Compile Only

```
c++ -c hello.cpp -o hello.o -std=c++23
```

Link

```
c++ hello.o -o hello
```

- Useful for large projects
- Only recompile files that changed

Three Kinds of Errors

- **Compile-time errors**

- Syntax errors
- Type errors

- **Link-time errors**

- Undefined references

- **Run-time errors**

- Crashes
- Undefined behavior

Debug vs Release Builds

Debug Build (Always Use for Development)

```
c++ hello.cpp -g -O0 -std=c++23
```

Release Build

```
c++ hello.cpp -O3 -std=c++23
```

- Debug builds are slower but debuggable
- Release builds are fast but hard to debug

Sanitizers (Extremely Important)

- Sanitizers detect bugs *while your program runs*
- They catch:
 - Out-of-bounds array access
 - Use-after-free
 - Integer overflow
 - Undefined behavior

Recommended Student Build

```
c++ hello.cpp -g -O0 -fsanitize=address,undefined -std=c++23
```

Important Optimization Warning

- Optimizers assume undefined behavior never happens
- They may:
 - Remove code
 - Reorder instructions
 - Produce surprising results
- Never debug optimized code
- Always debug with:
 - `-O0 -g -fsanitize`

- C++ programs must be compiled before running
- Compilation has multiple stages:
 - Preprocessing
 - Compilation
 - Linking
- Executables are machine code binaries
- Debug, release, and sanitized builds behave differently

- What did people think about the bandit game?

- Tmux is another useful tool
- You can open multiple windows
- It remains open on a server (and can be running your programs), even after you disconnect
- This allows for no ssh flakiness, and gives you the ability to run long programs

- On Linux:

```
sudo apt-get update && sudo apt-get install tmux
```

- On mac: brew install tmux (you may already have it!)

- type `tmux` to enter a `tmux` session
- Whenever you type a command, it's `ctrl+b`, a very brief pause, and then the next key
- `ctrl+b "` to split the panes top to bottom
- `ctrl + b %` splits them left to right
- `ctrl + b <arrow_key>` to move around
- `ctrl + b d` to detach
- `tmux attach` to reattach
- Many more useful commands, but that's it for now. A useful cheatsheet <https://gist.github.com/MohamedAlaa/2961058>

Questions?

#include in C++

- #include is a macro. Before the C++ compilation starts, the #include tells the C++ preprocessor to copy the entire contents of the other file into your source code.
- This includes .h files, which are the declarations for functions and classes (the contract)
 - .h files are the contract, .cpp files are the implementation of the contract
 - You can (and should) omit the .h from the standard library headers now

Include Example

```
#include <iostream>
```

#include in C++

- Where to pull from
 - <> indicates standard headers (pull from standard library, skip user paths)
 - "" indicates user headers (pull from user paths first, then standard library)
- Looks in
 - Standard library search paths
 - User provided paths
 - Can use -I pathname to indicate where to search during compilation (we'll see this later)
- Caveat: Must protect against multiple inclusions (we'll come back to this)

Include Example

```
#include <iostream>
```

- So what do these headers look like?
- Well, typically they end in either .h (C) or .hpp (C++)
- They declare the contract that is implemented in the C++ files
- Let's take a look with some of our code

header files in C++

- This is code that calls a function
- But remember, the compiler must see a declaration of a function before it's called
 - This is unlike other languages like Python
- We must write the function out beforehand if we don't first declare it

Example

```
#include <iostream>

void my_function(int x) {
    std::cout << x << std::endl;
}

int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

header files in C++

- Much like we can declare an integer before assigning it, we can declare a function to get implemented later on
- To do this, we just write the function stub, and we can write the implementation afterwards
- Now C++ knows what the function looks like, and this code runs just fine!
- This declaration can live in a header so multiple files can agree on the same function signature

Example

```
#include <iostream>
void my_function(int x);
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

- However, we can further separate out our function declarations, the contract, into header files
- Let's go ahead and move our function stub into a header file called func.hpp

Example

```
void my_function(int x);
```

header files in C++

- Our code runs just the same!
- Remember:
 - Brackets indicate standard library
 - Double quotes indicate a user defined library

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
void my_function(int x) {
    std::cout << x << std::endl;
}
```


header files in C++

- Still, this isn't great
- It'd be much nicer to have our implementation, our function, also in its own file
- That way we can clearly separate our implementations from our main function, our entry point

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
void my_function(int x) {
    std::cout << x << std::endl;
}
```

- So let's go ahead and move our function into a file called func.cpp

Example

```
#include <iostream>
#include "func.hpp" // We include this to prevent
                   // accidental file changes later on for func.hpp
void my_function(int x) {
    std::cout << x << std::endl;
}
```

- Now we can compile and run our code, and it's looking pretty nice!

Example

```
#include <iostream>
#include "func.hpp"
int main() {
    int y;
    y = 50;
    my_function(y);
    return 0;
}
```

- Now often times, especially when working on larger repos, you'll typically group files together
- You don't really want all of your headers and all of your cpp files in the same place
- So let's go ahead and group our files into:
 - An includes directory, where we'll have our .h and .hpp files
 - An src directory, where we'll have our .cpp files

- Now to compile our program, we can run `c++ src/main.cpp src/func.cpp -I include`
 - We must pass in the `-I` flag because we no longer have our header files within our current working directory
 - `-I` let's the compiler know "look for my header files here"
- Now we have a good working project structure! Looks great! Runs great!

- Bringing this home: This is what including `iostream` is doing!
- `iostream` is being included from the standard template library
 - Specifically the headers are being included
 - The compiler sees declarations from the header
 - The linker later connects your code to the standard library's compiled implementations
- Remember, brackets indicate standard library, double quotes indicate user library

Namespaces

- Namespaces are a way to group a lot of entities together
- You can put constants, variables, functions, etc in a namespace
- You can also nest namespaces!
- You can think of namespaces similar to how you interact with Python modules
 - But namespaces only affect naming at compile time
 - There is no overhead at runtime
- For example, in Python, you can import time, and call time.sleep

- In C++, we:
 - include `iostream`
 - `iostream` modifies the `std` namespace (allowed in C++!)
 - `iostream` does this by declaring names for classes, objects, functions, etc. inside the `std`
 - It does not modify an existing namespace at runtime
 - We can get functions and classes from the `std` namespace like `std::cout`
- C++ provides us with a lot of libraries that we can include, we'll go over some of these later

Hello World!

- What happens if we remove the `std` from our program?
- We will get a nice error message since `cout` is not in the global namespace.

Hello World Example

```
#include <iostream>
int main() {
    cout << "Hello, world!" << std::endl;
    return 0;
}
```

Hello World!

- We can bring all names from a namespace into the current scope
- This affects name lookup, not the variables lifetime

Hello World Example

```
#include <iostream>

int main() {
    using namespace std;
    cout << "Hello, world!" << endl;
    return 0;
}
```

Hello World!

- This is bad practice!
- Now we don't know what we just imported
- We may accidentally introduce a function like `std::sort` into our scope, and how would we even know? What if we also had a function named `sort`?
- This is not good and adds ambiguity
- This is similar to running `from time import *` in Python

Hello World Example

```
#include <iostream>

int main() {
    using namespace std;
    cout << "Hello, world!" << endl;
    return 0;
}
```

When to use namespaces globally

- We've seen the syntax for using namespace std globally
- However, this will create ambiguity with many function names that we don't see.
 - For example, std contains a function called sort
 - This is very similar in Python to the bad practice of
from <module> import *
- Instead, we can just "import" what we need
 - using std::cout;
 - Similar to from time import sleep

Hello World Example

```
#include <iostream>
using std::cout;
using std::endl;
int main() {
    cout << "Hello, world!" << endl;
    return 0;
}
```

Namespaces

- We can also add our own namespaces
- Let's write a function that adds two numbers
- See also how we can shorten our `std::cout` and `std::endl` statement

Hello World Example

```
#include <iostream>
using std::cout, std::endl;

int add(int a, int b) {
    return a + b;
}

int main() {
    cout << "Hello, world!" << add(10, 20) << endl;
    return 0;
}
```

Namespaces

- We can create our own namespace
- Calling it like this will get an error, since foo is now in the namespace

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    int add(int a, int b) {
        return a + b;
    }
}
int main() {
    cout << "Hello, world!" << add(10, 20) << endl;
    return 0;
}
```

Hello World With Namespaces

- Instead we can access this function through the namespace using the scope resolution operator (::)

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    int add(int a, int b) {
        return a + b;
    }
}
int main() {
    cout << "Hello, world!" << MATHFUNCS::add(10, 20) <<
        endl;
    return 0;
}
```

The Scope Resolution Operator (::)

- The operator `::` is called the *scope resolution operator*
- It tells C++ exactly where a name lives
- Syntax: `scope::name`

Why Scope Resolution Matters

- C++ allows the same name to exist in multiple scopes
- Without `::`, the compiler may not know which one you mean
- `::` removes ambiguity

Revisiting `std::cout`

- `cout` is not a global variable
- It lives inside the `std` namespace
- `std::cout` means:
 - “the name `cout` inside the namespace `std`”

What `#include <iostream>` Does

- Including `<iostream>` declares names inside `std`
- Examples:
 - `std::cout`
 - `std::endl`
 - `std::cin`
- Nothing is added to the global namespace

The Global Scope Operator

- `::name` refers to the global namespace
- This is useful when names are hidden by local scopes

Example

```
int x = 10;
int main() {
    int x = 5;
    std::cout << ::x << std::endl; // prints 10
}
```

Hello World With Namespaces

- What about nesting? How does that work?

Hello World Example

```
#include <iostream>
using std::cout, std::endl;
namespace MATHFUNCS {
    namespace ADDFUNCS {}
    int add(int a, int b) {
        return a + b;
    }
}
int main() {
    cout << "Hello, world!" << MATHFUNCS::add(10, 20) <<
        endl;
    return 0;
}
```

Hello World With Namespaces

- We declare namespaces the same way if we split them out into header and cpp files
- The namespace should be declared both in the .hpp and in the .cpp files where you declare and define add
- Let's walk through an example of this

- Now we've covered almost all aspects of our hello world program (brief refresher)
- So what is this return 0?

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

What is `main`?

- `main` is a special function in every C++ program
- It is the program's *entry point*
- When your program starts running, execution begins in `main`

Why Does `main` Return an `int`?

- `main` returns an integer to the operating system
- This value is called the program's *exit status*
- It tells the OS whether the program succeeded or failed

What Does return 0 Mean?

- By convention:
 - 0 means success
 - Non-zero values mean failure
- This convention is used across Unix, Linux, and Windows

Non-Zero Return Values

- You can return different values to indicate different errors
- Example:

Example

```
int main() {  
    return 1; // something went wrong  
}
```

Non-Zero Return Values

- We can see this by running `echo $?` in the terminal
 - This gets the exit status of the last command that was run in the terminal
- This exit status is used in a lot of places
 - For example, if you call a subprocess in Python, typically you will check the exit status code and error out if it's not zero
 - Additionally, we can demonstrate this in the terminal by changing two commands together

Example

```
int main() {  
    return 1; // something went wrong  
}
```

Non-Zero Return Values

- In our terminals we can change two commands together with `&&`
- So here we can do `./program.out && ./program.out` and our `program.out` will run twice
 - The caveat here is that the chain will "break" if the exit status code is anything other than zero

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 1; // something went wrong
}
```

Non-Zero Return Values

- When we run `./program.out && ./program.out`, Hello World is printed only once because the exit status code is a 1, and not a zero
- Fix our program to return a 0, and run it again
 - Now hello world prints out twice
- Another example of chaining: Compiling and running on the same line

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 1; // something went wrong
}
```

Do We Always Need `return 0`?

- In C++, reaching the end of `main` is equivalent to:
 - `return 0`;
- This is *special to main*
- But! We shouldn't do this, even if the compiler allows it.

Example

```
int main() {  
    std::cout << "Hello World" << std::endl;  
}
```

Why Write `return 0` Anyway?

- Makes intent explicit
- Clear to readers new to C++
- Consistent with older C and C++ code
- Common in teaching and professional codebases

Who Uses the Return Value?

- The operating system receives the return value
- Other programs can check it
- This is how scripts detect success or failure

main Is Just a Function

- `main` behaves like other functions:
 - It has a return type
 - It can return early
- The difference is that the caller is the operating system

- And with that, we've covered every aspect of what you need to know for this hello world program
- This is also how all C++ programs are going to run, and the fundamentals of how they compile and run
- Now we'll close out a few of the finer details of this, and then we'll quickly start expanding what we can do in C++

Example

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

Questions?

End of day 4

#include in C++

- Let's dive back briefly into iostream
- #include is a macro. Before the C++ compilation starts, the #include tells the C++ preprocessor to copy the entire contents of the other file into your source code.
- This includes .h files, which are the definitions for functions and classes
 - .h files are the contract, .cpp files are the implementation of the contract
 - You can (and should) omit the .h from the standard library headers now

Include Example

```
#include <iostream>
```

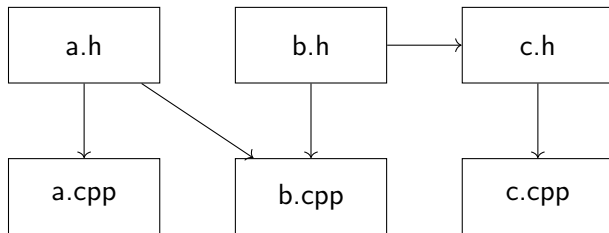
#include in C++

- Where to pull from
 - <> indicates standard headers (pull from standard library)
 - "" indicates user headers (pull from user paths)
- Looks in
 - Standard library search paths
 - User provided paths
 - Can use -I pathname to indicate where to search during compilation
- Caveat: Must protect against multiple inclusions

Include Example

```
#include <iostream>
```

Header and Source File Relationships



#include in C++

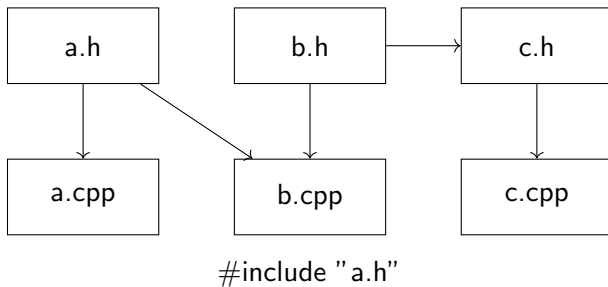
- Let's dive into this program we have here
- a.h, b.h, and c.h are going to be our header files for this program.
- These headers serve as the contract, and the .cpp files serve as the implementation of the contract

- So for example, a.cpp is the implementation of the contract defined in a.h
- Likewise b.h defines the contract that b.cpp implements
- The same is also true for c.h and c.cpp

#include in C++

- I've gone ahead and included these relationships within the diagram of how these files rely on each other
- So b.cpp RELIES on a.h. In our specific example, b.cpp is calling functions from a.h
- Because b.cpp relies on a.h and calls functions declared in a.h, the program will not compile unless we include the a.h function headers in b.cpp

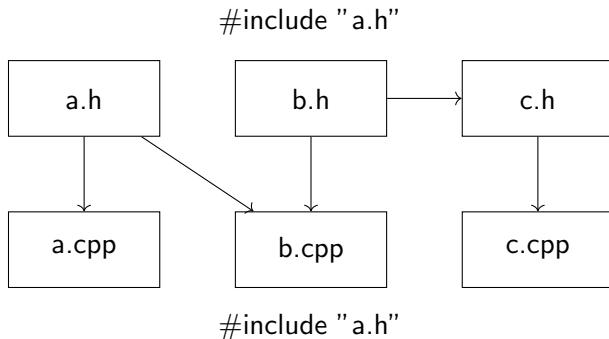
Header and Source File Relationships



#include in C++

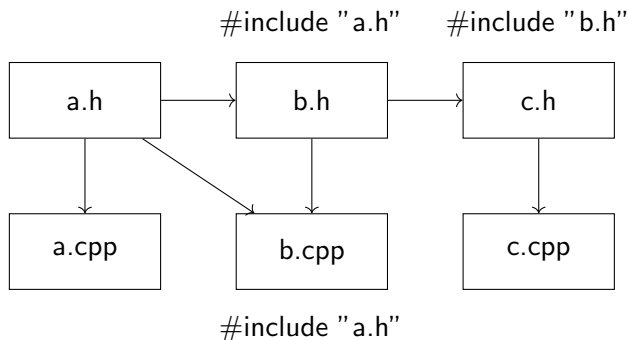
- So let's go ahead and make sure to include `#include "a.h"` in `b.cpp`.
- Next up is `c.h`
- In our example `c.h` is going to call functions declared in `b.h`
- Because of this we must make sure to include these in `c.h` as well, as shown here:

Header and Source File Relationships



- Hmm, this is not quite right.
- b.cpp calls functions declared in a.h, so should we add a `#include "a.h"` to c.h?
 - No!
 - We should instead include a.h in b.h.
 - That way include a.h will be copied everywhere that b.h is used automatically.
 - This is much cleaner than manually copy pasting include a.h everywhere that is using b.h.

Header and Source File Relationships



- a.h gets included twice!

- Now that we've included everything properly, we can compile this example program
- Remember that the preprocessor and compiler starts by:
 - Copying the declarations in a.h into a.cpp
 - And writes the output to a.o

#include in C++

- Next the preprocessor and compiler will move to b
- First b.cpp includes a.h since b.cpp is calling functions from a.h
- So the preprocessor will copy a.h into b
- Then the preprocessor moves to copy b.h into b
- But wait! b.h also includes a.h
- The preprocessor is now copying a.h into b twice, and declaring the same functions twice!
- This is terrible!

- Keep in mind: This happens even though our graph has no cycles!
- This double inclusion problem still occurs even with a directed acyclic graph
 - Directed: Arrows go in one direction
 - Acyclic: Has no cycles (has no loops)

- So what about a.h?
- Can we just not include a.h in b.cpp since a.h is already included in b.h?
- No!
- b.cpp calls functions from a.h so we must include a.h itself
- Relying on other headers downstream to include things properly is very bad practice
 - So we should just include a.h wherever we are using it
- Even if you use the a.h included in b.h, it's fragile code:
 - a.h could at some point be removed from b.h
 - Then your code would break!

#ifndef

- Use ifndef for every header to avoid this. Example below for b.h.
- a.h looks exactly the same.
- A_H is a name we make up, we can select it, but this is generally how it's done (FILENAME_H)
- (explanation of ifndef)

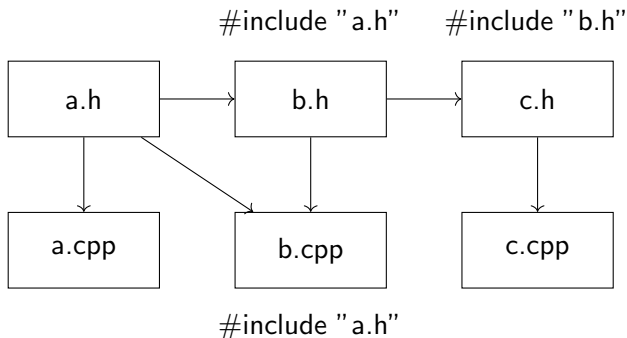
a.h

```
#ifndef A_H
#define A_H
//Insert a.h code here
#endif
```

b.h

```
#ifndef B_H
#define B_H
//Insert b.h code here
#endif
```

Putting it all together



a.h

```
#ifndef A_H
#define A_H
//Insert a.h code here
#endif
```

- So let's bring this concept home
- I've omitted b.h and c.h for space, but they look similar to a.h (just replace the letters with b or c).
- Compilation starts at a.cpp
- a.cpp will then pull in a.h.
- The preprocessor will go through the if not defined (and it is not defined), so it will define a.h, and then end if.
- This initial process will have now created a.o successfully

#include in C++

- Next the preprocessor and compiler will start on b
- b.cpp includes a.h
- a.h will not yet be defined for this .o file, and so the compiler will define it
- The compiler will then include b.h
- b.h includes a.h, but a.h is already defined!
- Therefore, due to our ifndef statement, a.h is not added a second time
- The compiler just continues with the b.h definition, and hits that endif
- b.o compiles successfully and compilation continues.

- After a.o, b.o, and c.o are generated (i.e. all the .o files are generated), the linker brings them together into an executable that we can use
- Easy peasy, we've fixed this double includes issue.
- Make sure to always do this for your header files, also known as header guards, also known as include guards.

Header Guards Summary

- That's how `ifndef` works, include it in every single header file
- Header guards are not enforced by the language itself
 - But they are good practice
- These are also a backwards compatible feature of the C language
 - Like much of C++, if you don't protect yourself and use these patterns, it will cause problems down the line!
 - Always use header guards!

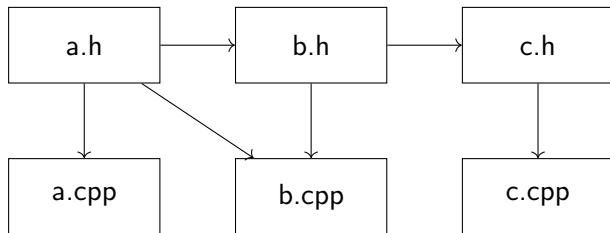
#pragma once vs include guards

- Standard, portable approach in C & C++:
#ifndef/#define/#endif include guards.
- #pragma once is a widely supported *non-standard* alternative in C/C++.
- Pros: simple (one line).
- Cons: not in the ISO standards; some niche/older compilers may not support it.
- Modern compilers optimize traditional guards, so build-time differences are usually small.
- Used by several major tech companies, including Nvidia with CUDA.

a.h

```
#pragma once  
// a.h contents
```

a.h, b.h, c.h example



Questions?

Standard Input: `std::cin`

- `std::cin` represents standard input (usually the keyboard)
- It lives in the `std` namespace
- It is an input stream, just like `std::cout` is an output stream

The Extraction Operator (>>)

- Input uses the extraction operator >>
- Syntax:
 - `std::cin >> variable;`
- This reads input from the user into a variable

Reading Input with `std::cin`

Example

```
#include <iostream>

int main() {
    int age;
    std::cout << "Enter your age: ";
    std::cin >> age;
    std::cout << "You entered " << age << std::endl;
    return 0;
}
```


Chaining Input Operations

- Like <<, the >> operator can be chained

Example

```
int x, y;  
std::cin >> x >> y;
```

Printing Newlines

- There are two common ways to print a newline:
 - `std::endl`
 - `'\n'`

What Is the Output Buffer?

- Output written to `std::cout` is *buffered*
- This means output is stored temporarily in memory
- The program does not write to the screen immediately
- Instead, characters are collected in an output buffer

Why Output Is Buffered

- Writing to the screen is relatively slow
- Writing one character at a time is inefficient
- Buffering allows many characters to be written at once
- Result:
 - Faster programs
 - Smoother output

When the Output Buffer Is Flushed

- The output buffer is flushed when:
 - The buffer becomes full
 - The program finishes execution
 - You explicitly flush it (e.g., `std::endl`)
- Flushing forces output to appear immediately

What `std::endl` Does

- Prints a newline
- Flushes the output buffer
- Flushing forces output to appear immediately

What `\n` Does

- Prints a newline character
- Does *not* flush the buffer
- Usually faster than `std::endl`

`std::endl` vs `\n`

- Prefer `\n` for normal output
- Use `std::endl` when you *need* to flush
 - Debugging
 - Prompting before input

Whitespace in Input

- The >> operator:
 - Skips leading whitespace
 - Stops reading at the next whitespace
- Whitespace includes:
 - spaces
 - tabs
 - newlines

Whitespace Example

Example

```
int x;  
std::cin >> x;
```

- Input: 42: OK
- Input: 42: OK
- Input: 42 99: Reads 42, leaves 99

A Common Input Pitfall

Example

```
#include <string>
int age;
std::string name;
std::cin >> age;
std::getline(std::cin, name);
```

- `name` may be empty!
- The newline from the previous input is still in the buffer
 - The newline gets eaten up by the `getline`
- This is a brief intro into strings, we'll come back to this later

Fixing Input Buffer Issues

Example

```
int age;  
std::string name;  
  
std::cin >> age;  
std::cin.ignore();  
std::getline(std::cin, name);
```

- `std::cin.ignore()` removes leftover characters
- Now `getline` works as expected

Input Rule of Thumb

- `>>` is good for:
 - numbers
 - single words
- `std::getline` is good for:
 - full lines
 - input with spaces

Streams Are Stateful

- Input and output streams remember what happened before
- Leftover characters stay in the buffer
- This is why input order matters

- Next we'll review C control flow
 - If
 - While
 - for
 - switch
 - etc

Branching with if

- Same as in C
- If there is a single instruction, you can omit the brackets

Example

```
int main() {  
    if(expression_or_condition){  
        //code  
    } else {  
        // code  
    }  
}
```


Comparison Operators

- Comparison operators compare two values
- The result of a comparison is always a bool
 - true or false

Operator	Meaning
==	equal to
!=	not equal to
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to

Comparison Operator Examples

Example

```
int x = 5;
if (x == 5) {
    // true
}
if (x != 3) {
    // true
}
if (x > 0) {
    // true
}
if (x <= 10) {
    // true
}
```

Logical Operators

- Logical operators combine boolean expressions
- They are commonly used in `if`, `while`, and `for` conditions
 - We'll cover `while` and `for` loops soon

Operator	Meaning
<code>&&</code>	logical AND
<code> </code>	logical OR
<code>!</code>	logical NOT

Logical Operator Examples

Example

```
bool correct_password = true;
bool has_2fa = false;

if (correct_password && has_2fa) {
    // user is allowed to log in
}

if (!correct_password || !has_2fa) {
    // login denied
}
```

Combining Comparisons and Logic

- Comparisons produce booleans
- Logical operators combine those booleans

Example

```
int score = 85;

if (score >= 70 && score <= 100) {
    std::cout << "Passing score" << std::endl;
}
```

Operator Precedence (Basics)

- Operator precedence determines the order expressions are evaluated
- You do **not** need to memorize everything
- Just remember the most common cases

High to Low Precedence (Common)

!

*, /, %

+, -

< <= > >=

== !=

&&

||

Operator Precedence Example

Example

```
int x = 5;

if (x > 0 && x < 10) {
    // evaluated as:
    // (x > 0) && (x < 10)
}
```

- Comparison operators run before logical operators
- Parentheses make intent explicit and improve readability

When in Doubt, Use Parentheses

- Parentheses () override precedence rules
- They make code:
 - Easier to read
 - Less error-prone

Example

```
if ((x > 0 && x < 10) || x == 42) {  
    std::cout << "Valid value" << std::endl;  
}
```


Branching with if

- Same as in C
- If there is a single instruction, you can omit the brackets

Example

```
int main() {  
    int x = 5;  
    if(x == 6){  
        //code  
    } else if (x == 5){  
        // code  
    } else {  
        // code  
    }  
}
```

Conditions Must Be Boolean

- Every condition in control flow must evaluate to a `bool`
- This means the expression must be either:
 - `true`
 - `false`
- A very common bug is confusing assignment with comparison

Conditions Must Be Boolean

Common Bug

```
int x = 5;  
if (x = 6) {    // WRONG: assignment, not comparison  
    // always true!  
}
```

Correct

```
if (x == 6) {   // comparison  
    // runs only if x equals 6  
}
```

Braces vs Single Statements

- If there is only a **single statement**, braces may be omitted
- However, this can easily cause bugs
- Only the *next statement* is controlled by the condition

Braces vs Single Statements

Buggy Code

```
if (x > 0)
    std::cout << x << std::endl;
    std::cout << "Done" << std::endl; // ALWAYS runs
```

Recommended

```
if (x > 0) {
    std::cout << x << std::endl;
    std::cout << "Done" << std::endl;
}
```

Scope Inside Control Flow

- Curly braces {} create a new scope
- Variables declared inside control flow blocks:
 - Only exist inside that block

Example

```
#include <iostream>
int main() {
    if (true) {
        int x = 10;
        std::cout << x << std::endl;
    }
    // std::cout << x << std::endl; // ERROR: x is out of
    // scope
}
```

Switch

- Same as in C
- Cases must be a constant expression evaluated at compile time

Example

```
#include <iostream>
using std::cout;
int main() {
    int day = 2;
    switch(day) {
        case 1:
            cout << "Monday";
            break; // DONT FORGET! (Fall through)
        case 2:
            cout << "Tuesday";
            break;
        // days 3-7 omitted for space
        default: // For when none of the cases match
            cout << "Invalid day";
    }
}
```

Switch Fallthrough

- If `break` is omitted, execution continues into the next case
- This is called **fallthrough**
- Sometimes intentional, often a bug

Example

```
switch (x) {  
    case 1:  
    case 2:  
        std::cout << "One or Two" << std::endl;  
        break;  
    default:  
        std::cout << "Other" << std::endl;  
}
```


While

- Same as in C
- Single line expressions don't need brackets

Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    int count = 0;
    while(count < 3) {
        cout << "Count: " << count << endl;
        count++;
    }
    // Case 2: Without brackets (single statement)
    int i = 5;
    while(i > 0)
        cout << "i = " << i-- << endl;
}
```

Do-While

- Same as in C
- Single line expressions don't need brackets
- Executes at least once (body runs before condition check)

Do-While Loop Examples

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    int count = 0;
    do {
        cout << "Count: " << count << endl;
        count++;
    } while(count < 3);
    // Case 2: Without brackets (single statement)
    int i = 5;
    do
        cout << "i = " << i-- << endl;
    while(i > 0);
}
```

For

- Same as in C
- Single line expressions don't need brackets
- Three parts: initialization, condition, increment

For Loop Examples

```
#include <iostream>
using std::cout, std::endl;
int main() {
    // Case 1: With brackets (multiple statements)
    for(int count = 0; count < 3; count++) {
        cout << "Count: " << count << endl;
        cout << "Additional statement" << endl;
    }
    // Case 2: Without brackets (single statement)
    for(int i = 5; i > 0; i--)
        cout << "i = " << i << endl;
}
```

Loop Variable Scope

- Variables declared in a for loop header:
 - Exist only inside the loop

Example

```
#include <iostream>
int main() {
    for (int i = 0; i < 3; i++) {
        std::cout << i << std::endl;
    }
    // std::cout << i << std::endl; // ERROR
}
```

Range-Based for Loop (C++)

- C++ provides a simpler loop for iterating over collections
- This is **not** available in C
- Syntax:
 - for (type variable : collection)

Example

```
#include <iostream>
int main() {
    int nums[4] = {1, 2, 3, 4};
    for (int x : nums) {
        std::cout << x << std::endl;
    }
}
```

break; continue

- Same as in C
- break: Breaks out of the innermost loop or switch
- continue: skip the remainder of the block and go to the next loop

break, continue

```
#include <iostream>
using std::cout, std::endl;
int main() {
    for(int i = 0; i < 10; i++) {
        if(i == 5)
            break; // Exit loop when i equals 5
        cout << "i = " << i << endl;
    }
    for(int j = 0; j < 5; j++) {
        if(j == 2)
            continue; // Skip when j equals 2
        cout << "j = " << j << endl;
    }
}
```

break vs continue vs return

- **break**
 - Exits the *nearest loop or switch*
- **continue**
 - Skips to the next iteration of the loop
- **return**
 - Exits the *entire function immediately*

break vs continue vs return

Example

```
void foo(){
    for (int i = 0; i < 10; i++) {
        if (i == 3)
            continue; // skip 3
        if (i == 7)
            break;      // exit loop
        if (i == 8)
            return;     // ends function
        std::cout << i << std::endl;
    }
}
```


Nested Conditionals

- A conditional can appear inside another conditional
- This is called a **nested conditional**
- The inner condition only runs if the outer condition is true

Example

```
bool logged_in = true;
bool is_admin = false;

if (logged_in) {
    if (is_admin) {
        std::cout << "Admin access" << std::endl;
    } else {
        std::cout << "User access" << std::endl;
    }
}
```

Nested Conditionals with else if

- Many nested conditionals can be simplified
- else if avoids deep nesting
- Improves readability

Example

```
int score = 85;

if (score >= 90) {
    std::cout << "A" << std::endl;
} else if (score >= 80) {
    std::cout << "B" << std::endl;
} else if (score >= 70) {
    std::cout << "C" << std::endl;
} else {
    std::cout << "F" << std::endl;
}
```

Why Deep Nesting Is Dangerous

- Deep nesting makes code:
 - Harder to read
 - Harder to debug
 - Easier to get wrong
- This is sometimes called “arrow code”

Hard to Read

```
if (a) {  
    if (b) {  
        if (c) {  
            if (d) {  
                do_something();  
            }  
        }  
    }  
}
```

Flattening Nested Conditionals

- Often you can simplify nested conditionals
- Combine conditions using logical operators

Better

```
if (a && b && c && d) {  
    do_something();  
}
```

The Ternary Operator

- The ternary operator is a compact if / else
- It is the only operator in C++ that takes three operands

Syntax

```
condition ? value_if_true : value_if_false;
```

Ternary Operator Example

Example

```
int score = 85;

std::string result = (score >= 70) ? "Pass" : "Fail";

std::cout << result << std::endl;
```

Ternary vs if / else

- Use the ternary operator when:
 - The logic is simple
 - You are choosing between two values
- Avoid ternary when:
 - The logic is complex
 - Side effects are involved

Good

```
int max = (a > b) ? a : b;
```

Bad

```
(a > b) ? do_this() : do_that();
```

Ternary Operator Pitfall

- Ternary operators can be nested
- This quickly becomes unreadable

Avoid This

```
std::string grade =  
    (score >= 90) ? "A" :  
    (score >= 80) ? "B" :  
    (score >= 70) ? "C" : "F";
```


Rule of Thumb

- Prefer clarity over cleverness
- If a conditional takes effort to understand:
 - Rewrite it
 - Add braces
 - Use `if / else`
- Ternary is a tool, not a requirement

- No

Name spaces in class exercise

Going over our in class exercise

End of Day 5

What Is Git?

- Git is a **version control system**
- It tracks changes to files over time
 - Basically: like saving a file, but you can save your entire repo
- Allows you to:
 - Undo mistakes
 - Experiment safely
 - Collaborate with others
- Git runs **locally** on your computer

What Is Git?

- Git is basically the version control almost every company uses
- If you don't know git on your resume, it's pretty much an instant reject
 - It's sort of like saying "I don't know how to use email"
- This is an extremely important tool to learn
 - It's a common instant-fail interview question

What Is GitHub?

- GitHub is a **website** that hosts Git repositories (codebases)
- It allows you to:
 - Store code online
 - Share code with others
 - Collaborate on projects
- Git = tool GitHub = service

What Is GitHub?

- The other reason GitHub is important: commit logs
- You can view and track someone's commit logs over time
- Recruiters and automated resume checkers look at this
 - They see: Do you actually code?
 - Or do you just "think" about coding with theory and classwork?
- See: [Github.com/jfuruness](https://github.com/jfuruness)
- Note: You can turn on in your settings the ability to have this graph populate with commits on private repos (repositories)

- A **repository** (repo) (codebase) is a project tracked by Git
- A repo contains:
 - Files
 - Commit history
 - Branches
- Repositories can be:
 - Local (on your computer)
 - Remote (on GitHub)

- A **commit** is a snapshot of your project
- Each commit has:
 - A unique ID
 - A message describing the change
- Commits should be:
 - Small
 - Frequent
 - Meaningful
- Think of it like a save point in a video game that you can go back to

Basic Git Workflow

- 1 Modify files
- 2 Stage changes
- 3 Commit changes
- 4 Push changes

Commands

```
git status  
git add file.cpp  
git commit -m "Describe the change"
```

Common Git Commands

Most Used Commands

```
git init           # create a repo
git status         # see repo state
git add            # stage files
git commit         # save changes
git log            # view history
git push           # push changes to github (if set up)
```

What Is a Branch?

- A **branch** is an independent line of development
- Branches let you:
 - Experiment without breaking main code
 - Work on features in parallel
- The default branch is usually called `main`

Why Branching Matters

- You should not develop directly on `main`
- Branches allow:
 - Safer development
 - Easier collaboration
 - Cleaner history
- If something breaks, `main` is still safe

Branching Commands

Working with Branches

```
git branch                # list branches
git checkout -b feature-x  # create branch
git checkout feature-x     # switch branches
git checkout main
```


Merging Branches

- When a feature is complete, it can be merged
- Merging combines branch history
- Sometimes Git needs help resolving conflicts

Merge Example

Example

```
git checkout main  
git merge feature-x
```

Local vs Remote Repositories

- Your computer has a **local repo**
- GitHub hosts a **remote repo**
- You sync them using:
 - push
 - pull

Push and Pull

Commands

```
git push  
git pull
```

What Is a Pull Request?

- A **pull request (PR)** is a request to merge code
- Created on GitHub
- Used to:
 - Review code
 - Discuss changes
 - Run automated checks

Pull Request Workflow

- 1 Create a branch
- 2 Make commits
- 3 Push branch to GitHub
- 4 Open a pull request
- 5 Review and merge

Why Pull Requests Are Important

- Encourage code review
- Catch bugs early
- Improve code quality
- Standard practice in industry
- This is how EVERYONE collaborates and works
 - Again, a very common interview question

Rule of Thumb

- Commit early and often
- Write meaningful commit messages
- Use branches for features
- Never be afraid to make a branch

- Python is another language that is closely related to C/C++
- A lot of times, Python is actually running C++ code under the hood to make it faster
- We'll use Python for our testing since it's a lot easier to use PyTest than it is to get into C++ testing at this stage
- We'll only touch briefly on this.
 - I'll provide you the tests, you won't have to write them (for now)
 - So we'll just cover how to run the tests, rather than write them, since this is a C++ course

The Python Ecosystem

- One of the best parts of the Python language is the Python ecosystem
- Unlike a lot of other languages, Python makes it very easy to develop and share code
- For pretty much anything that you might want to do, someone likely has already coded it up
- Often times, code you write, will just be connecting other people's tools!
- This saves a TON of time, as people have spent years and years building out tools that you don't have to write or debug (at least, hopefully you won't have to debug)

Terminology

- There are a few terms that are critical to know for this and you'll hear them all the time in industry
- The first is what's called a package
- When you want to share your code with someone else, you need to turn that code into a Python Package
- The word package and module are essentially interchangeable
- We want to use the PyTest package for testing

- Python gives us a tool to be able to install packages called pip
- pip goes and searches up whatever name we give pip
- It then tries to find that name in the Python Package Index, also known as PyPI, and install the package we are looking up!
- But... where does it install that package?
- This is where Python virtual environments come in!
 - We need to understand how Python virtual environments work, *before* we can use pip to install anything

- By default, pip will install code to our computer!
- This is also known as installing a module or package globally
 - Also known as installing a module/package directly onto the operating system
 - The operating system is what is running your computer
 - Such as Windows, MacOS, Ubuntu, etc.
- Unfortunately, installing a package/module globally is a very bad idea
- Often times these days, your computer will actually prevent you from global installs like this

Danger!

- Your computer has code that runs every time it turns on
- That code is called the operating system
 - The operating system has code for the display so that you can see things
 - There's code for the keyboard so that you can type things
 - There's code for pretty much everything that you do
- Thankfully, 99% of the time, your operating system works just fine, and you never have to worry about this!
 - Although you may be aware of some phenomena like the "blue screen of death" that windows sometimes has
 - This is when code in your operating system has a very bad bug

Danger!

- Now your operating system actually also uses Python!
- And your operating system also has modules and packages, just like your code!
- When you install a package globally, you could actually be overwriting a package that your operating system relies upon
 - Even if it's the same package, you could be installing a different version of it than the one your operating system uses to run your computer!
- In other words, by installing packages globally, you could be changing code that your operating system needs to function
- That's really not good!

Danger!

- Additionally, if you install a package globally, and the install is broken in some way... now what?
- Uninstalling or fixing that package becomes extremely difficult
- This is especially true when you're building your own Python packages
 - This is even more true if you do things like add C++ or Rust bindings to those packages
- But it can also be a problem for installing other people's packages as well

Virtual Environments to the Rescue!

- Fortunately, Python Virtual Environments fix all of these issues!
- That is why they are so critical
- A Python Virtual Environment is an isolated area that packages get installed to
- They don't conflict at all with what your operating system is using
- If an installation has a problem... you can simply delete it
 - And not have to worry about deleting part of your operating system
- Let's dive into how to make these

How to Make a Python Virtual Environment

- I'll be going over two ways here for how to make a Python Virtual Environment
- In the first way, I'll cover how to make a Python virtual environment using a terminal (Mac, Ubuntu, Windows with WSL2)

How to Make a Python Virtual Environment: Terminal

- If you are on a Mac or Apple computer, you should be all set here, as virtual environments come with the default Python installation
- Otherwise, there are two commands that you need to run before we get started

terminal

```
sudo apt update  
sudo apt install -y python3 python3-pip python3-venv
```

How to Make a Python Virtual Environment: Terminal

- From here on in it's the same for everyone:
- Don't forget to upgrade pip!
 - IDEs like VS Code will automatically upgrade it for you
 - But if you are in a terminal, you must do this yourself!

terminal

```
python3 -m venv env
source env/bin/activate
pip3 install pip --upgrade # don't forget this!
```

Terminal: How to Install a Package

- Now that we have activated our virtual environment, let's go ahead and install a package
- Let's install our testing library PyTest
 - PyTest is by far the most popular testing framework
 - There really is no reason to ever not use PyTest for Python tests

Terminal: How to Install a Package

- Go ahead and just run `python3 -m pip install PyTest`
- You have now installed PyTest in your local environment!
- To check this, you can run `python3 -m pip list` to see all of the packages that have been installed
- You can even type `python3` to execute python commands, and go ahead and import PyTest, and then `exit()`
- Mission success! We can import it!

Terminal: Global Install Differences

- Now let's see how this is different from a global install
- We can deactivate our virtual environment by running deactivate
- Now you should see those parenthesis disappear
 - That means we are no longer using any virtual environments
- Let's go ahead and run `python3 -m pip list`
 - pytest is no longer there!
- Try importing pytest after running `python3`
 - We can't! That's because it's not installed globally!

Terminal: Starting over?

- Now what if our installation was broken somehow and we wanted to start over?
- We could simply delete the virtual environment with an `rm -rf env`
- Then we can run through the process of creating a virtual environment again
- Let's go ahead and do that now for some practice

Terminal: Using pytest

- One more time let's just install pytest real quick
- Then let's dive back into our code and actually use it!
- Let's just write a simple test.py file
 - Note: Pytest files must start with the name test
- And now we can run our code (with our environment activated) with `python3 -m pytest!`
- We'll use this to test a lot of our code throughout this course

test.py

```
def test_add():  
    assert 1 + 1 == 2, "Fail"
```

- There will be an exam soon!
- Make sure to bring:
 - ID
 - Computer
 - Computer charger
- Cheaters will be given a zero for the course

- I recommend going over:
 - The slides
 - The in-class exercises
 - The quizzes
 - The homework
- Make sure you can do these problems without help in a timely manner!
- And I would do more studying outside of this

- The exam is going to be a similar format of the quizzes
- There will be some short answer questions
- There will be several coding problems worth the majority of the points on the exam
- This is designed like a basic C++ interview (without algorithms)
 - My goal is to align these tests with how most industry jobs will test this algorithms knowledge
 - Minus algorithms, since that isn't covered here

Function Overloading

- C++ allows multiple functions to share the same name
- This is called **function overloading**
- The compiler decides which function to call based on:
 - Number of arguments
 - Types of arguments
 - Order of arguments
- This decision happens at **compile time**

Why Function Overloading Exists

- Improves readability
- Lets you express the same *concept* for different types
- Avoids awkward naming like:
 - `add_int`
 - `add_double`
 - `add_float`
- Very common in the C++ standard library

Basic Function Overloading Example

Example

```
#include <iostream>

int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3) << std::endl;
    std::cout << add(2.5, 3.1) << std::endl;
    return 0;
}
```

How the Compiler Chooses

- The compiler looks at the **function signature**
- A function signature includes:
 - Function name
 - Parameter types
 - Parameter order
- Return type is **not** part of the signature

Return Type Is NOT Enough

Invalid Overload

```
int foo(int x);  
double foo(int x); // ERROR
```

- The compiler cannot choose based on return type
- This results in a compile-time error

Overloading with Different Parameters

Example

```
#include <iostream>
void print(int x) {
    std::cout << "int: " << x << std::endl;
}
void print(int x, int y) {
    std::cout << "two ints: " << x << ", " << y << std::endl;
}
int main() {
    print(5);
    print(3, 4);
    return 0;
}
```

Overloading Can Go Wrong

- Some overloads introduce ambiguity
- The compiler must find *exactly one* best match
- Otherwise: compile-time error

Ambiguous Example

```
void foo(int x);  
void foo(double x);  
  
foo(3);    // int or double?
```

Can You Overload `main`?

- **No** — `main` **cannot be overloaded**
- The C++ standard allows only specific signatures
- The operating system is the caller
- Overloading `main` would break program startup

Valid main Signatures

- C++ allows exactly two common forms of main

No Arguments

```
int main() {  
    return 0;  
}
```

With Arguments

```
int main(int argc, char* argv[]) {  
    return 0;  
}
```

Command-Line Arguments

- `argc` = argument count
- `argv` = argument vector
- These are passed in by the operating system
- Used to control program behavior from the terminal

Understanding argc and argv

- `argc` tells you how many arguments exist
- `argv` is an array of C-style strings
- `argv[0]` is always the program name

Using Command-Line Arguments

Example

```
#include <iostream>

int main(int argc, char* argv[]) {
    std::cout << "Argument count: " << argc << std::endl;
    for (int i = 0; i < argc; i++) {
        std::cout << "argv[" << i << "]: "
                  << argv[i] << std::endl;
    }
    return 0;
}
```


Running with Arguments

- Compile:

Compile

```
c++ main.cpp -o program
```

- Run:

Run

```
./program hello world 42
```

C Refresher... is over!

- Up until now, almost everything has hopefully been a refresher
 - The terminal
 - Pytest
 - All the basic C functionality
- We haven't even reviewed pointers yet, but we will do this later on
- Now we will start to dive deep into C++

Day 6 Quiz

- divisible by 5 or 7 quiz

End of Day 6

Exam (End of day 7)

Strings in C and C++

- C++ supports two kinds of strings:
 - **C-style strings** (from C)
 - **C++ strings** (`std::string`)
- You will encounter both in real-world code
- However, `std::string` is almost always preferred

C-Style Strings

- A C-style string is:
 - An array of `char`
 - Ending with a null terminator: `\0`
- This comes directly from the C language
- C++ still supports this for compatibility
- They have less overhead, and therefore are sometimes still used

C-Style String Example

Example

```
#include <iostream>

int main() {
    char name[6] = "Justin"; // BUG!
    std::cout << name << std::endl;
}
```

- Why is this broken?
- "Justin" needs **7** characters:
 - J u s t i n \0

C-Style String Problems

- You must manually manage memory size
- Easy to overflow buffers
- No automatic resizing
- Comparing strings is painful
- Many security bugs come from this

C++ Strings: `std::string`

- C++ provides `std::string`
- It lives in the `std` namespace
- Handles memory management for you
- Much safer and easier to use

Why `std::string` Is Easier

- Automatically manages memory
- Can grow and shrink dynamically
- Supports:
 - Assignment
 - Comparison
 - Concatenation
 - Length queries
- Much harder to misuse

Getting String Length

Example

```
std::string s = "Hello";  
std::cout << s.length() << std::endl;  
std::cout << s.size() << std::endl;
```

- `length()` and `size()` are identical
- Runs in constant time

Later class ended here

String Concatenation

Example

```
std::string first = "Hello";  
std::string second = "World";  
  
std::string result = first + " " + second;  
std::cout << result << std::endl;
```

- Much cleaner than C-style concatenation
- No manual buffers

Comparing Strings

Example

```
std::string a = "cat";  
std::string b = "dog";  
  
if (a == b) {  
    std::cout << "Equal" << std::endl;  
} else {  
    std::cout << "Not equal" << std::endl;  
}
```

- ==, !=, <, > all work
- Lexicographical (dictionary) ordering

Example

```
std::string word = "hello";  
  
std::cout << word[0] << std::endl; // 'h'  
std::cout << word[4] << std::endl; // 'o'
```

- Strings are indexable like arrays
- Indices start at 0

Modifying Strings

Example

```
std::string s = "hello";  
s[0] = 'H';  
s += " world";  
  
std::cout << s << std::endl;
```

- Strings are mutable
- Automatically resize when needed

Reading Strings from Input

Single Word

```
std::string name;  
std::cin >> name;
```

Full Line

```
std::string line;  
std::getline(std::cin, line);
```

- Same input rules you saw earlier apply

C-Style vs `std::string`

Feature	C-Style	<code>std::string</code>
Automatic resizing	No	Yes
Memory safety	No	Yes
Easy concatenation	No	Yes
Easy comparison	No	Yes
Beginner friendly	No	Yes

Rule of Thumb

- Use `std::string` by default
- Only use C-style strings when:
 - Interfacing with C libraries
 - Required by low-level APIs
- Modern C++ code should prefer safety and clarity unless you're looking for extreme speed

Mixing String Types

- C++ supports:
 - `std::string`
 - C-style strings (`char*`, string literals)
- These types can interact — but with rules
- Some combinations work
- Some combinations do **not** compile

Adding a C-Style String to `std::string`

Works

```
#include <string>

std::string s = "Hello";
s = s + " world";
```

- " world" is a C-style string literal
- `std::string` knows how to absorb it
- Result is a new `std::string`

Also Works

```
#include <string>

std::string s = "world";
std::string result = "Hello " + s;
```

- This still works
- Why?
 - C++ converts the C-style string into a `std::string`
 - Then performs string concatenation

C-Style + C-Style Does NOT Work

Does NOT Compile

```
char a[] = "Hello ";  
char b[] = "world";  
  
auto result = a + b; // ERROR
```

- This is NOT string concatenation
- a and b are pointers / arrays
- + is undefined for C-style strings

Why This Fails

- C-style strings are just memory addresses
- Adding them would mean:
 - Pointer arithmetic
 - Not string concatenation
- C has no built-in string type
- That's why C uses functions like:
 - `strcpy`
 - `strcat`

Fix: Convert to `std::string`

Correct Way

```
char a[] = "Hello ";  
char b[] = "world";  
  
std::string result = std::string(a) + b;
```

- Convert at least one side to `std::string`
- After that, concatenation works normally

Concatenation Rules

- `std::string + std::string`: works
- `std::string + C-style string`: works
- `C-style string + std::string`: works
- `C-style string + C-style string`: fails

Earlier class ended here

Iterating Over a String

- A `std::string` is a sequence of characters
- You can loop over it just like an array
- C++ provides a cleaner way: **range-based for**

Index-Based String Iteration

Example

```
std::string word = "hello";  
  
for (size_t i = 0; i < word.size(); i++) {  
    std::cout << word[i] << std::endl;  
}
```

- Works
- But verbose
- Easy to make off-by-one errors

Range-Based for with Strings

Cleaner Version

```
std::string word = "hello";  
  
for (char c : word) {  
    std::cout << c << std::endl;  
}
```

- No indices
- No bounds checks
- Much easier to read

Finding substrings with .find

- `string::find()` searches for a substring or character
- Returns the index (starting at 0) where it is found
- Returns `string::npos` if not found

Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;

int main() {
    string s = "hello world";
    size_t pos = s.find("world");
    if (pos != string::npos) {
        cout << "Found at position " << pos << endl;
    }
}
```


What is size_t?

- size_t is the safe type to use for lengths and indices, since it's guaranteed to hold the size of any object in memory.

size_t Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;

int main() {
    string s = "hello";
    size_t len = s.size();
    cout << "Length = " << len << endl;
}
```

.find: int vs size_t

- `string::find()` returns `size_t`
- If the substring is not found, it returns `string::npos`
- `npos` = the largest possible `size_t` value
- Using `int` works for small examples, but cannot truly represent `npos`

size_t Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;
int main() {
    string s = "hello";
    // Using size_t (recommended)
    size_t pos1 = s.find("world");
    if (pos1 == string::npos)
        cout << "Not found (size_t)" << endl;
    // Using int (sometimes seen in C++ tutorials - wrong)
    int pos2 = s.find("world");
    if (pos2 == -1) // overflowed value
        cout << "Not found (int)" << endl;
}
```

Extracting substrings with .substr

- `string::substr(pos, len)` returns a substring
- `pos` is the starting index
- `len` is the number of characters (optional; defaults to rest of string)

.substr Example

```
#include <iostream>
#include <string>
using std::cout, std::endl, std::string;
int main() {
    string s = "Alice Smith";
    string first = s.substr(0, 5);        // "Alice"
    string last  = s.substr(6);           // "Smith"
    cout << first << " " << last << endl;
}
```

Checking if a String Contains Another

- A very common operation:
 - Does this string contain some text?
- Examples:
 - Searching commands
 - Parsing input
 - Validating user data

std::string::contains (C++23)

Example

```
#include <string>
#include <iostream>
int main() {
    std::string text = "hello world";
    if (text.contains("world")) {
        std::cout << "Found it!" << std::endl;
    }
}
```

- Introduced in C++23
- Very readable
- Returns a bool

How contains Works

- Checks whether a substring exists
- Case-sensitive
- Equivalent to:
 - `find(...) != std::string::npos`
- Prefer contains when available

Uppercase and Lowercase in C++

- Unlike some languages:
 - C++ does NOT have `string.toUpperCase()`
 - Or `string.toLowerCase()`
- Instead, C++ uses character-based functions
- This gives more control (and more responsibility)

Character Case Conversion

- Provided by the C++ standard library
- Located in `<cctype>`
- Common functions:
 - `std::toupper`
 - `std::tolower`
- Operate on **single characters**

Uppercase and Lowercase: Two Approaches

- There are two common ways to change string case:
 - Build a new string (safe and simple)
 - Modify the string in place (more direct)
- Both are valid
- Which one you use depends on context, although in industry, side effects typically are not good

Uppercase (Build a New String)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
std::string upper = "";
for (char c : text) {
    upper += std::toupper(c);
}
```

- Iterates by value
- Original string is unchanged
- Very easy to reason about

Lowercase (Build a New String)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
std::string lower = "";
for (char c : text) {
    lower += std::tolower(c);
}
```

- Same pattern as uppercase
- Only the function changes

Uppercase (Modify In Place)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
for (size_t i = 0; i < text.size(); i++) {
    text[i] = std::toupper(text[i]);
}
```

- Uses indexing
- Directly changes the original string
- No references required

Lowercase (Modify In Place)

Example

```
#include <string>
#include <cctype>
std::string text = "Hello World";
for (size_t i = 0; i < text.size(); i++) {
    text[i] = std::tolower(text[i]);
}
```

- Same loop structure
- Modifies characters one at a time

Why Show Both Approaches?

- Building a new string:
 - Easier to understand
 - No side effects
- Modifying in place:
 - More efficient
 - Less memory usage

Common Functions in <cctype>

- Character classification functions:
 - `std::isalpha`: Is it a letter?
 - `std::isdigit`: Is it a number?
 - `std::isalnum`: Letter or number?
 - `std::isspace`: Whitespace?
 - `std::islower`: Lowercase letter?
 - `std::isupper`: Uppercase letter?
 - `std::ispunct`: Punctuation?
- All operate on single characters
- All return a boolean-like value

Example: Checking for Digits

Example

```
#include <cctype>
#include <iostream>

std::string s = "abc123";

for (char c : s) {
    if (std::isdigit(c)) {
        std::cout << c << " is a digit\n";
    }
}
```

- Very useful for input validation
- Common in parsing problems

Example: Counting Letters

Example

```
#include <cctype>
#include <iostream>

std::string text = "Hello 123!";
int letters = 0;

for (char c : text) {
    if (std::isalpha(c)) {
        letters++;
    }
}

std::cout << "Letters: " << letters << std::endl;
```

Example: Removing Spaces

Example

```
#include <cctype>
#include <string>

std::string text = "Hello World";
std::string result;

for (char c : text) {
    if (!std::isspace(c)) {
        result += c;
    }
}
```

- `std::isspace` detects:
 - Spaces
 - Tabs
 - Newlines

Important: Signed vs Unsigned char

- All `<cctype>` functions expect:
 - unsigned char
 - or EOF
- Passing a negative char is undefined behavior

Safer Pattern

```
for (unsigned char c : text) {  
    if (std::isdigit(c)) {  
        ...  
    }  
}
```

Real-World Uses of `<cctype>`

- Validating user input
- Parsing command-line arguments
- Tokenizing text
- Cleaning data
- Implementing simple compilers/interpreters
- The big idea is that these functions operate at the character level

- Input

Example

```
#include <iostream>
#include <string>
using std::cin, std::cout, std::endl;
int main() {
    string firstName;
    string password;
    // Ask for first name
    cout << "Enter your first name: ";
    cin >> firstName; // No spaces expected
    // Ask for password
    cout << "Enter your password: ";
    cin.ignore(); // Clear leftover newline
    getline(cin, password); // Allows spaces
    // Print inputs exactly as entered
    cout << "First Name: " << firstName << endl;
    cout << "Password: " << password << endl;
    return 0;
}
```

Strings and Numbers

- Very common task in real programs:
 - Read input as a string
 - Convert it to a number
- Also common in the other direction:
 - Convert numbers to strings for output
- C++ provides standard library tools for both

String to Number Functions

- Provided by the C++ standard library
- Located in `<string>`
- Common functions:
 - `std::stoi` (string to int)
 - `std::stod` (string to double)
 - `std::stol` (string to long)

Example: std::stoi

Example

```
#include <string>
#include <iostream>

int main() {
    std::string s = "42";
    int x = std::stoi(s);

    std::cout << x + 10 << std::endl;    // 52
}
```

Example: std::stod

Example

```
#include <string>
#include <iostream>

int main() {
    std::string s = "3.14";
    double d = std::stod(s);

    std::cout << d * 2 << std::endl;
}
```

Important Notes

- These functions expect valid numeric strings
- Examples of valid input:
 - "42"
 - "-10"
 - "3.14"
- Invalid input causes runtime errors
- We will handle errors later in the course

Number to String

- Converting numbers to strings is also common
- C++ provides:
 - `std::to_string`
- Works with:
 - `int`
 - `double`
 - other numeric types

Combining Numbers and Strings

Example

```
int score = 95;
std::string message =
    "Score: " + std::to_string(score);

std::cout << message << std::endl;
```

Command-Line Arguments Are Strings

- Arguments passed into `main` are:
 - C-style strings (`char*`)
- These are often numeric values
- We usually convert them to numbers immediately

Example: Integer Argument

Example

```
#include <string>
#include <iostream>

int main(int argc, char* argv[]) {
    if (argc < 2)
        return 1;

    int x = std::stoi(argv[1]);
    std::cout << x * 2 << std::endl;

    return 0;
}
```


Running the Program

Terminal

```
./program 21
```

- Output:
 - 42

Example: Two Numeric Arguments

Example

```
#include <string>
#include <iostream>
int main(int argc, char* argv[]) {
    if (argc < 3)
        return 1;
    int a = std::stoi(argv[1]);
    int b = std::stoi(argv[2]);
    std::cout << a + b << std::endl;
    return 0;
}
```

What Is a Stream?

- A **stream** is a flow of data
- Data flows:
 - Into your program (input stream)
 - Out of your program (output stream)
- Think of it like water flowing through pipes
- C++ models input/output using streams

The `<iostream>` Library

- Provided by the C++ standard library
- Header: `#include <iostream>`
- Defines:
 - `std::cin` (input stream)
 - `std::cout` (output stream)
 - `std::cerr` (error stream)
- "io" = input/output

Standard Streams

- Every C++ program automatically has:
 - **Standard Input** (`stdin`)
 - **Standard Output** (`stdout`)
 - **Standard Error** (`stderr`)
- In C++ these correspond to:
 - `std::cin`
 - `std::cout`
 - `std::cerr`

Output Stream: `std::cout`

- Recall: Connected to standard output
- Usually prints to the terminal
- Uses the insertion operator: `<<`
 - Recall: We can chain with this

Example

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
}
```

Input Stream: `std::cin`

- Connected to standard input
- Usually reads from the keyboard
- Uses the extraction operator: `>>`
- Recall: Can also be used in conjunction with `getline`

Example

```
#include <iostream>

int main() {
    int age;
    std::cin >> age;
    std::cout << "You are " << age << std::endl;
}
```

Standard Error: `std::cerr`

- Connected to standard error
- Used for:
 - Error messages
 - Debug output
- Separate from normal output

Example

```
std::cerr << "Something went wrong!" << std::endl;
```


Why Separate cout and cerr?

- Allows output redirection
- Example in terminal:

Redirect Output

```
./program > output.txt
```

- stdout goes to file
- stderr still prints to screen

The terminal: Redirections

- Continued

- cat
- Redirecting output with `&`
- Navigating to the home directory with `tilda`
- aliases and exports in `/.bashrc` and `/.zshrc`
- Permissions with `chown` and `chmod` (owner, group, others and read, write, execute)
 - First character of `ls -l` denotes file type

The terminal: The last useful peices

- Continued

- top (processes)
- kill -9 <pid> (kill a background process)
- du -sh (memory in dir)
- free -h (memory on system)
- watch -n 2 free -h (repeat)
- grep -IR and grep -v and pipe (searching files)
- sudo !! (run the last command with sudo)

- Streams track their internal state
- Common states:
 - Good
 - Fail
 - End-of-file (EOF)

Checking State

```
int x;  
if (std::cin >> x) {  
    std::cout << "Valid input\n";  
}
```

Looping with Input

Classic Pattern

```
int x;
while (std::cin >> x) { // Non integers end loop without
    error!
    std::cout << "Read: " << x << std::endl;
}
```

- Stops on invalid input or EOF
- Common pattern in real programs

End of Day 8 (first class)

- Streams are not limited to the terminal
- C++ supports file streams
- Header: `#include <fstream>`
- Main classes:
 - `std::ifstream` (input file stream)
 - `std::ofstream` (output file stream)
 - `std::fstream` (input and output)

Reading From a File

Example

```
#include <fstream>
#include <string>

int main() {
    std::ifstream file("data.txt");
    std::string line;

    while (std::getline(file, line)) {
        std::cout << line << std::endl;
    }
}
```


Writing To a File

Example

```
#include <fstream>

int main() {
    std::ofstream file("output.txt");
    file << "Hello file!" << std::endl;
}
```

File Open Modes

- Files can be opened with different modes
- Modes are in `std::ios`

Common Modes

```
std::ios::in      // Read
std::ios::out     // Write
std::ios::app     // Append
std::ios::binary  // Binary mode
std::ios::trunc   // Truncate file
std::ios::ate     // Start at end
```

Example

```
std::ofstream file(  
    "log.txt",  
    std::ios::app  
);  
file << "New entry\n";
```

- app preserves existing content
- Default for ofstream is out | trunc
 - Truncate wipes all existing content by default

Combining Modes

- Modes can be combined with |
- This allows multiple behaviors when opening a file

Example

```
std::fstream file(  
    "data.txt",  
    std::ios::in | std::ios::out    // Read or write  
);
```

Checking If File Opened

- It's critical to check to make sure you can actually open a file correctly
- Files may not exist, have the wrong permissions, etc.

Example

```
std::ifstream file("data.txt");  
  
if (!file) {  
    std::cerr << "Failed to open file\n";  
    return 1;  
}
```

- Files close automatically when:
 - The stream object goes out of scope
- You can close manually:

Manual Close

```
file.close(); // Flushes buffer
```

- Streams unify:
 - Keyboard input
 - Terminal output
 - Error reporting
 - File input/output
- Same operators:
 - <<
 - >>
- This is one of C++'s core abstractions

In-Class Exercise 1: Words Per Line

- You are given a file called `data.txt`
- Each line contains a sentence
- Your task:
 - Read the file line by line
 - Count how many words are in each line
 - Print: `Line X: Y words`
- A word is separated by whitespace
- Do NOT use `std::istringstream` (we will go over what this is)

Exercise 1 Solution (No stringstream)

Solution

```
#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
int main() {
    std::ifstream file("data.txt");
    if (!file) {
        std::cerr << "Failed to open file\n";
        return 1;
    }
    std::string line;
    int lineNumber = 1;
```

Exercise 1 Solution (No stringstream)

Solution

```
while (std::getline(file, line)) {
    int wordCount = 0;
    bool inWord = false;
    for (char c : line) {
        if (!std::isspace(c)) {
            if (!inWord) {
                wordCount++;
                inWord = true;
            }
        } else {
            inWord = false;
        }
    }
    std::cout << "Line " << lineNumber << ": "
              << wordCount << " words\n";
    lineNumber++;
}
```

What Is `std::istringstream`?

- Defined in `<sstream>`
- Treats a string like an input stream
- Allows use of:
 - `>>` extraction operator
- Very useful for parsing lines of text

Basic Example of istreamstringstream

Example

```
#include <sstream>
#include <string>
#include <iostream>

int main() {
    std::string line = "10 20 30";

    std::istringstream iss(line);

    int a, b, c;
    iss >> a >> b >> c;

    std::cout << a + b + c << std::endl;    // 60
}
```

Using istream to Count Words

Example

```
#include <sstream>
#include <string>

std::string line = "C++ streams are powerful";
std::istringstream iss(line);

std::string word;
int count = 0;

while (iss >> word) {
    count++;
}
```

- Extraction automatically splits on whitespace
- Much shorter than manual character scanning

Rewriting Exercise 1 with stringstream

Alternative Solution

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

int main() {
    std::ifstream file("data.txt");

    if (!file) {
        std::cerr << "Failed to open file\n";
        return 1;
    }

    std::string line;
    int lineNumber = 1;
```

Rewriting Exercise 1 with stringstream

Alternative Solution

```
while (std::getline(file, line)) {  
    std::istringstream iss(line);  
    std::string word;  
    int wordCount = 0;  
    while (iss >> word) {  
        wordCount++;  
    }  
    std::cout << "Line " << lineNumber << ": "  
              << wordCount << " words\n";  
    lineNumber++;  
}
```

```
}
```

Comparing the Two Approaches

- Manual character scanning:
 - More control
 - Teaches how whitespace works
 - More code
- `istringstream`:
 - Cleaner
 - Uses familiar `>>` operator
 - Very common in real-world parsing
- Both are correct

In-Class Exercise 2: Writing Multiple Inputs

- Your program will:
 - Ask the user how many entries they want to write
 - Loop that many times
 - Ask for a full line of input each time
 - Write each line to `entries.txt`
- You must use:
 - `std::ofstream`
 - `std::getline`
 - `std::cin.ignore`

Exercise 2 Solution

Solution

```
#include <iostream>
#include <fstream>
#include <string>
int main() {
    int count;
    std::cout << "How many entries? ";
    std::cin >> count;
    std::cin.ignore(); // clear leftover newline
    std::ofstream file("entries.txt");
    if (!file) {
        std::cerr << "Failed to open file\n";
        return 1;}
    for (int i = 0; i < count; i++) {
        std::string line;
        std::cout << "Enter text: ";
        std::getline(std::cin, line);
        file << line << std::endl;
    }    file.close(); // Class q: Is this needed?
```

Done with C refresher... mostly

- Prior to this point typically this is how far we would have gotten before the first exam
- Generally, in before the next exam, I intend to cover:
 - pointers
 - const
 - references
 - Stack vs heap
 - exceptions (including the new C++23 standard)
 - Various data structures
 - Vectors
 - Sets
 - Hashmaps
 - templates
 - Potentially... concepts (c++20)
 - Iterators
 - Smart pointers
 - And more! Like, a lot more

Done with C refresher... mostly

- Again, prior to this, the majority of the material was review from other classes
- Even still: We haven't covered pointers again
- Now we're going to be going over a lot of in depth C++ topics
- If you fall behind, please use office hours! The TAs staff 214 almost every hour of the day from 9-5, I have office hours after class, and also beach on the third floor of ITE

Quiz

End of day 9

Raw Pointers in C++

- A **pointer** is a variable that stores a memory address
- A pointer does *not* store the value directly
- Instead, it stores the location of another variable
- Pointers are one of C++'s most powerful (and dangerous) features

Why Learn Raw Pointers?

- Raw pointers appear everywhere in real code:
 - C libraries
 - Low-level systems code
 - Performance-critical code
 - Legacy C++ projects
- Smart pointers exist, but raw pointers are still important
- You need to understand pointers to understand:
 - References
 - Arrays
 - Strings (`char*`)
 - Stack vs heap

Pointer Syntax

- Pointer type: `int*`, `double*`, `string*`, etc.
- The `*` means:
 - "This variable stores an address of an `int`"
- Example:

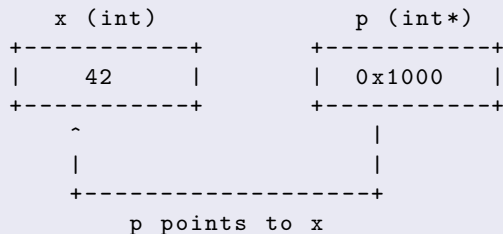
Example

```
int x = 42;  
int* p = &x;
```

- `&x` means: "address of `x`"
- `p` stores that address

Visual: Pointer Points to a Variable

Memory Diagram



- x stores the value 42
- p stores the **address** of x

Dereferencing a Pointer

- Dereferencing means: **go to the address and access the value**
- Dereference operator: *****
- Example:

Example

```
int x = 42;  
int* p = &x;  
  
std::cout << "p=" << p << "*p=" << *p << std::endl; //  
prints 42
```

- **p** is the address
- ***p** is the value stored at that address

Visual: Dereferencing

Memory Diagram

x (int)	p (int*)
+-----+	+-----+
42	0x1000
+-----+	+-----+

^
|
*p reads 42

- *p means: "follow the arrow"
- This is why pointers are sometimes called *indirection*

Changing a Value Through a Pointer

- If a pointer points to a variable, you can modify it
- This is one way functions can change caller variables

Example

```
int x = 10;  
int* p = &x;  
  
*p = 99;  
  
std::cout << x << std::endl; // prints 99
```

- `*p = 99` modifies `x`

Pointer Values Can Change

- A pointer is a normal variable
- It can be reassigned to point somewhere else

Example

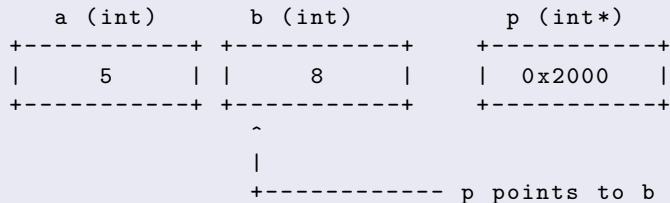
```
int a = 5;
int b = 8;

int* p = &a;
std::cout << *p << std::endl; // 5

p = &b;
std::cout << *p << std::endl; // 8
```

Visual: Pointer Reassignment

Memory Diagram



- Same pointer variable, different target

Null Pointers

- Sometimes you want a pointer that points to **nothing**
- This is called a **null pointer**
- In modern C++ use: `nullptr`

Example

```
int* p = nullptr;

if (p == nullptr) {
    std::cout << "p points to nothing\n";
}
```


Important Rule: Do NOT Dereference nullptr

- Dereferencing `nullptr` causes a crash (undefined behavior)
- This is one of the most common pointer bugs

Bad Example

```
int* p = nullptr;  
std::cout << *p << std::endl; // CRASH
```

- You must check pointers before dereferencing if they might be null

Pointers and Functions

- A pointer parameter lets a function modify a variable
- This is a C-style technique (C++ often prefers references)

Example

```
#include <iostream>

void setToZero(int* p) {
    *p = 0;
}

int main() {
    int x = 100;
    setToZero(&x);
    std::cout << x << std::endl;    // 0
}
```

Visual: Pointer Parameter

Memory Diagram

main():

x (int)

+-----+

| 100 |

+-----+

^

|

*p modifies x

setToZero(p):

p (int*)

+-----+

| &x |

+-----+

- The function receives a copy of the address
- But it still points to the same variable

Pointer Mistake #1: Uninitialized Pointers

- If you declare a pointer but do not initialize it...
- It contains garbage memory
- Dereferencing it is undefined behavior

Bad Example

```
int* p;           // uninitialized!  
*p = 10;          // undefined behavior
```

- Always initialize pointers
- Use `nullptr` if you don't have a target yet

Safer Pointer Initialization

Good Pattern

```
int* p = nullptr;

if (p != nullptr) {
    std::cout << *p << std::endl;
}
```

- `nullptr` is a real C++ keyword (not an int)
- Safer than using 0 or NULL

Pointer Mistake #2: Dangling Pointers

- A **dangling pointer** points to memory that is no longer valid
- Example: pointer to a local variable after it goes out of scope

Bad Example

```
int* bad() {  
    int x = 10;  
    return &x; // returning address of local variable  
}
```

- After `bad()` returns, `x` is destroyed
- The pointer points to "dead memory"

Visual: Dangling Pointer

Memory Diagram

Inside bad():

```
x (int)
```

```
+-----+
```

```
|    10    |
```

```
+-----+
```

After return:

```
x is gone (stack reused)
```

```
pointer still holds old address
```

```
dereferencing becomes undefined behavior
```

- This bug is extremely common in beginner code

Arrays and Pointers

- In C++, arrays and pointers are closely related
- The array name behaves like a pointer to the first element
- Example:

Example

```
int arr[5] = {10, 20, 30, 40, 50};  
  
int* p = arr; // same as &arr[0]
```

- p now points to the first element

Visual: Array in Memory

Memory Diagram

```
arr:
+---+---+---+---+---+
| 10 | 20 | 30 | 40 | 50 |
+---+---+---+---+---+
  ^
  |
p points to arr[0]
```

- Arrays are stored contiguously in memory
- Each element is right next to the next one

Pointer Arithmetic

- You can move a pointer forward and backward
- Adding 1 moves to the next element
- Example:

Example

```
int arr[3] = {10, 20, 30};  
int* p = arr;  
  
std::cout << *p << std::endl;      // 10  
std::cout << *(p + 1) << std::endl; // 20  
std::cout << *(p + 2) << std::endl; // 30
```

- Pointer arithmetic automatically scales by element size

Visual: Pointer Arithmetic Over an Array

Memory Diagram

```
arr:
+---+---+---+---+---+
| 10 | 20 | 30 | 40 | 50 |
+---+---+---+---+---+
```

```
p      -> arr[0]
p + 1  -> arr[1]
p + 2  -> arr[2]
p + 3  -> arr[3]
p + 4  -> arr[4]
```

- `*(p + i)` accesses element `i`

Iterating Over an Array Using Only Pointers

- You can loop using a pointer instead of an index
- Start at the beginning of the array
- Stop at one-past-the-end

Example

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    int* p = arr;
    int* end = arr + 5;

    for (; p < end; p++) {
        std::cout << *p << std::endl;
    }
}
```

Visual: Pointer Loop Movement

Memory Diagram

```
arr:
+---+---+---+---+---+
| 10 | 20 | 30 | 40 | 50 |
+---+---+---+---+---+
  ^
  |
  p starts here
```

```
Each loop:
    print *p
    p++
```

```
Stop when:
    p == arr + 5
```

- `arr + 5` is one past the last element
- This is a very common pattern in C and C++

Pointer Iteration: Cleaner Version

- Same loop, but written more compactly
- This looks similar to iterator-style loops in C++

Example

```
int arr[5] = {10, 20, 30, 40, 50};  
  
for (int* p = arr; p < arr + 5; p++) {  
    std::cout << *p << std::endl;  
}
```

- This uses no indexing at all
- The pointer itself is the loop variable

Important: Don't Walk Past the End

- Pointer iteration is powerful, but dangerous
- If you go past the end of the array:
 - Undefined behavior
 - You might read garbage
 - You might crash
- This is why C++ prefers containers like `std::vector`

Bad Example

```
int arr[3] = {1, 2, 3};  
  
for (int* p = arr; p <= arr + 3; p++) {  
    std::cout << *p << std::endl; // BUG at the end  
}
```

- The condition must be `p < arr + 3`, not `<=`

The High-Level Memory Model in C++

- A running C++ program uses multiple regions of memory
- At a high level, we usually talk about:
 - **Static storage** (globals, static variables)
 - **Stack** (local variables, function calls)
 - **Heap** (dynamic memory: `new`, `delete`)
- Understanding these explains:
 - Why pointers can dangle
 - Why returning `&x` is wrong
 - Why memory leaks happen

Visual: Memory Layout (Simplified)

Conceptual Diagram

```
+-----+
| Static Storage          |
| (globals, static variables) |
+-----+
| Heap (dynamic memory)   |
| grows upward (conceptually) |
| new / delete            |
+-----+
| Stack (function calls)  |
| grows downward (conceptually) |
| local variables         |
+-----+
```

- This is simplified, but good enough for learning

Static Storage (Globals)

- Static storage contains:
 - Global variables
 - `static` variables
- These exist for the **entire lifetime of the program**
- They are created before `main()` starts
- They are destroyed after `main()` finishes

Why Static Storage Is Usually Avoided

- Globals make programs harder to reason about
- They create hidden dependencies
- They break modular design
- They cause:
 - Unexpected side effects
 - Bugs that depend on program order
 - Testing nightmares
- They are also not very thread friendly
- **Globals are bad. Don't use them!**

Static Storage: A Quick Example

Example

```
#include <iostream>

int globalCount = 0; // static storage

void foo() {
    globalCount++;
}

int main() {
    foo();
    foo();
    std::cout << globalCount << std::endl; // 2
}
```

- This works, but it creates global state
- Most modern code avoids this

The Stack

- The **stack** is used for:
 - Function calls
 - Local variables
 - Return addresses
- Stack memory is:
 - Fast
 - Automatic
 - Freed when a function (or scope) ends
- This is why local variables do not need delete

The Stack

- The stack is also fairly unique to low-level languages!
- Languages like Java and Python put everything on the heap
- This is a lot less efficient than how C, C++, Rust, etc does it

Visual: Stack Frames

Conceptual Diagram

Stack (top grows downward)

```
+-----+
| foo() frame   |
| local variables |
+-----+
| main() frame  |
| local variables |
+-----+
| (older frames...) |
+-----+
```

- Each function call pushes a new **stack frame**
- Returning from a function pops that frame

Stack Example: Local Variables

Example

```
#include <iostream>

void foo() {
    int x = 10;    // stack variable
    std::cout << x << std::endl;
} // x is destroyed here

int main() {
    foo();
}
```

- x exists only while foo() is running

Stack Bug: Dangling Pointer (Again)

- The stack is temporary
- Returning a pointer to a local variable is a classic bug

Bad Example

```
int* bad() {  
    int x = 123;    // stack variable  
    return &x;      // BUG  
}
```

- After the function returns, `x` is gone

Visual: Why Returning &x Is Wrong

Memory Diagram

Inside bad():

```
+-----+  
| x = 123      |  
+-----+
```

After return:

```
+-----+  
| stack reused  |  
| (x no longer valid)|  
+-----+
```

Pointer still holds old address

Dereferencing becomes undefined behavior

The Heap

- The **heap** is used for dynamic memory allocation
- Heap memory is allocated manually using:
 - `new`
 - `delete`
- Heap memory:
 - Lives until you free it
 - Does NOT disappear when a function ends
 - Is slower than stack memory

Heap Example: new and delete

Example

```
#include <iostream>

int main() {
    int* p = new int(42); // heap allocation

    std::cout << *p << std::endl;

    delete p; // free heap memory
}
```

- p is on the stack
- The int is on the heap

Visual: Pointer to Heap Memory

Memory Diagram

Stack:

```
+-----+  
|  p    |  
| 0x9000 |  
+-----+
```

Heap:

```
+-----+  
|  42   |  
| (int) |  
+-----+
```

----->

- Stack holds the pointer
- Heap holds the allocated object

Heap Memory Does Not Go Away Automatically

- Stack memory is cleaned up automatically
- Heap memory is **not**
- If you forget `delete`, the memory stays allocated
- This is called a **memory leak**

Memory Leak Example

```
int* p = new int(5);  
  
// forgot delete
```

Heap Bug: Use After Free

- Another common bug:
 - Free memory
 - Keep using the pointer anyway
- This is called **use-after-free**

Bad Example

```
int* p = new int(99);  
delete p;  
  
std::cout << *p << std::endl; // undefined behavior
```

Fix: Reset Pointers After delete

- After delete, the pointer becomes dangling
- A common safety trick:
 - Set it to `nullptr`

Better Pattern

```
int* p = new int(99);  
delete p;  
p = nullptr;    // safer  
  
if (p != nullptr) {  
    std::cout << *p << std::endl;  
}
```


Stack vs Heap (Key Differences)

Feature	Stack	Heap
Allocation	Automatic	Manual (<code>new</code>)
Deallocation	Automatic	Manual (<code>delete</code>)
Speed	Fast	Slower
Lifetime	Function scope	Until freed
Common bugs	Dangling refs	Leaks, double free

To drive this home: The memory model

- C++ Memory model is the same as in C
- There's three pools of memory:
 - Static (lifetime of the program)
 - Stack (lifetime of the function/scope)
 - Heap (you choose the lifetime)
- Each pool of memory has behaves differently
 - Everything stay in memory for different amounts of time
 - Memory get allocated (reserved/created) and deallocated (freed back to the OS) differently in each pool

- In the Static memory pool:
 - Memory is allocated at compile time
 - There is only one copy. It's not dynamic!
 - Uses the static keyword
 - Very rare because static globals are not thread friendly and generally are bad practice
- Lifetime:
 - Statics live the entire execution of the program
- To summarize:
 - Easy to use
 - Restrictive
 - Not thread-friendly
 - Often an anti-pattern

- In the Stack memory pool:
 - Memory comes from local variables of the scope, such as in functions or for loops
 - Allocated automatically when entering into scope
 - De-allocated automatically when you leave the scope
- What about if we use recursion, where a function calls itself?
 - With each recursive call, there is a new scope, so there are new stack variables
- In general:
 - Stack memory is easy to manage
 - There is variable amounts of memory over time
 - The scope controls the lifetime (functions, loops, etc)

- In the heap memory pool:
 - Memory comes from on-the-fly allocations
 - Allocations stand for reserving some memory to store a variable in that memory.
 - Deallocation means you "release" that memory to the OS
- Two simple ways to do this in C++
 - new (allocates a block of bytes)
 - delete (frees a previously allocated block)
- Who's in charge?
 - The programmer for both allocation and deallocation
- What is the lifetime of heap memory blocks?
 - Forever, as long as they are not freed
 - Even beyond the lifetime of the program... don't make this mistake!

- When taking the address of:
 - A static?
 - The address is never going to go "bad"
 - The static lives as long as the program!
 - A stack variable?
 - The address is valid as long as the variable is!
 - When the function returns, the address becomes bogus
 - To reiterate: Don't use a pointer to a stack variable that has gone out of scope!
 - A heap variable?
 - The address is valid as long as the variable is!
 - The address goes bad when the variable is explicitly deallocated/freed
 - To reiterate: Don't use a pointer to a heap variable that has been freed!

Dynamic Arrays on the Heap

- Sometimes you need an array whose size is not known at compile time
- Stack arrays require a constant size:
 - `int arr[10];` (size must be known)
- Heap arrays can be created with `new[]`
- Heap arrays must be freed with `delete[]`

Allocating an Array with new[]

Example

```
#include <iostream>

int main() {
    int n = 5;
    int* arr = new int[n];    // heap array

    arr[0] = 10;
    arr[1] = 20;

    std::cout << arr[0] << std::endl;

    delete[] arr;    // free heap array
}
```

- arr is a pointer
- The array lives on the heap

Visual: Heap Array Allocation

Memory Diagram

Stack:

```
+-----+  
|  arr   |  
| 0x8000 |  
+-----+
```

Heap:

```
+-----+-----+-----+-----+-----+  
|  ??  |  ??  |  ??  |  ??  |  ??  |  
+-----+-----+-----+-----+-----+  
      0       1       2       3       4
```

- `new int[n]` allocates `n` contiguous ints
- The values are uninitialized (garbage) by default

Value-Initializing a Heap Array

- By default, `new int[n]` does NOT initialize the values
- If you want them all to start as 0, use:
 - `new int[n]{};`

Example

```
int n = 5;

int* arr1 = new int[n];      // uninitialized
int* arr2 = new int[n]{};    // all zeros

delete[] arr1;
delete[] arr2;
```

Iterating Over a Heap Array

- Heap arrays work just like normal arrays
- You can use indexing
- Or pointer iteration

Index Loop

```
int n = 5;
int* arr = new int[n]{};

for (int i = 0; i < n; i++) {
    arr[i] = i * 10;
}

delete[] arr;
```

Pointer Iteration Over a Heap Array

- This works the same way as stack arrays
- The pointer points to the first element

Pointer Loop

```
int n = 5;
int* arr = new int[n]{};

int* p = arr;
int* end = arr + n;

for (; p < end; p++) {
    *p = 7;
}

delete[] arr;
```

IMPORTANT: `delete[]` vs `delete`

- C++ has two different delete operations:
 - `delete p;`
 - `delete[] p;`
- If you allocate with:
 - `new T` → use `delete`
 - `new T[n]` → use `delete[]`

Common Bug: Deleting an Array Incorrectly

- If you allocate with `new[]`
- But free with `delete`
- This is undefined behavior

Bad Example

```
int* arr = new int[5];  
  
delete arr;    // BUG (should be delete[])
```

- Sometimes it "seems to work"
- That makes it even more dangerous

Common Bug: Memory Leak with Arrays

- Forgetting `delete[]` leaks the entire array
- The program loses access to the memory

Memory Leak Example

```
int* arr = new int[1000];  
  
// forgot delete[]
```

- This is why modern C++ prefers `std::vector`, which we'll get into shortly

End of Day 10

Heap Allocation: Arrays

- When working with arrays, use brackets for new and delete
- Then a block of memory for the entire array is allocated
- The size can be dynamically determined at runtime
- Make sure you deallocate the entire array, not just the first block (use brackets)

Example

```
#include <iostream>
using std::cout, std::endl;
int main(){
    int* px = new int[10]; // 10 blocks of memory allocated
    *px = 20; // Overwrites the first block of memory with
              20
    cout << "px= " << *px << endl;
    cout << "px= " << px << endl;
    // Don't forget!! (now with brackets)
    delete[] px;
}
```

Heap Allocation: Arrays part 3

- Let's go over a similar example, but writing using pointers
- First example of using a for loop as well - hopefully CSE3100 refresher
- Remember, picture 10 contiguous blocks!

Example

```
#include <iostream>
using std::cout, std::endl;
int main(){
    int* px = new int[10]; // 10 blocks of memory allocated
    for (int i=0; i<10; ++i){
        // Remember, arrays are pointers to the first block
        px[i] = i; // So pointer/array syntax are the same
    }
    for (int i=0; i<10; ++i){
        cout << "px[" << i << "]=" << px[i] << endl;
    }
    cout << "px= " << px << endl;
    delete[] px;
}
```

Heap Allocation: Arrays part 4

- Let's change our example to use pointer arithmetic
- Remember, adding one to a pointer just points to the next block

Example

```
#include <iostream>
using std::cout, std::endl;
int main(){
    int* px = new int[10]; // 10 blocks of memory allocated
    for (int* cx = px; cx < px + 10; cx++){
        // Subtracting memory address blocks
        // You can figure out how far apart 2 ptrs are (same
        type)
        *cx = cx - px;
    }
    for (int i=0; i<10; ++i){
        cout << "px[" << i << "]=" << px[i] << endl;
    }
    cout << "px= " << px << endl;
    delete[] px;
}
```

Stack vs Heap

- One thing that helped me a lot to understand the stack vs the heap, was to look at examples surrounding arrays
 - What does it look like when we have an array on a stack, but the values are on the heap?
 - What does it look like when we have an array on the heap, but the values are on the stack?
 - What does it look like when we have an array on the heap, and values on the heap?
- Let's dive into these examples

Pattern 1: Heap Array, Stack Values

```
int a = 10, b = 20, c = 30;

int* arr = new int[3]; // array on heap

arr[0] = a;
arr[1] = b;
arr[2] = c;

delete[] arr;
```

Pattern 1: What's Happening?

- The array itself is on the **heap**
- The values come from **stack variables**
- Values are **copied** into the heap array
- Only one allocation $\Rightarrow$ one `delete[]`

Pattern 2: Stack Array, Heap Values

```
int* a = new int(10);  
int* b = new int(20);  
int* c = new int(30);  
  
int* arr[3] = {a, b, c}; // array on stack  
  
delete a; // Or delete arr[0];  
delete b;  
delete c;
```

Pattern 2: What's Happening?

- The array is on the **stack**
- Each element is a pointer to the **heap**
- The array is cleaned up automatically
- Each heap allocation must be deleted manually

Pattern 3: Heap Array + Heap Values

```
int** arr = new int*[3];

for (int i = 0; i < 3; i++) {
    arr[i] = new int(i * 10);
}

for (int i = 0; i < 3; i++) {
    delete arr[i];
}

delete[] arr;
```

Pattern 3: What's Happening?

- The array is on the **heap**
- Each element points to **heap memory**
- Two levels of allocation
- Must delete:
 - Inner objects first
 - Then the outer array

Quiz: Pointers

Write a C++ program that implements this function:

```
int* create_reversed_copy(int* arr, int size);
```

Requirements for main:

- 1 In your main function, create a small, stack-allocated integer array to use as a test case (e.g., `int original[] = {10, 20, 30, 40, 50};`).
- 2 Call your `create_reversed_copy` function, storing the returned pointer in a new variable.
- 3 Write a loop to print the elements of the reversed array to verify your function works correctly.
- 4 **Most importantly:** Free the heap memory allocated by your function using `delete[]` to prevent a memory leak.

In-Class Exercise: Pointers (Solution)

Complete Program

```
#include <iostream>

int* create_reversed_copy(int* arr, int size) {
    int* copy = new int[size];
    for (int i = 0; i < size; i++) {
        copy[i] = arr[size - 1 - i];
    }
    return copy; // caller must delete[]
}

int main() {
    int original[] = {10, 20, 30, 40, 50};
    int size = sizeof(original) / sizeof(original[0]);
    int* reversed = create_reversed_copy(original, size);
    for (int i = 0; i < size; i++) {
        std::cout << reversed[i] << " ";
    }
    std::cout << std::endl;
    delete[] reversed;
    return 0;
}
```

Why Modern C++ Avoids `new[]` Arrays

- Raw heap arrays are difficult to manage safely
- Common problems:
 - Forgetting `delete[]`
 - Using `delete` instead of `delete[]`
 - Losing the pointer (leak)
 - Not tracking the size correctly
- Prefer:
 - `std::vector<T>`
 - `std::array<T, N>` (fixed size)

`std::vector`: The Real Array of C++

- In C++, the most common array-like type is `std::vector`
- A vector is a **dynamic array**
- It can grow and shrink at runtime
- It stores elements contiguously in memory (like an array)
- It lives in the standard library: `#include <vector>`

Why `std::vector` Is Safer Than C Arrays

- C arrays have major problems:
 - No built-in size tracking
 - Easy to overflow
 - No resizing
 - Arrays decay into pointers easily
 - Hard to return from functions safely
- `std::vector` solves most of this:
 - Tracks its own size
 - Resizes automatically
 - Works well with functions
 - Automatically manages heap memory

A C Array Does NOT Know Its Own Size

Example

```
#include <iostream>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    // Works ONLY in this scope:
    std::cout << sizeof(arr) << std::endl;           // 20
    bytes
    std::cout << sizeof(arr) / sizeof(arr[0]) << std::endl;
    // 5
}
```

- Once passed into a function, arr becomes a pointer
- The size information is lost

Vectors Always Know Their Size

Example

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};

    std::cout << v.size() << std::endl; // 5
}
```

- `v.size()` always works
- Even if you pass the vector into a function

Vectors Automatically Manage Memory

- A vector stores its elements on the heap internally
- But you do NOT manually call `new` or `delete`
- The vector cleans itself up automatically
- This is called:
 - **RAII** (Resource Acquisition Is Initialization)

Visual: C Array vs Vector (High Level)

Conceptual Diagram

C array (stack):

```
+-----+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+-----+
```

std::vector (stack + heap):

Stack:

```
+-----+
| v (vector object) |
| size = 5           |
| capacity = 5       |
+-----+
```

Heap:

```
+-----+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+-----+
```

- The vector object is small and lives on the stack
- The actual elements live on the heap

What a Vector Stores Under the Hood

- A vector is NOT just the elements
- Internally, it stores (conceptually):
 - A pointer to heap memory
 - A size (how many elements are used)
 - A capacity (how many elements fit before resizing)
- When the vector grows beyond capacity:
 - It allocates a bigger array
 - Copies/moves all elements
 - Frees the old memory

Visual: Vector Size vs Capacity

Conceptual Diagram

```
v.size() = 3  
v.capacity() = 6
```

Heap storage:

10	20	30	??	??	??
0	1	2	3	4	5

- Size = real elements in use
- Capacity = reserved space

Vector Growth: Why Copying Happens

- When you push elements, vectors grow automatically
- But resizing is expensive:
 - allocate new memory
 - copy elements
 - free old memory
- Vectors usually grow by doubling capacity
- This makes average push-back fast

Example: push_back and Growth

Example

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;

    for (int i = 0; i < 10; i++) {
        v.push_back(i);
        std::cout << "size=" << v.size()
                  << " capacity=" << v.capacity()
                  << std::endl;
    }
}
```

- You will see capacity jump occasionally
- That jump means a reallocation occurred

Creating Vectors

- Common ways to create vectors:

Examples

```
std::vector<int> a;           // empty
std::vector<int> b(5);       // 5 zeros
std::vector<int> c(5, 7);    // 5 copies of 7
std::vector<int> d = {1, 2, 3}; // initializer list
```

- Vectors default-initialize their elements

Accessing Elements in a Vector

- Vectors support indexing like arrays:
 - `v[i]`
- Vectors also support bounds-checking:
 - `v.at(i)`

Example

```
std::vector<int> v = {10, 20, 30};  
  
std::cout << v[1] << std::endl;    // 20  
std::cout << v.at(1) << std::endl; // 20
```

`v[i]` vs `v.at(i)`

- `v[i]`:
 - No bounds checking
 - Out-of-range access is undefined behavior
 - Faster
- `v.at(i)`:
 - Checks bounds
 - Throws an exception if out of range
 - Safer for beginners

Common Vector Functions

- `v.size()` = number of elements
- `v.empty()` = is it empty?
- `v.push_back(x)` = add element to end
- `v.pop_back()` = remove last element
- `v.clear()` = remove all elements
- `v.front()` = first element
- `v.back()` = last element

Example: Common Vector Functions

Example

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};

    v.push_back(4);    // {1,2,3,4}
    v.pop_back();      // {1,2,3}

    std::cout << v.front() << std::endl; // 1
    std::cout << v.back() << std::endl;  // 3

    std::cout << v.size() << std::endl;  // 3
}
```

End of day 11 Class 1

Iterating Over a Vector

- Vectors are easy to loop over
- Index loop:
- Range-based loop:

Example

```
std::vector<int> v = {10, 20, 30};

// Index loop
for (size_t i = 0; i < v.size(); i++) {
    std::cout << v[i] << std::endl;
}

// Range-based loop
for (int x : v) {
    std::cout << x << std::endl;
}
```

Vectors and Pointers

- A vector stores elements contiguously (like an array)
- This means you can still use pointers if you want
- The pointer to the first element is:
 - `&v[0]` (if not empty)
 - `v.data()` (preferred)

Example

```
std::vector<int> v = {10, 20, 30};  
int* p1 = &v[0];  
int* p2 = v.data();  
int* end = v.data() + v.size();  
for (; p2 < end; p2++) {  
    std::cout << *p2 << std::endl;  
}
```

Important Warning: Vector Reallocation

- If a vector grows, it may reallocate its internal array
- If that happens:
 - All pointers to elements become invalid
 - All references to elements become invalid
- This is a major difference from C arrays

Example

```
std::vector<int> v;  
v.push_back(1);  
  
int* p = v.data();  
  
v.push_back(2); // may reallocate  
  
// p might now be dangling!
```


Fix: Reserve Space When You Know the Size

- If you know how many elements you will store:
 - use `v.reserve(n)`
- This reduces reallocations
- This keeps pointers valid (until you exceed reserved space)

Example

```
std::vector<int> v;  
v.reserve(1000); // allocate once  
  
for (int i = 0; i < 1000; i++) {  
    v.push_back(i);  
}
```

Important: Vector Pointer Invalidation

- With `new[]` arrays:
 - pointers stay valid until you delete the array
- With vectors:
 - pointers stay valid until the vector reallocates
- Reallocation can happen on:
 - `push_back`
 - `insert`
 - resizing beyond capacity

Important Point:

- Strings and Vectors are typically created on the stack
- But even though the outer container is on the stack, it still has memory managed on the heap under the hood
- Strings and Vectors are essentially convenient wrappers around managing C objects for C-Style strings and C-style arrays
- When Strings and Vectors go out of scope (when they are on the stack), they call delete and deallocate their internal objects.
- Strings and Vectors handle new and delete for you.

End of day 11 Class 2 (final version)

Pass by Value vs Pointer vs Reference

- In C++, there are three main ways to pass data into functions:
 - **Pass by value** (copy)
 - **Pass by pointer** (explicit address)
 - **Pass by reference** (alias)
- We will now compare all three using the same problems

Example 1: addOne (Pass by Value)

```
void addOne(int x) {  
    x = x + 1;    // modifies only the copy  
}  
  
int main() {  
    int a = 5;  
    addOne(a);  
    std::cout << a << std::endl;    // still 5  
}
```

- A copy of `a` is passed into the function
- The original variable is unchanged

Example 1: addOne (Pointer)

```
void addOne(int* x) {  
    *x = *x + 1;    // dereference to modify original  
}  
  
int main() {  
    int a = 5;  
    addOne(&a);  
    std::cout << a << std::endl;    // now 6  
}
```

- We pass the address using &a
- Must use *x to access the value
- Let's now go over a third way of doing this: References!

How References Work

- A reference is an **alias** for another variable
- It does **not create a new object**
- It simply provides another name for the same memory location

```
int x = 5;  
int& r = x;
```

```
r = 10;
```

- Both names refer to the same value

```
std::cout << x << std::endl; // 10  
std::cout << r << std::endl; // 10
```

- Changing the reference changes the original variable

References vs Pointers

Pointer

```
int x = 5;  
int* p = &x;  
*p = 10;
```

- Stores a memory address
- Requires * to dereference
- Can be reassigned
- Can be nullptr

Reference

```
int x = 5;  
int& r = x;  
r = 10;
```

- Alias for an existing variable
- No dereferencing needed
- Cannot be reassigned
- Cannot be null

Important Properties of References

- References behave like **aliases** for existing variables (similar to pointers)
- References must be initialized when created

```
int x = 5;
int& r = x;    // Good
int& bad;      // Error! Must initialize!
r = 1 // Now you have changed x!
```

- References cannot be **null**
- References cannot be **reassigned**

```
int x = 5;
int y = 10;
int z = 20;
int& r = x;
r = y;    // assigns y's value to x. You change the value of
          whatever the ref refers to.
          // reference still refers to x! No reassignment!
x = z;    // Both x and r now change. y and z are unchanged.
```

Helpful Way to Think About References

A reference is basically a pointer that the compiler automatically dereferences for you and never lets be null.

- References behave like the original variable
- You don't need * or & after initialization
- This makes code cleaner and safer for many function parameters
- In modern C++:
 - Use **references** for most function parameters
 - Use **pointers** when null or pointer behavior is needed

Example 1: addOne (Reference)

```
void addOne(int& x) {  
    x = x + 1;    // directly modifies original  
}  
  
int main() {  
    int a = 5;  
    addOne(a);  
    std::cout << a << std::endl;    // now 6  
}
```

- No & or * needed at call site
- Cleaner syntax than pointers
- With the benefits of pointers. No big object copying!

Example 2: swap (Pass by Value)

```
void swap(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int x = 1, y = 2;  
    swap(x, y);  
    // x = 1, y = 2 (unchanged)  
}
```

- Only copies are swapped
- Original variables are unchanged

Example 2: swap (Pointer)

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 1, y = 2;  
    swap(&x, &y);  
    // x = 2, y = 1  
}
```

- Addresses are passed in
- Must dereference to access values

Example 2: swap (Reference)

```
void swap(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int x = 1, y = 2;  
    swap(x, y);  
    // x = 2, y = 1  
}
```

- Same behavior as pointers
- Much cleaner syntax
- No big object copying!

Example 3: doubleAll (Pass by Value)

```
#include <vector>

void doubleAll(std::vector<int> v) {
    for (int x : v) {
        x *= 2;
    }
}

int main() {
    std::vector<int> v = {1,2,3};
    doubleAll(v);
    // v unchanged
}
```

- Entire vector is copied (expensive!)
- Changes affect only the copy

Example 3: doubleAll (Pointer)

```
#include <vector>

void doubleAll(std::vector<int>* v) {
    for (int& x : *v) {
        x *= 2;
    }
}

int main() {
    std::vector<int> v = {1,2,3};
    doubleAll(&v);
    // v = {2,4,6}
}
```

- Pointer to vector is passed
- Must dereference to access container

Example 3: doubleAll (Reference)

```
#include <vector>

void doubleAll(std::vector<int>& v) {
    for (int& x : v) {
        x *= 2;
    }
}

int main() {
    std::vector<int> v = {1,2,3};
    doubleAll(v);
    // v = {2,4,6}
}
```

- No copying!
- Clean and efficient

Example 4: Framed Message (Pass by Value)

```
#include <iostream>
#include <string>

void printFrame(std::string message) {
    int width = message.size() + 4;

    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;

    std::cout << "* " << message << " *" << std::endl;

    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;
}
```

- The string is copied into the function

Example 4: Framed Message (Pointer)

```
#include <iostream>
#include <string>

void printFrame(std::string* message) {
    int width = message->size() + 4;

    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;

    std::cout << "* " << *message << " *" << std::endl;

    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;
}
```

- Uses pointer syntax (-> and *)

What Does `->` Actually Do?

- The `->` operator is just shorthand
- This:

```
message->size()
```

- Is exactly the same as:

```
(*message).size()
```

- Steps:
 - `*message` → get the object being pointed to
 - `.size()` → call function on that object

Why -> Exists

- Without ->, pointer code gets messy:

```
(*message).size()  
(*message).length()  
(*message).clear()
```

- With ->, it's cleaner:

```
message->size()  
message->length()  
message->clear()
```

- It avoids parentheses and improves readability

Example 4: Framed Message (Reference)

```
#include <iostream>
#include <string>
void printFrame(std::string& message) {
    int width = message.size() + 4;
    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;
    std::cout << "* " << message << " *" << std::endl;
    for (int i = 0; i < width; i++) std::cout << "*";
    std::cout << std::endl;
}
```

- Same efficiency as pointer version
 - Generally references when compiled to machine code are simply pointers
 - Although the C++ standard does not require this
- Much cleaner syntax

Pointers vs References in Practice

- In modern C++, references are usually preferred for function parameters
 - Cleaner syntax
 - Cannot be null
 - Safer for beginners
- However, pointers are still necessary in some situations:
 - When a parameter may be **null**
 - When doing **pointer arithmetic**
 - When working with **arrays or dynamic memory**
 - When implementing certain **data structures** (think: linked lists)
 - If you don't know linked lists, no worries! We'll dive deeper on these later on
- Generally:
 - Use **references** by default, especially in function parameters
 - Use **pointers** when you specifically need pointer behavior

Pointers vs References, Python

- Pointers are more explicit, you always know what is a pointer due to *
- References are more implicit. Pointer logic is abstracted away, and you only know a is a ref due to the func definition
- This is in contrast to other languages like Python
 - In Python, all mutable objects are passed by reference
 - Everything else is passed by value
- In C++ you explicitly choose!
 - Defaults to by value

Introducing const

- `const` means **"this value cannot be changed"**
- Unlike in other languages like Python, you can simply add the keyword `const` and make a variable immutable
- It helps:
 - Prevent bugs
 - Document intent (way more readable)
 - Enable compiler optimizations
 - Junior developer proof!

Basic const Example

```
const int x = 10;  
x = 20;    // ERROR
```

- Once initialized, x cannot be modified

const with References

```
void print(const std::string& s) {  
    std::cout << s << std::endl;  
}
```

- No copy is made
- Function promises not to modify s
- This is the default way to pass large objects into functions

Why This Matters

```
void foo(std::vector<int>& v);
```

- This function **can modify** the vector

```
void foo(const std::vector<int>& v);
```

- This function **cannot modify** the vector

const Prevents Bugs

```
void printSize(const std::vector<int>& v) {  
    // v.push_back(10); // ERROR  
    std::cout << v.size() << std::endl;  
}
```

- Compiler stops accidental modification

End of Day 12 for class 1

Framed Message with const

```
void printFrame(const std::string& message) {  
    int width = message.size() + 4;  
  
    for (int i = 0; i < width; i++) std::cout << "*";  
    std::cout << std::endl;  
  
    std::cout << "* " << message << " *" << std::endl;  
  
    for (int i = 0; i < width; i++) std::cout << "*";  
    std::cout << std::endl;  
}
```

- Let's take our framed message function here
- Imagine if this function was 1000 lines long
- You don't know whether or not your reference is modifying the original message or not
 - In comes const!
 - Now you and all developers know, and accidental changes are prevented
- This version is the ideal version of this function
- You'll see this const reference for parameters A LOT

Framed Message with const

```
#include <iostream>
#include <string>
using std::cout, std::endl;
int main(){
    std::string s = "Hello World";
    for (const char& c : s){
        cout << c << endl;
    }
}
```

- Typically you also use const references for range based for loops
- Unless you explicitly need mutation or copying, this is the default way of doing a range-based for loop
- Range based for loops are generally preferred over normal for loops
 - With a range based for loop, the data structure no longer matters
 - You can go from a vector to a string to a linked list with the same loop
 - We'll get into linked lists later on

End of Day 12 final version

Pointers and const

- Now that we've gone over const and references, how does this work with pointers?
- We know that pointers give us a memory address that we can read and write from
- So does const
 - Make the memory address const?
 - Or perhaps the value at the address?
- You choose!

Pointers and const

- Remember, integers aren't like arrays, they aren't already pointers
 - So we must use & when getting the address to assign it to a pointer to that address
- We set the pointer to be a const
- Here the value that the pointer is pointing to is constant
 - We can no longer assign to the value px is pointing to. It will error!

Const Pointer Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    int x = 5;
    const int* px = &x;
    cout << "px= " << px << endl;
    cout << "*px= " << *px << endl;
    *px = 50;    // This will error out! Not allowed!
    return 0;
}
```

Pointers and const

- Even though the value px points to is const... x is not!
 - We can still change x
 - This is a legitimate operation
- Remember, px is simply pointing to the memory address of x
 - The value that px points to is the value of x!

Const Pointer Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    int x = 5;
    const int* px = &x;
    cout << "px= " << px << endl;
    cout << "*px= " << *px << endl;
    x = 50;    // Allowed
    *px = 50;  // This will error out! Not allowed!
    return 0;
}
```

Pointers and Const

- Currently, the value that px points to is const
- But what if we wanted to make the memory address of px const instead?
- All we have to do is move const to the other side!

Const Pointer Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    int x = 5;
    // Old code: const int* px = &x;
    int* const px = &x;
    cout << "px= " << px << endl;
    cout << "*px= " << *px << endl;
    *px = 50; // This will no longer error
    px = px + 1; // This will error
}
```

- To determine what is const, read backwards:
 - px is a constant pointer to an int
 - Unlike the old code, where px was a pointer to an int that is constant
- Depending on where we put the const, we can:
 - Make the memory address of our pointer constant
 - Make the value at the memory address of our pointer constant

Pointers and Const

- So could we set both at once? Yes!
- Remember, we read const expressions from right to left. Here:
 - px is a constant pointer to an int that is also constant

Const Pointer Example

```
#include <iostream>
using std::cout, std::endl;
int main() {
    int x = 5;
    // const int* px = &x;  // Keeping value constant
    // int* const px = &x;  // Keeping ptr (address) const
    const int* const px = &x;  // Keeping ptr & value const
    cout << "px= " << px << endl;
    cout << "*px= " << *px << endl;
    *px = 50;  // This will error due to const
    px = px + 1;  // This will error due to const
    x = 10;  // This still works
}
```


const pointer summary:

Declaration	Meaning
<code>const int* p</code>	pointer to a const int
<code>int* const p</code>	const pointer to an int
<code>const int* const p</code>	const pointer to a const int

- `const int* p`
 - Cannot modify the value through p
 - Pointer itself can change
- `int* const p`
 - Pointer cannot change
 - Value can be modified
- `const int* const p`
 - Neither the pointer nor the value can change

In class exercises: Const and Pointers

Q1. Write the following declarations and explain:

```
const int* p1;  
int* const p2 = nullptr;  
const int* const p3 = nullptr;
```

- Which part is constant? The pointer or the value?
- Which operations are allowed? Which are forbidden?

Q2. Modify to compile and explain:

```
int x = 42;  
const int* p = &x;  
*p = 100;    // why error?  
x = 100;     // why okay?
```

How to read types in C++

- Types in C/C++ can be quite complex. Just take const pointers:
 - Is it the pointer, the value, or both?
 - This can be confusing
- If only there were a neat way to read C/C++ types...

Introducing the spiral rule

- <https://c-faq.com/decl/spiral.anderson.html>
- We can use the spiral rule to read types
 - Both in C
 - And in C++

Spiral Type Example

```
char *myvar[10];
```

- Is this a pointer to a character array of size 10?
- Or is it an array of 10 pointers to characters?
- We must be able to tell the difference! They are not the same!

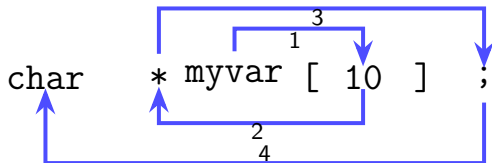
Spiral rule example

- Start from the name of the type (myvar)
- Spiral as close as possible
 - Starting with opening bracket (myvar is an array)
 - Spiral to pointers (myvar is an array of pointers)
 - Spiral to the semicolon - adding no new info
 - Skip the closing bracket and size
 - There's no new type info there
 - Spiral from the semicolon to the char
 - myvar is an array of pointers to a char

Spiral Type Example

```
char *myvar[10];
```

Spiral rule example



1. `myvar` is an array `[10]`
2. of pointers `*`
3. (continue to semicolon)
4. to `char`

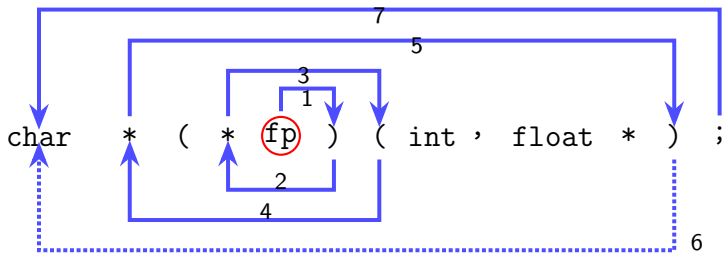
Spiral rule complex example

- Start with the name, fp
- Go from fp to the closing parens
- From closing parens to the star
 - We get that fp is a pointer
- We go from the star to the open parens
 - Open parens means function
 - fp is a pointer to a function
 - A function that takes an integer and a pointer to the float
- Then we go to the other star
 - func returns pointer... but to what?
- Go to closing parens
- Go back to char... func returns a pointer to the character

Spiral Type Example: Complex

```
char *(*fp)(int, float *);
```

Spiral rule complex example



1. `fp` is...
2. a pointer
3. to a function (`int, float*`)
4. returning a pointer
6. to `char`

In class exercise: Spiral Rule Challenge

Interpret these declarations using the **spiral rule**:

```
char *names[5];  
int (*matrix)[10];  
std::vector *(*myvar)(double *, int);
```

- Write a short English description of each variable.
- Hint: start at the variable name and spiral outward!

Types in C++

- C++ is a **strongly typed language**
- Every variable has a specific **type**

```
int x = 10;  
double y = 3.14;  
char c = 'A';
```

- The type determines:
 - How much memory is used
 - How the data is interpreted
 - What operations are allowed
- Because of this, C++ usually does not allow mixing unrelated types directly.

What Is Type Conversion?

- Sometimes we want to convert a value from one type to another.
- This is called a **type conversion** or **type cast**.

```
int x = 5;  
double y = x;
```

- Here an `int` is converted into a `double`.
- C++ allows some conversions automatically.

Implicit Conversions

- Some conversions happen **automatically**.
- These are called **implicit conversions**.

```
int x = 5;  
double y = x;    // int -> double
```

- C++ promotes smaller or simpler types to larger ones.
- These are generally safe conversions.

```
int x = 10;  
double d = x;
```

Explicit Conversions

- Sometimes C++ will not convert automatically.
 - This is especially true when we are going from a larger value in memory to a smaller value in memory
 - Or when we are losing information (like the decimal values in a double)
- In these cases we must explicitly request the conversion.
- This is called an **explicit cast**.

```
double d = 3.14;  
int x = (int)d;
```

- The decimal part is removed.
- Result:

```
x == 3
```

C-Style Casts

- Older C code used this syntax:

```
int x = (int)3.14;
```

- This works in C++ but is generally discouraged.
- The problem:
 - It hides what kind of conversion is happening
 - It can perform unsafe conversions
- Modern C++ prefers safer, more explicit casts.

The Four C++ Casts

- C++ provides four specific casting operators:
- `static_cast`
- `const_cast`
- `reinterpret_cast`
- `dynamic_cast`
- Each one is designed for a specific purpose.
- This helps make code safer and easier to understand.

- `static_cast` converts between **compatible types**.
- The conversion is checked at **compile time**.
 - This is huge! Incompatible type conversions will be rejected at compile time.
 - Very unlike C style casts, which may result in undefined behavior when incompatible type conversions occur
- Often used for numeric conversions.

```
int x = 10;
```

```
double d = static_cast<double>(x);
```

- This makes the conversion explicit and easy to read.

static_cast Example: Avoid Integer Division

```
#include <iostream>

int main() {

    int a = 5;
    int b = 2;

    double result =
        static_cast<double>(a) / b;

    std::cout << result << std::endl;
}
```

- Without the cast:
 - $5 / 2 = 2$
- With static_cast:
 - $5.0 / 2 = 2.5$

- `const_cast` changes the **constness** of a variable.
- It can add or remove the `const` qualifier.
- Typical reasons this is used:
 - Interfacing with legacy APIs that don't use `const`
 - Passing data to functions expecting different `const` qualifiers
- In general:
 - **Avoid removing `const` whenever possible**

const_cast Example: Removing const

```
#include <iostream>

int main() {

    const int x = 10;

    int* p =
    const_cast<int*>(&x);

    *p = 20;    // Undefined behavior

    std::cout << x << std::endl;
}
```

- Modifying a value that was originally declared const is **undefined behavior**.
- This might work, crash, or produce incorrect results.

const_cast Example: Adding const

```
#include <iostream>
int main() {
    int x = 42;
    int* p = &x;
    const int* cp =
const_cast<const int*>(p);

    std::cout << *cp << std::endl;
    *p = 20; // Still allowed
}
```

- Adding const is **safe**.
- It prevents modification through this pointer.
- The original non-const pointer can still modify the value.

- reinterpret_cast converts between completely unrelated types.
- It simply reinterprets memory as another type.
- Almost no safety checks are performed.

```
int x = 65;
```

```
char* p =  
reinterpret_cast<char*>(&x);
```

- This treats the memory of x as characters.
- In this case because 65 fits into one byte, it works... but just by chance!
 - If x was larger than one byte this would start to produce nonsense

- `reinterpret_cast` is very low level.
- It should almost never be used in normal applications.
- There are almost no safety checks with this and other type conversions are always safer, and only slightly slower
- Common uses include:
 - low-level systems programming
 - hardware access
 - serialization

- `dynamic_cast` is used with **class hierarchies**.
- It safely converts pointers between related classes.
- The check happens at **runtime**.
- If the cast fails:
 - the pointer becomes `nullptr`
- Don't worry about this for now, we'll cover classes later.

Summary of C++ Casts

Cast	Purpose
<code>static_cast</code>	normal type conversions
<code>const_cast</code>	change const qualifiers
<code>reinterpret_cast</code>	low-level memory reinterpretation. Danger!!!
<code>dynamic_cast</code>	safe casting in class hierarchies

Q1. For each cast type, give:

- An example of valid usage
- `static_cast`
- `const_cast`
- `reinterpret_cast` (Just list what this does)
- `dynamic_cast` (Just list what this does)

In-Class Exercise: Alternating Case

- Write a function that modifies a string **in-place**.
- The function should:
 - Convert every other character to **uppercase**
 - Convert the remaining characters to **lowercase**
- Requirements:
 - Use a **range-based for loop**
 - Modify the string **directly** (no copies)
 - Use the standard library functions `toupper()` and `tolower()`

Example

Input: "hello world"

Output: "HeLlO WoRlD"

Solution: Alternating Case

Implementation

```
#include <cctype>
void alternateCase(std::string& s) {
    bool upper = true;
    for (char& c : s) {
        if (upper)
            c = std::toupper(c);
        else
            c = std::tolower(c);
        upper = !upper;
    }
}
```

- The string is passed by **reference** so it is modified in-place.
- The loop uses `char& c` so each character is modified directly.
- A boolean variable keeps track of whether the next character should be uppercase or lowercase.

In-Class Exercise: Sum of Characters

Goal: Write a function that adds together the numeric values of all characters in a string.

Function Signature

```
int sumCharacters(const std::string& s);
```

Requirements:

- The string must be passed as a **const reference**
- Use a **range-based for loop**
- Convert each character to an integer using `static_cast<int>`
- Return the total sum

Example

Input: "ABC"

Output: 198

In-Class Exercise: Sum of Characters (Solution)

Implementation

```
#include <string>
int sumCharacters(const std::string& s) {
    int total = 0;
    for (const char& c : s) {
        total += static_cast<int>(c);
    }
    return total;
}
```

- The string is passed as a **const reference**
 - No copying
 - Cannot modify the original string
- The range-based loop uses `const char&`
- `static_cast<int>(c)` explicitly converts each character to its numeric (ASCII) value

Why Do We Need Hashmaps?

- Many real problems require **key-value pairs**
- Examples:
 - Student ID → Student Name
 - Word → Definition
 - Username → Password hash
 - Character → Frequency count
- We want:
 - Fast lookup by key
 - Fast insertion
 - No duplicates of keys

First Attempt: Use Two Vectors

Example

```
std::vector<std::string> names;  
std::vector<int> ids;  
  
names.push_back("Alice");  
ids.push_back(1001);
```

- `names[i]` corresponds to `ids[i]`
- Problems:
 - Easy to get out of sync
 - Lookup requires linear search ($O(n)$)
 - Inserting duplicates is easy

Second Attempt: Store Pairs in a Vector

Better Attempt

```
std::vector<std::pair<int, std::string>> students;  
  
students.push_back({1001, "Alice"});
```

- Cleaner: ID is tied to name
- But lookup still requires scanning entire vector:

```
for (auto& p : students) {  
    if (p.first == 1001)  
        std::cout << p.second;  
}
```

- This is still $O(n)$, i.e. we must look at (at worst) each element before finding the one we want!

The Problem

- Linear search is slow for large datasets
- If we have:
 - 10 items $\rightarrow$ fine
 - 10,000 items $\rightarrow$ slow
 - 1,000,000 items $\rightarrow$ very slow
- We want:
 - Average $O(1)$ lookup (instant)

Enter: Hashmaps

- In C++, hashmaps are implemented using:
 - `std::unordered_map`
- Include:

```
#include <unordered_map>
```

- Stores unique keys mapped to values
- Uses a **hash function**
- Average lookup: $O(1)$

Simple Hashmap Example

Example

```
#include <unordered_map>

std::unordered_map<int, std::string> students;

students[1001] = "Alice";
students[1002] = "Bob";

std::cout << students[1001];
```

- Direct lookup by key
- No manual searching required

How Does a Hashmap Work?

- A hash function converts a key into a number
- That number determines where the value is stored
- Internally:
 - Array of buckets
 - Keys are placed into buckets using `hash(key)`
- If two keys hash to the same bucket:
 - This is called a **collision**
 - Handled internally (usually with chaining)

Important: `std::map` (Different Implementation)

- C++ also provides:
 - `std::map`
- This is NOT a hash table.
- It is implemented using a:
 - **Red-Black Tree**
- That means:
 - Keys are always sorted
 - Lookup is $O(\log n)$
 - Insert is $O(\log n)$

std::map vs std::unordered_map

Feature	unordered_map	map
Implementation	Hash table	Red-Black Tree
Average lookup	$O(1)$	$O(\log n)$
Sorted keys	No	Yes
Worst-case lookup	$O(n)$	$O(\log n)$

- Use `unordered_map` for speed
- Use `map` if you need sorted keys

Iterating Over a Hashmap

Example

```
std::unordered_map<char, int> freq;

freq['a'] = 3;
freq['b'] = 5;

for (const auto& pair : freq) {
    std::cout << pair.first
               << " -> "
               << pair.second << std::endl;
}
```

- pair.first = key
- pair.second = value

In-Class Exercise: Character Frequency

Goal: Count how many times each character appears in a string.

- Use:
 - `std::unordered_map<char, int>`
- Use a range-based for loop
- Increment the count for each character

Example

Input: "hello"

Output:

h -> 1

e -> 1

l -> 2

o -> 1

In-Class Exercise: Character Frequency (Solution)

Implementation

```
#include <iostream>
#include <unordered_map>
#include <string>
void countFrequency(const std::string& s) {
    std::unordered_map<char, int> freq;
    for (const char& c : s) {
        freq[c]++; // increments value, default initializes
                   // to 0
    }
    for (const auto& pair : freq) {
        std::cout << pair.first
                   << " -> "
                   << pair.second
                   << std::endl;
    }
}
```

End of day 14

What Is a Set?

- A **set** stores unique elements
- No duplicate values allowed
- You can think of it as:
 - A **hashmap without values**
- Instead of:
`key: value`
- We only store:
`key`

Set vs Hashmap Mental Model

- Hashmap:

```
std::unordered_map<std::string, int>
```

- Stores:

```
"apple" -> 3  
"banana" -> 5
```

- Set:

```
std::unordered_set<std::string>
```

- Stores:

```
"apple"  
"banana"
```

- A set answers:

- "Does this exist?"

When Would We Use a Set?

- Suppose we want to track:
 - Unique usernames that have logged in
- We do NOT care:
 - How many times they logged in
 - Extra data about them
- We only care:
 - Has this user appeared before?

Bad Approach: Using a Vector

Problematic Approach

```
std::vector<std::string> users;  
  
users.push_back("alice");  
users.push_back("bob");  
users.push_back("alice"); // duplicate
```

- Duplicates allowed
- Checking existence requires linear search $O(n)$

Better Approach: Use a Set

Using unordered_set

```
#include <unordered_set>

std::unordered_set<std::string> users;

users.insert("alice");
users.insert("bob");
users.insert("alice"); // ignored
```

- Duplicates automatically prevented
- Fast lookup (average $O(1)$)
- Cleaner intent

Checking If an Element Exists

Example

```
if (users.find("alice") != users.end()) {  
    std::cout << "User exists\n";  
}
```

- `find()` returns iterator. We'll cover these in depth later on in this course.
- If not found, it returns `end()`

Feature	unordered_set	set
Implementation	Hash table	Red-Black Tree
Average lookup	$O(1)$	$O(\log n)$
Sorted	No	Yes

- Use unordered_set for speed
- Use set if sorted order matters

Iterating Over a Set

Example

```
for (const auto& user : users) {  
    std::cout << user << std::endl;  
}
```

- No first or second
- Only values

In-Class Exercise: Remove Duplicates

Goal: Write a function that removes duplicate characters from a string.

- Use:
 - `std::unordered_set<char>`
- Return a new string containing only unique characters
- Order does NOT matter

Example

Input: "banana"

Output: "ban" (or any order of b, a, n)

In-Class Exercise: Remove Duplicates (Solution)

Implementation

```
#include <unordered_set>
#include <string>

std::string removeDuplicates(const std::string& s) {
    std::unordered_set<char> seen;
    std::string result;

    for (const char& c : s) {
        if (seen.find(c) == seen.end()) {
            seen.insert(c);
            result += c;
        }
    }

    return result;
}
```

In-Class Exercise: Detect Duplicates

Goal: Write a function that returns `true` if a string contains any duplicate characters.

- Use:
 - `std::unordered_set<char>`
- Return:
 - `true` if duplicates exist
 - `false` otherwise

Examples

```
Input:  "hello"    -> true
Input:  "world"    -> false
Input:  "abcde"    -> false
Input:  "banana"   -> true
```

In-Class Exercise: Detect Duplicates (Solution)

Implementation

```
#include <unordered_set>
#include <string>

bool hasDuplicates(const std::string& s) {
    std::unordered_set<char> seen;
    for (const char& c : s) {
        if (seen.find(c) != seen.end()) {
            return true; // duplicate found
        }
        seen.insert(c);
    }
    return false;
}
```

- We immediately return when a duplicate is found
- Early exit improves performance
- Average time complexity: **$O(n)$**

The C++ Standard Library Has Many Data Structures

- The C++ Standard Library provides many ready-to-use data structures
- We've covered the most common ones:
 - `std::vector`
 - `std::unordered_map` / `std::map`
 - `std::unordered_set` / `std::set`
- But there are many more:
 - Linked lists
 - Stacks
 - Queues
 - Priority queues
 - Deques

Examples: Other Standard Containers

Linked List, stack, queue

```
#include <list>
std::list<int> linkedList;

#include <stack>
std::stack<int> s;

#include <queue>
std::queue<int> q;
```

- We've covered a bunch of these data structures, and this isn't a full data structures course
- The important point is pretty much every data structure is already implemented in the C++ standard library

End of Day 13

Why Do We Need Exceptions?

- Programs encounter errors:
 - File not found
 - Invalid input
 - Division by zero
 - Out-of-range access
- We need a way to:
 - Detect errors
 - Report errors
 - Handle errors safely
- Historically, C-style code handled errors differently...

The Old Way: Error Codes

Example

```
int divide(int a, int b) {  
    if (b == 0)  
        return -1; // error code  
    return a / b;  
}
```

- Problem:
 - What if -1 is a valid result?
 - Caller must remember to check
- Errors can be silently ignored

Old Pattern: Output Parameters

Example

```
bool divide(int a, int b, int& result) {  
    if (b == 0)  
        return false;  
  
    result = a / b;  
    return true;  
}
```

- Works, but:
 - Clutters function signatures
 - Easy to misuse
 - Hard to scale in large systems

Enter Exceptions

- C++ provides structured exception handling:
 - `throw`
 - `try`
 - `catch`
- Instead of returning an error:
 - We **throw** an exception
 - The caller can **catch** it

Basic Exception Example

Example

```
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0)
        throw std::runtime_error("Division by zero");
    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 0);
    }
    catch (const std::exception& e) {
        std::cout << "Error: " << e.what();
    }
}
```

What Happens When You Throw?

- When an exception is thrown:
 - The current function stops immediately
 - The stack begins **unwinding**
 - Objects are destroyed properly
- Control transfers to the nearest matching catch

Common Standard Exceptions

- `std::runtime_error`
- `std::logic_error`
- `std::out_of_range`
- `std::invalid_argument`
- `std::bad_alloc`
- All inherit from:
 - `std::exception`
- You can catch them generically:

```
catch (const std::exception& e)
```


Modern C++: Rethinking Exceptions

- Exceptions are powerful
- But they have downsides:
 - Hidden control flow
 - Harder to reason about in large systems
 - Performance overhead in some contexts
- Modern C++ sometimes prefers:
 - `std::optional`
 - `std::expected` (C++23)

C++23: std::expected

- `std::expected<T, E>` represents:
 - Either a value of type `T`
 - Or an error of type `E`
- Makes error handling explicit
- No hidden control flow

Example

```
#include <expected>

std::expected<int, std::string>
divide(int a, int b) {
    if (b == 0)
        return std::unexpected("Division by zero");
    return a / b;
}
```

Example

```
auto result = divide(10, 0);  
  
if (result) {  
    std::cout << result.value();  
} else {  
    std::cout << result.error();  
}
```

- Error handling is explicit
- No try/catch required
- Very clear control flow

Exceptions vs `std::expected`

Feature	Exceptions	<code>std::expected</code>
Control flow	Implicit	Explicit
Performance predictability	Less predictable	Predictable
Best for	Rare errors	Expected failures
Syntax	<code>try/catch</code>	<code>if/else</code>

- Use exceptions for:
 - Truly exceptional situations
- Use `expected` for:
 - Regular error conditions

Modern C++ and Rust: Similar Error Philosophy

- Rust does NOT use exceptions.
- Instead, it uses:
 - `Result<T, E>`
- C++23 introduces:
 - `std::expected<T, E>`
- Both represent:
 - Either a valid value
 - Or an error
- Errors are part of the type system.

Why This Matters

- With exceptions:
 - Control flow is hidden
 - A function may throw
 - Caller might forget to catch
- With `std::expected`:
 - The return type forces you to handle errors
 - You cannot ignore the possibility of failure
- The compiler helps enforce correctness.

Where Exceptions Are Not Allowed

- Some environments completely forbid exceptions:
 - Embedded systems
 - Real-time systems
 - Game engines
 - Low-latency trading systems
 - OS kernels
- Why?
 - Stack unwinding cost
 - Unpredictable timing
 - Binary size concerns
- In these systems:
 - Error codes or expected-style handling is required

Forcing Explicit Error Handling

With Exceptions

```
int value = divide(10, 0); // might throw
```

- If not caught:
 - Program terminates

With `std::expected`

```
auto result = divide(10, 0);
```

- You **MUST** check:
 - Is there a value?
 - Or is there an error?

Summary: Three Error Handling Styles

Style	Era
Error codes	C-style
Exceptions	Classic C++
<code>std::expected</code>	Modern C++23

- All three still exist
- Modern C++ increasingly prefers explicit error types
- Choose based on context

In-Class Exercise 1: C-Style Error Handling

Goal: Write a function that divides two integers.

- If division by zero occurs:
 - Return an error code
- Otherwise:
 - Return the result

Signature

```
int divide(int a, int b);
```

Hint:

- Return a special value like -1 for errors

In-Class Exercise 2: Classic C++ Exceptions

Goal: Rewrite the same function using exceptions.

- If division by zero:
 - Throw `std::runtime_error`
- Catch it in `main()`

Signature

```
int divide(int a, int b);
```

Use:

- `throw`
- `try`
- `catch`

In-Class Exercise 3: C++23 std::expected

Goal: Rewrite the function using `std::expected`.

- If division by zero:
 - Return `std::unexpected("Division by zero")`
- Otherwise:
 - Return the result

Signature

```
std::expected<int, std::string>  
divide(int a, int b);
```

In main():

- Check if result has value
- Otherwise print error

Solution 1: C-Style Error Handling

Implementation

```
int divide(int a, int b) {  
    if (b == 0)  
        return -1; // error code  
    return a / b;  
}  
  
int main() {  
    int result = divide(10, 0);  
    if (result == -1)  
        std::cout << "Error: Division by zero\n";  
    else  
        std::cout << result << std::endl;  
}
```

- Simple
- But error code may conflict with valid values
- Caller must remember to check

Solution 2: Classic C++ Exceptions

Implementation

```
#include <stdexcept>
int divide(int a, int b) {
    if (b == 0)
        throw std::runtime_error("Division by zero");
    return a / b;
}
int main() {
    try {
        std::cout << divide(10, 0);
    }
    catch (const std::exception& e) {
        std::cout << "Error: " << e.what();
    }
}
```

- Clean separation of logic and error handling
- But control flow is implicit

Solution 3: C++23 std::expected

Implementation

```
#include <expected>
#include <string>

std::expected<int, std::string>
divide(int a, int b) {
    if (b == 0)
        return std::unexpected("Division by zero");
    return a / b;
}

int main() {
    auto result = divide(10, 0);

    if (result) {
        std::cout << result.value();
    } else {
        std::cout << result.error();
    }
}
```

Performance Considerations

- C-Style Error Codes:
 - Very predictable
 - No overhead
- Exceptions:
 - Zero cost when not thrown
 - Expensive when thrown (stack unwinding)
 - Harder to predict latency
- `std::expected`:
 - Slightly larger return type
 - No stack unwinding
 - Fully predictable control flow
- In performance-critical systems:
 - Exceptions are often disabled, `std::expected` is not

End of day 15

What Is a Struct?

- A **struct** is a custom data type
- It lets us group related pieces of data together
- Instead of separate variables floating around, we bundle them into one meaningful unit
- A struct helps model real-world things in code

Why Are Structs Useful?

- Organize related data into one object
- Improve readability
- Make code easier to reason about
- Allow us to create our own types
- Help model real-world entities

Without structs:

- name
- age
- id

With a struct:

- A single Student

Structs Define New Types

- A struct defines a **type**
- Just like:
 - `int`
 - `double`
 - `std::string`
- But this time:
 - **We create the type ourselves**
- A struct describes what something looks like

Structs Describe Shared Properties

- A struct describes a set of objects that:
 - Share some properties
 - Meet some requirements
- Example:
 - All students have:
 - a name
 - an ID
 - a GPA
- The struct describes what every student object must contain

Struct = Blueprint or Recipe

- A struct is like a **blueprint**
- Or like a **recipe**
- The blueprint tells you:
 - What parts exist
 - How they are arranged
- The recipe tells you:
 - What ingredients are required

Objects Are Instances of a Struct

- The struct itself is **not** the thing
- It only describes the thing
- When we create a variable of that struct type:
 - That variable is called an **object**
 - Or an **instance**
- The object is the real, usable data

Recipe Example

- Suppose we have a cookie recipe
- The recipe describes:
 - Flour
 - Sugar
 - Eggs
 - Baking time
- But I do not eat the recipe
- I eat the cookies created from it

Connecting the Analogy

- The struct is the recipe
- The object (instance) is the cookie
- You can create:
 - One cookie
 - Ten cookies
 - One hundred cookies
- All follow the same recipe

Example: Student Struct

Struct Definition

```
struct Student {  
    std::string name;  
    int id;  
    double gpa;  
};
```

- This defines the blueprint
- It does not create a student yet

Creating Objects (Instances)

Creating Instances

```
Student s1;  
Student s2;
```

- s1 and s2 are objects
- They are two separate students
- Both follow the same blueprint

Example

```
Student s1;  
  
s1.name = "Alice";  
s1.id = 1001;  
s1.gpa = 3.8;
```

- Use the dot operator .
- Each object has its own copy of the data

Multiple Instances

Example

```
Student s1;  
Student s2;  
  
s1.name = "Alice";  
s2.name = "Bob";
```

- Same struct
- Different objects
- Different data stored inside each

C-Style Struct (Using typedef)

C-Style with typedef

```
typedef struct {  
    char name[50];  
    int id;  
    double gpa;  
} Student;
```

- In C, the type name is normally `struct Student`
- `typedef` creates an alias
- Now we can simply write:

```
Student s1;
```

Modern C++ Syntax

```
struct Student {  
    std::string name;  
    int id;  
    double gpa;  
};
```

- In C++, the struct name automatically becomes a type
- No typedef required
- We can directly write:

```
Student s1;
```

Key Differences Between C and C++ Struct Syntax

- C commonly uses:
 - `typedef struct {...} Student;`
- C++ allows:
 - Just `struct Student {...};`
 - C++ automatically treats the struct name as a type
 - C++ modernized and simplified the syntax

Nested Structs

- A struct can contain another struct as a member
- This is called a **nested struct**
- It helps model more complex data
- Useful when one object naturally contains another

Struct Inside Another Struct

Example

```
struct Address {  
    std::string street;  
    std::string city;  
};  
  
struct Student {  
    std::string name;  
    int id;  
    Address address;    // nested struct  
};
```

- A Student "has an" Address

Accessing Nested Members

Example

```
Student s;  
  
s.name = "Alice";  
s.address.street = "123 Main St";  
s.address.city = "Chicago";
```

- Use the dot operator multiple times
- Each level accesses the next layer

Struct Defined Inside Another Struct

Example

```
struct Student {  
    struct Address {  
        std::string street;  
        std::string city;  
    };  
  
    std::string name;  
    int id;  
    Address address;  
};
```

- The inner struct belongs to Student
- Its full name is Student::Address
- This way of nesting structs is fairly rare, but it is allowed!

Why Use Nested Structs?

- Organizes related data cleanly
- Improves readability
- Models real-world relationships
- Keeps helper types grouped with the main type

Structs Can Have Functions

- In C++, structs can contain:
 - Data members
 - Member functions
- This means structs can have behavior
- Not just data

Example: Function Inside a Struct

Example

```
struct Rectangle {  
    double width;  
    double height;  
  
    double area() {  
        return width * height;  
    }  
};
```

- The function belongs to the struct
- It operates on the struct's data

Member Functions Automatically Know the Object

- When a function is inside a struct:
 - It automatically has access to member variables
- You do NOT need to pass them in
- You do NOT need to specify which object explicitly

Accessing Attributes Directly

Example

```
struct Rectangle {  
    double width;  
    double height;  
  
    double area() {  
        return width * height;  
    }  
};
```

- Notice:
 - We wrote `width`
 - Not `this->width`
- C++ assumes member variables

What Is this?

- Inside a member function,
- `this` is a pointer to the current object
- It represents:
 - The instance that called the function

Example

```
struct Rectangle {  
    double width;  
    double height;  
  
    double area() {  
        return this->width * this->height;  
    }  
};
```

- This works
- But usually unnecessary

Why We Usually Do Not Use `this`

- C++ automatically assumes:
 - Unqualified names refer to member variables
- So `this` is equivalent:
 - `width`
 - `this->width`
- We only need `this` in special situations

When this Is Needed

Example

```
struct Rectangle {  
    double width;  
  
    void setWidth(double width) {  
        this->width = width;  
    }  
};
```

- Parameter name shadows member name
- `this->width` refers to the member variable

Defining Functions Outside the Struct

- Functions can be:
 - Declared inside the struct
 - Defined outside the struct
- This keeps code cleaner in larger programs

Declare Inside, Define Outside

Declaration

```
struct Rectangle {  
    double width;  
    double height;  
  
    double area();  
};
```

Definition

```
double Rectangle::area() {  
    return width * height;  
}
```

- Use StructName::functionName

Scope Resolution Operator

- The `::` operator means:
 - This function belongs to this struct
- Example:
 - `Rectangle::area`
- Required when defining functions outside

Const Member Functions

- If a function does not modify the object:
 - It should be marked `const`
- This guarantees:
 - The function will not change member variables

Example: Const Function

Example

```
struct Rectangle {  
    double width;  
    double height;  
  
    double area() const {  
        return width * height;  
    }  
};
```

- `const` appears after the parameter list
- The function cannot modify members

Const Function Defined Outside

Declaration

```
struct Rectangle {  
    double area() const;  
};
```

Definition

```
double Rectangle::area() const {  
    return width * height;  
}
```

- const must appear in both places

Why Const Matters

- Prevents accidental modification
- Allows calling the function on const objects
- Improves code safety and clarity

Why Use Getters and Setters?

- Getters and setters control access to data
- They protect the internal state of an object
- They help enforce rules (invariants)
- They improve maintainability
- Instead of allowing direct access, we control how values are read and modified

The Problem With Public Data

Example

```
struct BankAccount {  
    double balance;  
};
```

Any code can do:

```
account.balance = -999999;
```

- No protection
- No validation

- **Encapsulation** means:
 - Hiding internal data
 - Controlling access through functions
- We make data `private`
- And provide controlled access through:
 - Getters
 - Setters

Using Private Data + Getter/Setter

Example

```
struct BankAccount {  
private:  
    double balance;  
  
public:  
    void setBalance(double amount) {  
        if (amount >= 0)  
            balance = amount;  
    }  
  
    double getBalance() const {  
        return balance;  
    }  
};
```

- Now we prevent invalid values
- The struct enforces its own rules

Why Not Just Make Everything Public?

- Harder to maintain
- Harder to debug
- Outside code can break invariants
- No central place to enforce rules
- Getters and setters keep control inside the struct

Access Specifiers in Structs

- C++ provides three access levels:
 - `public`
 - `private`
 - `protected`
- These control who can access members

- Accessible from anywhere
- Typically used for:
 - Functions
 - Public interface
- In structs, default access is public

- Only accessible inside the struct
- Outside code cannot directly access it
- Used to protect internal data
- Commonly used with getters and setters

- Accessible inside the struct
- Also accessible in derived structs/classes
- Not accessible from unrelated outside code
- Mainly used with inheritance

Access Specifiers Can Appear Multiple Times

Example

```
struct Example {  
public:  
    void func1();  
  
private:  
    int secret;  
  
public:  
    void func2();  
  
protected:  
    int semiSecret;  
  
private:  
    int hiddenAgain;  
};
```

How Access Specifiers Work

- When you write `public`:
- Everything after it is public
- Until another access specifier appears
- Same for:
 - `private`:
 - `protected`:

Default Access in a Struct

- In C++, everything inside a struct is **public by default**
- If you do not write an access specifier,
- The members are automatically public

Example: Struct Default Is Public

Example

```
struct Person {  
    std::string name;  
    int age;  
};  
  
int main() {  
    Person p;  
    p.name = "Alice";    // Allowed  
    p.age = 25;          // Allowed  
}
```

- No `public:` label is required
- Members are public automatically

Is Public-by-Default a Good Idea?

- Should internal data be freely modifiable?
- In many cases:
 - We want to protect internal state
 - We want to enforce invariants
- It would often make more sense if members were private by default

Why Private-by-Default Makes Sense

- Encourages encapsulation
- Forces controlled access
- Prevents accidental misuse
- Makes invariants easier to maintain
- Safer design by default

Enter Classes

- C++ introduces another keyword:
 - `class`
- A class is almost identical to a struct
- Except for one important difference

The Only Real Language Difference

Keyword	Default Access
struct	public
class	private

- That is the only built-in language difference
- Everything else works the same

Example: Class Default Is Private

Example

```
class Person {  
    std::string name;  
    int age;  
};  
  
int main() {  
    Person p;  
    p.name = "Alice";    // Error  
}
```

- Without `public:`, members are private

Making Members Public in a Class

Example

```
class Person {  
public:  
    std::string name;  
    int age;  
};
```

- Now behavior matches a struct
- The only difference was the default

Struct vs Class: Conceptually

- struct:
 - Historically used for simple data
 - Public by default
- class:
 - Designed for encapsulated objects
 - Private by default
- Mechanically:
 - They are nearly identical

When to Use Struct?

- Public-by-default: use `struct`
- Private-by-default: use `class`
- That is the core built-in difference in C++
- Everything else is style and convention
- Typically `Struct` is used if there is pure data with no functionality, and `class` is used for everything else

What Is a Constructor?

- A constructor is a special member function
- It runs automatically when an object is created
- Its job is to initialize the object's data
- It has the same name as the class
- It has no return type (not even void)

Basic Constructor Example

Example

```
class Person {  
public:  
    Person(std::string n, int a) {  
        name = n;  
        age = a;  
    }  
  
private:  
    std::string name;  
    int age;  
};
```

- The constructor assigns values inside the body

What Happens When We Create an Object?

- When we write:
 - `Person p("Alice", 25);`
- The constructor runs automatically
- The object is initialized before we can use it

The “Normal” Constructor Style

- Parameters are passed in
- Inside the constructor body:
 - We assign values to member variables
- This works, but it is not always ideal

Normal Assignment Inside Constructor

Example

```
class Rectangle {  
public:  
    Rectangle(double w, double h) {  
        width = w;  
        height = h;  
    }  
  
private:  
    double width;  
    double height;  
};
```

- Members are default-initialized first
- Then reassigned inside the constructor body

Member Initializer Lists

- C++ provides a better way to initialize members
- Called a **member initializer list**
- It initializes members directly
- Happens before the constructor body runs

Constructor Using Initializer List

Example

```
class Rectangle {  
public:  
    Rectangle(double w, double h)  
        : width(w), height(h) {}  
  
private:  
    double width;  
    double height;  
};
```

- Members are initialized directly
- No reassignment inside the body

Why Initializer Lists Are Better

- More efficient
- Avoids double initialization
- Required for:
 - const members
 - reference members
- Considered best practice in modern C++

Why Do They Differ?

- Assignment inside constructor:
 - Member is created first
 - Then overwritten
- Initializer list:
 - Member is constructed with the correct value immediately
- Initialization $\neq$ Assignment

Example Showing the Difference

Important Case

```
class Example {  
public:  
    Example(int v) : value(v) {}  
  
private:  
    const int value;  
};
```

- This works
- If we tried assignment inside the body, it would not compile

Default Constructor

- A default constructor takes no parameters
- If you do not define any constructor,
- The compiler may generate one automatically

Example: Default Constructor

Example

```
class Point {  
public:  
    Point() : x(0), y(0) {}  
  
private:  
    int x;  
    int y;  
};
```

- Initializes members to safe default values

What Is an Enum?

- We've seen how we can format data with structs and classes
- Now we'll cover a new data type: Enums
- **enum** stands for **enumeration**
- It is a user-defined type
- It represents a fixed set of named constants
- Used when a variable should only take one of several specific values

Why Are Enums Useful?

- Improve readability
- Prevent invalid values
- Make intent clearer
- Replace “magic numbers” with meaningful names

Instead of:

- `status = 2;`

We can write:

- `status = COMPLETED;`

Example

```
enum Status {  
    PENDING,  
    APPROVED,  
    REJECTED  
};
```

- Defines a new type: Status
- Defines named constants

Using an Enum

Example

```
Status s = APPROVED;  
  
if (s == APPROVED) {  
    std::cout << "Approved\n";  
}
```

- Only defined enum values should be used
- Makes code more readable

Underlying Integer Values

- By default:
 - First value = 0
 - Next values increase by 1

Example:

- PENDING = 0
- APPROVED = 1
- REJECTED = 2

Example

```
enum ErrorCode {  
    OK = 0,  
    WARNING = 100,  
    ERROR = 200  
};
```

- You can manually assign values
- Useful when matching external systems

Enums as Types

- Enums create a new type
- Helps restrict variables to specific values
- Prevents accidental misuse

Example:

- A day of the week
- A state in a workflow
- A game difficulty level

Enum Example: Days of the Week

Example

```
enum Day {  
    MONDAY ,  
    TUESDAY ,  
    WEDNESDAY ,  
    THURSDAY ,  
    FRIDAY ,  
    SATURDAY ,  
    SUNDAY  
};
```

Using Enum in a Switch Statement

Example

```
Day today = MONDAY;
```

```
switch (today) {  
    case MONDAY:  
        std::cout << "Start of week\n";  
        break;  
    case FRIDAY:  
        std::cout << "Almost weekend\n";  
        break;  
    default:  
        std::cout << "Midweek\n";  
}
```

Modern C++: Scoped Enums

- C++11 introduced **enum class**
- Also called scoped enums
- Safer and more strongly typed
- Do not pollute the global namespace

Example: enum class

Example

```
enum class Status {  
    PENDING,  
    APPROVED,  
    REJECTED  
};
```

- Values must be qualified

Example

```
Status s = Status::APPROVED;  
  
if (s == Status::APPROVED) {  
    std::cout << "Approved\n";  
}
```

- Prevents accidental name conflicts
- Encouraged in modern C++

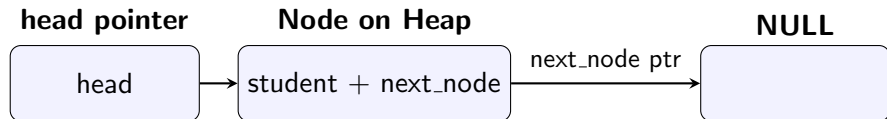
End of day 16

- We've briefly touched on pointers
- Now let's dive into C++ pointers (Smart pointers)
 - C and RAW pointers
 - C++ unique pointers
 - C++ shared pointers
 - C++ weak pointers

- Straight from C
 - You are in charge of everything
 - Allocating
 - Deallocating
- Typical challenges
 - Who should deallocate something and when?

- Let's say in C you have an abstract data type for a linked list
- For example, let's say it's a struct of student (our famous Student.h example)
- We might have the head pointer
- That head pointer would point towards a node, and that node's next_node pointer points to nothing
- That node may include student inside of it as the value
 - But if student was on the heap, then we've copied a student on the heap into the list
 - We also need to deallocate the original copy

RAW Pointers Example



Student data

Student (Heap copy)

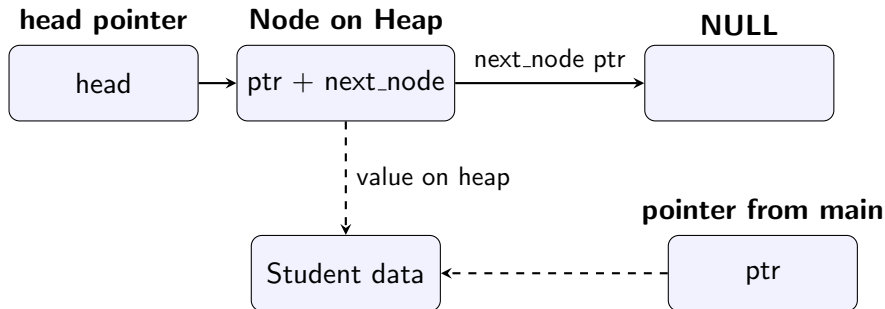
If the student was dynamically allocated, this list node stores a copy on the heap.

We must remember to free both the student and the node.

RAW pointers

- That's inefficient!
- Of course we can instead have a pointer to the Student instead, and store that in our node
- But then in our code we must always remember who is pointing to our Student object
- Unlike before, if we deallocated it, we'll have problems!
 - Since our linked list is now a second pointer pointing to the same object
- But the linked list has the same issue... they are wondering should I deallocate student or will someone else?
- Don't mess up or you'll have a leak!
 - Neither could free
 - Or both
 - Or use after free
- Sidenote: in Rust, everything is owned by only one object at a time.
- Smart pointers fix this! C style discipline is overrated!

RAW Pointers Example



Student (Heap copy)

Now both the list and main have pointers to the same student object.

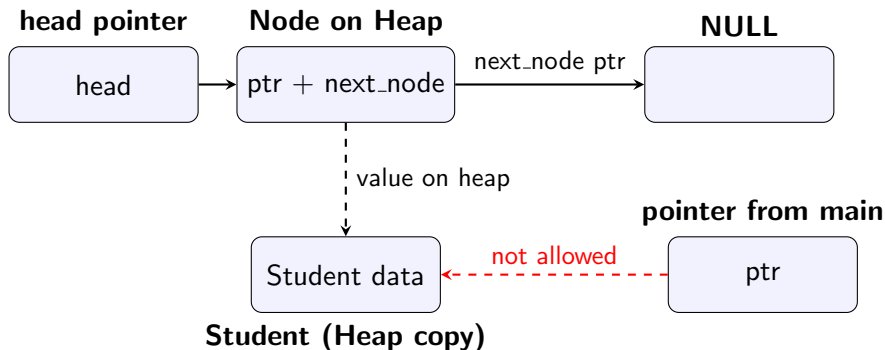
Who deletes it? Be careful!

Unique pointers

- The Strict Ownership Doctrine (Unique to C++ and Rust)
- This is a pointer that points to something that only ever have one owner
- Just like selling your car, and giving the keys to someone else. Now they can drive it and you can't.
- From the docs:
 - `std::unique_ptr` is a smart pointer that retains sole ownership of an object through a pointer and destroys that object when the `unique_ptr` goes out of scope. No two `unique_ptr` instances can manage the same object.
- Imagine that! You don't even need to use `delete`!
 - When the pointer goes out of scope, the object dies and gets freed/reclaimed

Unique Pointers Example

Not allowed! Use `std::move` instead.



Unique pointers prevent this kind of dual ownership.
Transfer ownership using `std::move()` instead.

Unique pointers

- You may find in the docs a `get` method
- This returns a raw pointer
- It exists only for legacy reasons
- Never use this, since it breaks the doctrine of single ownership.
- These were added for the purpose of converting smart pointer to normal pointers so that they can interact with very old libraries that were around when unique pointers didn't exist

Unique pointers

Example:

```
#include <iostream>
#include <memory>

template <class T>
std::ostream& operator<<(std::ostream& os, const std::
    unique_ptr<T>& p) {
    return os << p.get(); // Just for this example, don't
        use get() in production
}

int main() {
    // Old way: std::unique_ptr<int> p(new int);
    // *p = 42;
    auto p = std::make_unique<int>(42);
    std::cout << p << " " << *p << std::endl;
    // Old way: std::unique_ptr<int> z(std::move(p));
    std::unique_ptr<int> z = std::move(p); // can't z=p;
    std::cout << "p=" << p << std::endl;
    std::cout << "z=" << z << " *z=" << *z << std::endl;
```

unique pointers

Example: pass by value not allowed!

```
#include <iostream>
#include <memory>
template <class T>
std::ostream& operator<<(std::ostream& os, const std::
    unique_ptr<T>& p) {
    return os << p.get();
}

void foo(std::unique_ptr<int> z){
    std::cout<<"z=" << z << "*z=" << *z << std::endl;}

int main() {
    auto p = std::make_unique<int>(42);
    std::cout << p << " " << *p << std::endl;
    foo(std::move(p)); // Must move! Can't copy!
    std::cout << "p=" << p << std::endl; // Gone...
}
```

Unique pointers

- We move our unique pointer into the function
- Now p is empty, a nullptr
- We want however the ability to get p back
- How might we do this?

unique pointers

Example: pass by value not allowed!

```
#include <iostream>
#include <memory>
template <class T>
std::ostream& operator<< (std::ostream& os, const std::
    unique_ptr<T>& p) {
    return os << p.get();
}

std::unique_ptr<int> foo(std::unique_ptr<int> z){
    std::cout<<"z=" << z << " *z=" << *z << std::endl;
    return std::move(z);
}

int main() {
    auto p = std::make_unique<int>(42);
    cout << p << " " << *p << endl;
    //p = foo(std::move(p)); // Must move! Can't copy!!!
    p = std::move(foo(std::move(p))) // Must move!
    std::cout << "p=" << p << std::endl; // Back!
}
```

Unique pointers

- Awesome! We can move p all the way around
- But maybe we can make this a bit simpler
- Maybe we don't need to move p at all...

unique pointers

Example:

```
#include <iostream>
#include <memory>
template <class T>
std::ostream& operator<<(std::ostream& os, const std::
    unique_ptr<T>& p) {
    return os << p.get();
}

void foo(std::unique_ptr<int>& z){ // Ref here!
    std::cout<<"z=" << z << " *z=" << *z << std::endl;
}

int main() {
    auto p = std::make_unique<int>(42);
    cout << p << " " << *p << endl;
    foo(p);
    std::cout << "p=" << p << std::endl; // Back!
}
```

Unique pointers

- Cool! This works too!
- Don't forget... references under the hood are often implemented as pointers. They are just an alias to the object.
- This is just an alias, likely under the hood compiled to a pointer, to a unique pointer. `p` never lost ownership of the object.
- Notice also in every case, no delete! Even though `p` is on the heap.
- My personal opinion, I don't prefer references, or pointers, or aliases, to other pointers. Too many layers of indirection. Generally to me it's best to move in and out, and be explicit, but it's possible that other programmers may think differently, I don't know enough to know.

- The shared ownership doctrine
- Docs: `shared_ptr` is a smart pointer that retains shared ownership of an object through a pointer. Several `shared_ptr` objects may own the same object. The object is destroyed and its memory is deallocated when either of the following happens:
 - The last remaining `shared_ptr` owning the object is destroyed
 - The last remaining `shared_ptr` owning the object is assigned another pointer via `operator=` or `reset()`

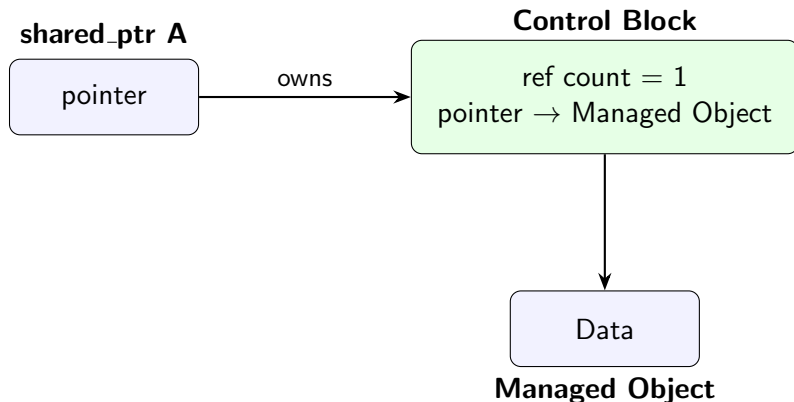
Illustrations

- Creation
- Sharing
- Destruction
- Assignments

Illustrations: Creation

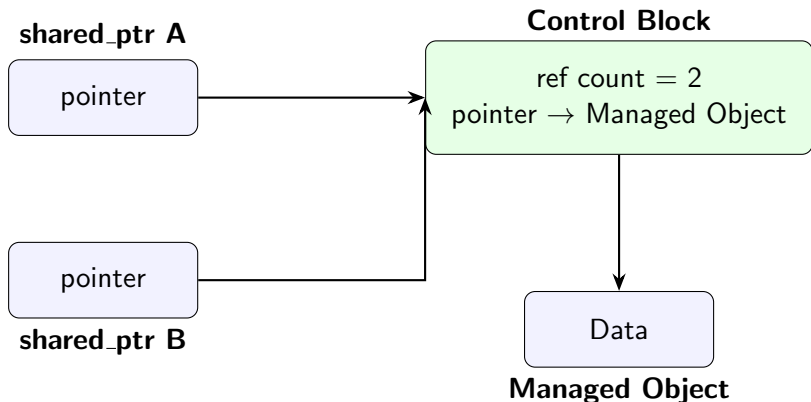
- When we create a new object, we get back a pointer to the object
- When we create a shared pointer however, we get a pointer to a tiny object that contains another pointer and a reference count.

Shared Pointer Diagram (Reference Count = 1)



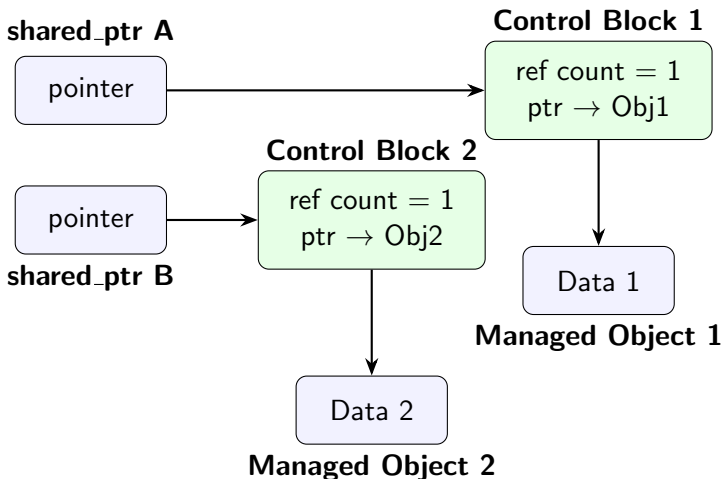
The control block stores both the pointer to the managed object and the reference count. Only one shared pointer currently owns it (ref count = 1).

Shared Pointer Diagram (Reference Count = 2)



Now both `shared_ptr A` and `shared_ptr B` share ownership of the object.
The control block keeps the pointer and a reference count = 2.

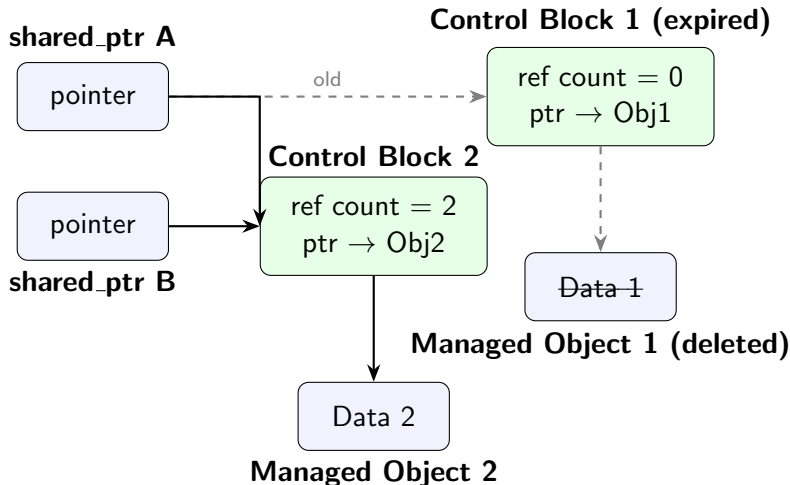
Shared Pointer Reassignment Example



Managed Object 2

shared_ptr B has been reassigned to manage a different object.
Now A and B point to separate control blocks and independent managed objects.

Shared Pointer Reassignment (Old Object Freed)



Managed Object 2
shared_ptr A now points to the same control block as B (ref count = 2).

The old Control Block 1's reference count dropped to 0, so its object is freed and the control block goes away.

To get or not to get

- There is a method just like with unique pointers called `get`
- The "get" method should not be used
 - This is breaking the abstractions of shared pointer
- It only exists for legacy reasons
 - This is for passing a shared pointer to legacy libraries that don't understand shared pointers
- `use_count` API
 - Returns the reference count

- Quick detour - Python
- This is exactly how Python's memory management system works, and this actually does come in handy
- Everything is on the heap
- Everything gets a reference count
- When the reference count hits zero, the object is reclaimed!
- If the reference count never hits zero due to cyclical references (A refers to B and B refers to A), the garbage collector once in a while runs, detects these cycles that have no references to the entire cycle, and reclaims the memory at that time
- To avoid cycles in Python, use weakref or proxy (similar to weak pointers, which we will cover shortly)

Shared pointers

Example

```
#include <iostream>
#include <memory>
template <class T>
std::ostream& operator<<(std::ostream& os, const std::
    shared_ptr<T>& p) {
    return os << p.get() << '[' << p.use_count() << '>';
}

struct Foo {
    int a;
    Foo(int x): a(x) {};
    ~Foo() {
        std::cout << "Foo(" << this << ") is destroyed" <<
            std::endl;
    }
}
```

Shared pointers

Example

```
void test1() {
    // Old way (avoid):
    // std::shared_ptr<int> p(new int);
    auto p = std::make_shared<int>(42);
    std::cout << "p=" << p << " *p=" << *p << std::endl;
    std::shared_ptr<int> z;
    z = p; // increases ref count
    std::cout << "p=" << p << std::endl;
    std::cout << "z=" << z << " *z=" << *z << std::endl;
};

int main() {
    test1();
    std::cout << "done!" << std::endl;
    return 0;
}
```

Old vs New Shared Pointer Syntax

- Two ways to create a shared pointer:
 - Old way (pre-C++11): `std::shared_ptr<int> p(new int(42));`
 - New way (modern): `auto p = std::make_shared<int>(42);`
- Both create a `shared_ptr` that owns an object on the heap
- But there are important differences under the hood

Why the Old Way Is Discouraged

- Looks fine, but it's **not exception safe!**
- Consider this:

```
foo(std::shared_ptr<int>(new int(1)),  
    std::shared_ptr<int>(new int(2)));
```

- If the second `new` throws an exception, the first `new int(1)` leaks since it was never owned by a shared pointer
- Two heap allocations are also performed:
 - One for the control block
 - One for the managed object

Why `make_shared` Is Better

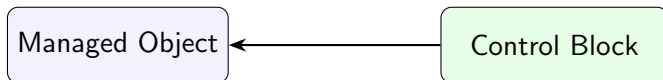
- The new way is both faster and safer:

```
auto p = std::make_shared<int>(42);
```

- It performs only **one heap allocation**:
 - The control block and object are allocated together
- The pointer never escapes during creation — exception safe
- Better memory locality and performance

Memory Layout Comparison

Old Way: `shared_ptr<T>(new T)`



New Way: `make_shared<T>()`



`make_shared` merges the allocations, improving speed and safety.

- `std::shared_ptr<T>(new T):`
 - Two allocations
 - Not exception safe
 - Easy to misuse
- `std::make_shared<T>():`
 - One allocation
 - Exception safe
 - Better performance
- Always use `make_shared`; There is pretty much never a reason to use the old way.

Shared pointers

Example

```
void fun(std::shared_ptr<Foo> p){
    std::cout << "p in fun=" << p << std::endl;}
void funToo(std::shared_ptr<Foo> p){
    std::cout << "p in funToo=" << p << std::endl;}
void test1() {
    auto p = std::make_shared<Foo>(42);
    auto q = p;
    std::cout << "q is" << q << std::endl;
    fun(q);
    std::cout << "after fun q is " << std::endl;
    std::cout << "Before funToo=" << q << std::endl;
    funToo(q);
    std::cout << "After funToo" << q << std::endl;
    q.reset();
    std::cout << "After q.reset()" << p << std::endl;
    p.reset();
    std::cout << "After p.reset()" << p << std::endl;
}
```

What about this?

- Remember that all objects have methods with an implicit argument called "this".
- "This" is a pointer to the object, a C styled pointer
- So what happens if we want to return the pointer to the object to someone else, how do we do it with the shared pointer doctrine?
- Doing it this way (below) breaks the shared ownership doctrine

Example

```
struct Foo {  
    void hello(){std::cout << "I'm " << this << std::endl;}  
    Foo* getThis() {return this;}  
};
```

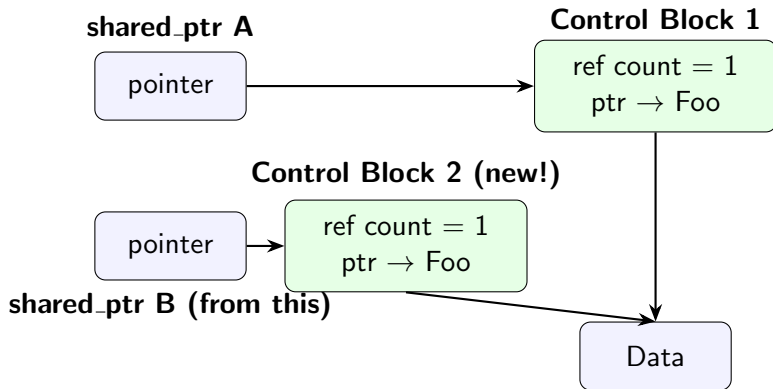
Returning a pointer

- Instead, we can just return a shared pointer from "this"! Or can we...

Example

```
struct Foo {  
    void hello(){std::cout << "I'm " << this << std::endl;}  
    //Foo* getThis() {return this;}  
    std::shared_ptr<Foo> getThis() {  
        return std::shared_ptr<Foo>(this);  
    }  
};
```

Incorrect Shared Pointer Creation from "this"



Foo object (this)

Returning `std::shared_ptr<Foo>(this)`
creates a **second control block**.

Both shared pointers think they own the same object alone.

When one is destroyed, the object is deleted
— leaving the other with a dangling pointer.

Returning a pointer

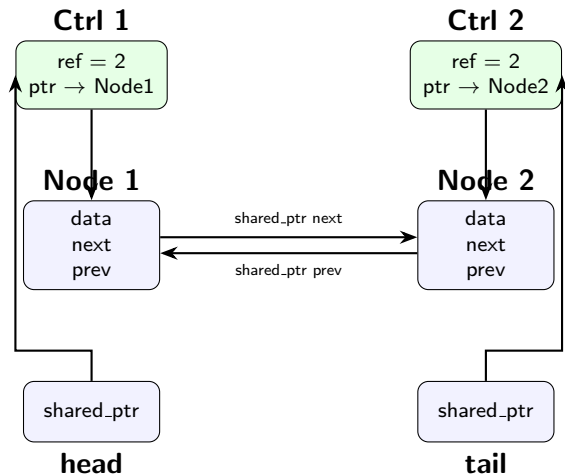
- Instead we can use a special class for this
- `std::enable_shared_from_this`

Example

```
struct Foo : std::enable_shared_from_this<Foo> {  
    void hello(){std::cout << "I'm " << this << std::endl;}  
    //Foo* getThis() {return this;}  
    //std::shared_ptr<Foo> getThis() {  
    //    return std::shared_ptr<Foo>(this);  
    //}  
    std::shared_ptr<Foo> getThis() {  
        return shared_from_this();  
    }  
};
```

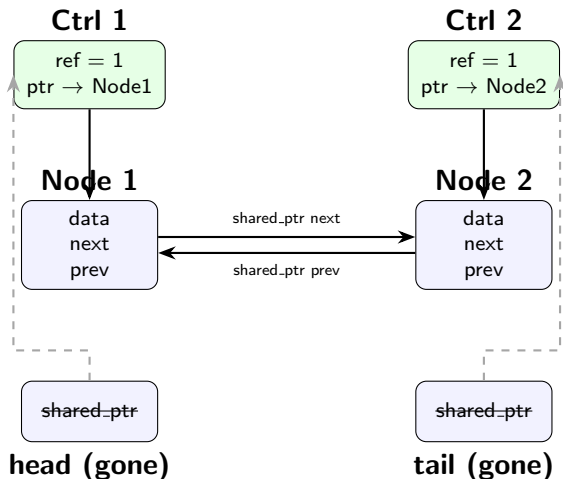
- The weak ownership doctrine
- docs: `weak_ptr` is a smart pointer that holds a non-owning (weak) reference to an object that is managed by an `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object.

Doubly Linked List with Shared Pointers (Cycle Problem)



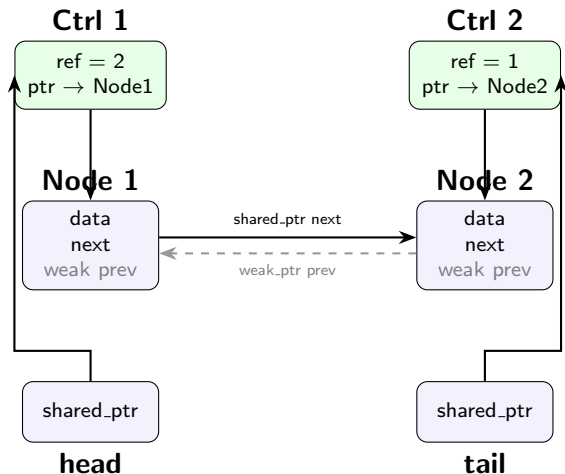
Each node's control block has `ref = 2`: one from the head/tail and one from the other node. Because both `next` and `prev` are `shared_ptr`s, neither reference count reaches zero — the nodes are never freed.

After Head/Tail Go Out of Scope (Still Leaking)



head and tail were destroyed, reducing each control block's count to 1. However, because the nodes still reference each other via shared pointers, the cycle keeps both objects alive — they can never be freed. This is a memory leak!

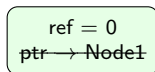
Doubly Linked List with Weak Pointers (No Cycle)



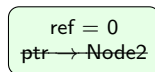
Now only next is a shared pointer, while prev is a weak pointer.
The weak pointer does not increase the reference count, breaking the cycle.
When head and tail go out of scope, reference counts will reach zero.

After Head/Tail Go Out of Scope (No Leak!)

Ctrl 1 (deleted)



Ctrl 2 (deleted)



Node 1



Node 2



shared_ptr

head (gone)

shared_ptr

tail (gone)

head and tail are gone, reducing the ref counts to 0.
Because prev was a weak pointer, no circular ownership remains.
Both nodes and control blocks are safely deallocated — no leaks!

End of day 17

Printing Shared and Weak Pointers

Example

```
#include <iostream>
#include <memory>
template <class T>
std::ostream& operator<< (std::ostream& os, const std::
    shared_ptr<T>& p) {
    return os << p.get() << '[' << p.use_count() << '];'
}
template <class T>
std::ostream& operator<< (std::ostream& os, std::weak_ptr<T>
    & p) {
    T* raw = nullptr;
    {
        auto sp = p.lock();
        raw = sp.get();
    }
    return os << raw << '[' << p.use_count() << '];'
}
```

Printing Shared and Weak Pointers

Example

```
struct Foo {
    int a;
    Foo(int x) {a=x;}
    ~Foo() {std::cout << "Foo(" << this << ") is destructed"
              << std::endl;}
    void printMe(){
        std::cout << "Printing " << this << "is " << a <<
            std::endl;
    }
}

void fun(std::weak_ptr<Foo> p){
    std::cout << "p in fun=" << p << std::endl;
    auto up = p.lock();
    std::cout << "up in fun" << up << std::endl;
    up->printMe();
}
```

Printing Shared and Weak Pointers

Example

```
void test(){
    std::shared_ptr<Foo> p = std::make_shared<Foo>(42);
    std::weak_ptr<Foo> q(p);
    fun(q);
    std::cout << "before q.reset()" << q << std::endl;
    q.reset();
    std::cout << "after q.reset()" << q << std::endl;
    p.reset();
    std::cout << "after p.reset()" << p << std::endl;
}

int main(){
    test();
    std::cout << "done" << std::endl;
    return 0;
}
```


Course Project

- Course Project

- Generics are an important part of the libraries that come with C++
 - We'll look at the STL and what it provides
 - We'll focus on vectors
 - We'll learn how to write iterators
 - And the mysterious keyword auto

STL: Standard Template Library

- Three fundamental pillars
 - Containers
 - Ex: Stack, list, array list (of any given type)
 - Iterators
 - Algorithms
- Icing on the cake: Fully generic

Generic - Why?

- Consider making a stack
 - Of integers, double, character, strings, students
 - Note: Must all be the same types
- Icing on the cake: Fully generic

Generic Example

- For initializing generics, you can simply use `container<type>`
- Stack is an object that comes from a library implemented in C++
 - This is the STL that accompanies the C++ language
- You can see stack class methods `push`, `top`, `pop`, etc

Const Pointer Example

```
#include <iostream>
#include <stack>
using std::cout, std::endl, std::stack;
int main() {
    stack<int> si; // stack<double> works here too
    for(int i=0; i<5; ++i){
        si.push(i);
    }
    while(!si.empty()){
        cout << "stackval=" << si.top() << endl;
        si.pop();
    }
    return 0;
}
```

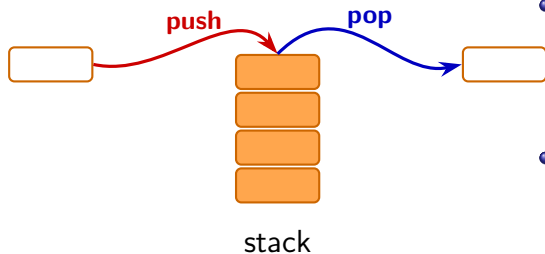
Stack, Last in first out (LIFO)

- Remember from CSE2050:

Stack Implementation

```
class Stack:
    def __init__(self):
        self._data = [] # Composition, Stack has a list
    def push(self, item):
        self._data.append(item)
    def pop(self):
        return self._data.pop()
    def peek(self):
        return self._data[-1]
    def __len__(self):
        return len(self._data)
    def is_empty(self):
        return not self._data
```

Stack



- A list with the restriction that insertion and deletion can only be performed from one end, the top
- LIFO: Last in first out

plates.png

think
pancakes
or plates

pancakes.png

Generic Example

- We can create a stack of any type
 - And yet, there's only a single implementation of stack
- Can use any user-defined type

Const Pointer Example

```
int main() {  
    stack<int> si;  
    stack<double> si;  
    stack<string> si;  
    stack<Spaceship> si;  
    return 0;  
}
```

Kind	Examples
Containers	array, vector, deque, list, forward_list, stack, queue, priority_queue, set, multiset, map, multimap, unordered_set, unordered_map
Iterators	input_iterator, output_iterator, forward_iterator, bidirectional_iterator, random_access_iterator
Algorithms	sort, find, binary_search, count, accumulate, for_each, copy, transform, remove, unique, merge

- Can define similar to stacks: `vector<T>`
- Vector provides:
 - A sequence, similar to C array
 - Dynamically sized
 - C array allocated in globals, size is known at compile time
 - C array allocated in stack, size is known when entering function
 - C array is declared on the heap with a call to `new`
 - No matter how it's declared C arrays never change size
 - Vectors however are dynamically sized
 - Vectors will grow to accomodate elements you are adding
 - Very similar to C arrays except for added safety
 - Ability to add/remove anywhere
 - Random access (indexing like an array)
 - Bounds safety (unlike C)

Vectors

- Behind the range loop, there is an iterator...

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;

int main() {
    vector<int> vec;
    for(int i=0; i<10; ++i){
        vec.push_back(i)
    }
    vec[0] = 1000; // Just like C arrays
    //vec[10] = 10000; no memory corrupted
    for(int x: vec){
        cout << x << " ";
    }
    cout << endl;
    return 0;
}
```

- Example: Adding values within a vector

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int sum(vector<int>& vec){
    int ttl = 0;
    for(int x : vec)
        ttl += x;
    return ttl;
}
int main() {
    vector<int> vec;
    for(int i=0; i<10; ++i)
        vec.push_back(i)
    for(int x: vec)
        cout << x << " ";
    cout << endl;
    cout << sum(vec) << endl;
```

Vectors

- How to have a sum of double?
 - You can define multiple types functions, even with the same names
 - Called overloading
- What if we compile without it?
- Looks terrible.... Copy pasting causes tons of problems

Vector Example

```
int sum(vector<int>& vec){
    int ttl = 0;
    for(int x : vec)
        ttl += x;
    return ttl;
}

double sum(vector<double>& vec){
    double ttl = 0;
    for(double x : vec)
        ttl += x;
    return ttl;
}
```

Vectors

- Proper way. All that's required, plus be defined for T

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
template <class T>
T sum(vector<T>& vec){
    T ttl{}; // NOTE: Can't set to zero, depends on type T
    for(T x : vec) ttl += x;
    return ttl;
}
int main() {
    vector<int> vec;
    for(int i=0; i<10; ++i)
        vec.push_back(i);
    for(int x: vec)
        cout << x << " ";
    cout << endl;
    cout << sum(vec) << endl;
}
```

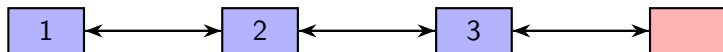
- Can work with anything, even strings

Vector Example

```
#include <iostream>
#include <vector>
#include <string>
using std::cout, std::endl, std::vector, std::string;
template <class T>
T sum(vector<T>& vec){
    T ttl{}; // NOTE: Can't set to zero, depends on type T.
    for(T x : vec) ttl += x;
    return ttl;
}
int main() {
    vector<int> vec;
    vec.push_back(string("Hello"));
    vec.push_back(string("World"));
    for(int x: vec) cout << x << " ";
    cout << endl;
    cout << sum(vec) << endl;
```

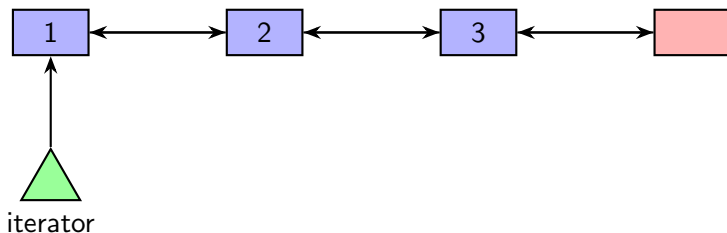

- Using generics in our functions and data types allows us to write one function that works with many types
- Whether it's strings, doubles, ints, or Cars, functions can operate the exact same way when we use templates.

- Iterators are abstractions for indexes into containers
 - The equivalent of a finger pointing to the right place within the container
- Simplest example:
 - An integer
 - To index into a flat C array
- But what if our type gets more complex? Example, a linked list?

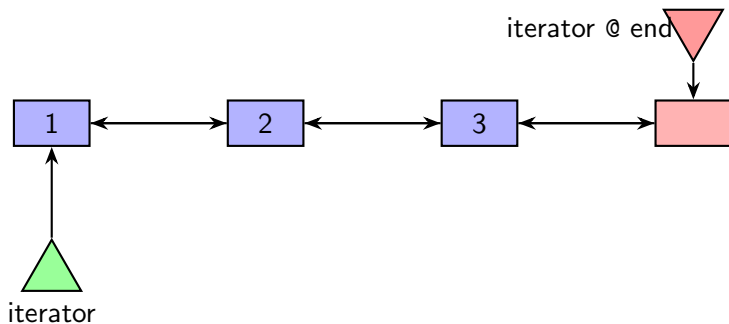


- New concept: Past the end
 - The red block doesn't exist in our three node linked list and is past the end

Iterators

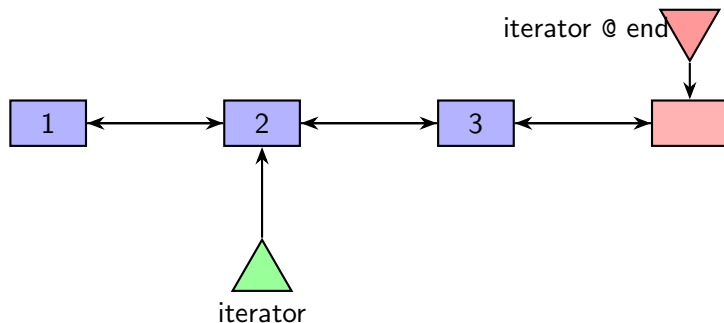


- New concept: Iterator
 - Iterator is pointing at the beginning
 - Iterator is an abstraction that allows you to point to the structure without knowing how it's represented
 - Example: Could iterate over a binary tree



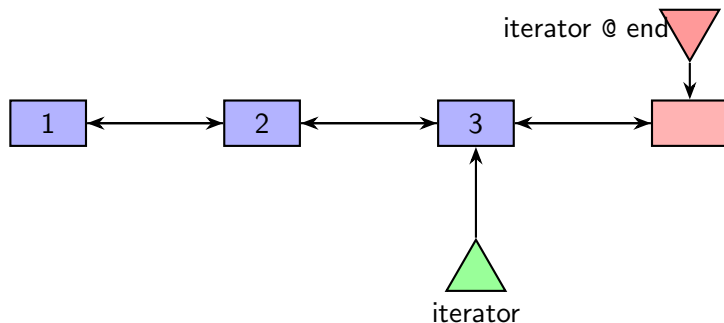
- New concept: Iterator at the end
 - Iterator is pointing at the beginning
 - End iterator conceptually points one past the last element

Iterators



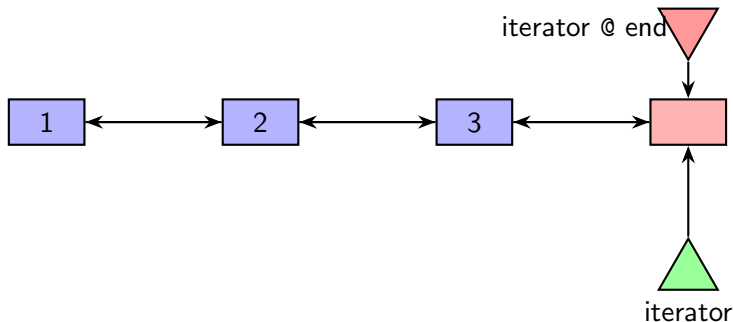
- I can now move the iterator forward...

Iterators



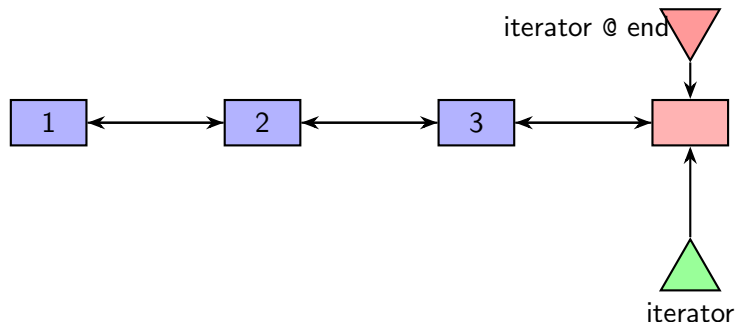
- I can now move the iterator forward...

Iterators



- Now both iterators are pointing to the same place
 - The place past the end
- Now you know your iteration is finished
 - Both iterators are pointing to the same place, we're at the end
- You can use this abstraction for any data structure, regardless of how it looks under the hood

Iterators



- One final important concept on this - ranks
- We have an array with three ranks. At rank 0 we have the value 1. At rank 1 we have the value 2, at rank 2 we have the value 3.
- In an array, ranks == index
 - Not an accident, by design!
 - But for other data types, we use ranks to talk about the *i*th element

- You can
 - Make the iterator point to the beginning
 - Make the iterator point past the end
 - Move the index forward
 - Move the index backwards
 - Move the index to a rank
- Our standard range loop is using these iterators under the hood

C Array Example

- It fits the requirements of an iterable sequence
 - Sequence has n elements
 - Iterator k is simply a variable of type `int`
 - Set at the beginning: $k = 0$
 - Set past the end: $k = n$
 - Set at rank i : $k = i$
 - Move forward: $k++$
 - Move backward: $k--$
 - Test at the end? $k == n$

Consider a simple loop

- You can think of *i* as the iterator, and *n* as the past the end iterator

C Array Example

```
int n = 9;
for(int i=0; i < n; ++i){
    //code
}
```

- The container has methods to manufacture iterators
 - Pointing at the right place (front or end)
 - Going in a direction (forward or backwards or bidirectional)
 - Your code will ask the container to manufacture these iterators for you to use
- An iterator is an object that represents an index
 - Can move forward
 - Can look up the value it is indexing to
 - Can be compared with iterators (ex: to test end)

Vector Example

- Again i is the iterator, but this isn't generic

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0; i<5; ++i){
        vec.push_back(i);
    }
    for(int i=0; i < vec.size(); ++i){
        cout << vec[i] << endl;
    }
    return 0;
}
```

Vector Example

- Now we have a generic loop

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0;i<5;++i){
        vec.push_back(i);
    }
    for(
        vector<int>::iterator it = vec.begin();
        it != vec.end();
        it++
    ){
        cout << *it << " ";
    }
    cout << endl << "Reverse!" << endl;
    return 0;
}
```

Vector Example

- This is going in the reverse direction, do you see the difference?

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0; i<5; ++i){
        vec.push_back(i);
    }
    for(
        vector<int>::reverse_iterator it = vec.rbegin();
        it != vec.rend();
        it++
    ){
        cout << *it << " ";
    }
    cout << endl << "Reverse!" << endl;
    return 0;
}
```


Vector Example

- We can also initialize iterators similar to pointers
- You can both read and write from iterators

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0; i<5; ++i){
        vec.push_back(i);
    }
    vector<int>::iterator on3 = vec.begin() + 2;
    cout << "Rank3 is=" << *on3 << endl;
    *on3 = 17;
    cout << "rank3=" << *on3 << endl;
    return 0;
}
```

Containers Provide...

- Several iterator types (just showing two!)
 - `vector<int>::iterator`
 - `vector<int>::revers_iterator`
 - The loop goes in the iterators direction
- Methods to obtain iterators
 - At either end
 - Either forward or reversed
- The end iterator is pointing past the last element

Iterators Provide...

- A way to move forward
 - The ++ operator
- A way to read at the current index
 - The * operator
- A way to test for the end
 - By comparing to an iterator set at the end
- A way to jump to any rank
 - By performing arithmetic on the iterator
- A way to write at the current index
 - The * operator as a right value

It works!

- Hides how one iterates
- Code is identical regardless of container
- You can have multiple iterators at once
 - Think: Nested loops!

End of day 18

But...

- The code is ugly
- The types are big, unreadable, and hard to remember

Type Inference

- Type inference! Let the compiler figure it out.

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0; i<5; ++i){
        vec.push_back(i);
    }
    for(
        vector<int>::iterator it = vec.begin();
        it != vec.end();
        it++
    ){
        cout << *it << " ";
    }
    cout << endl << "Reverse!" << endl;
    return 0;
}
```

Type Inference

- We shouldn't have to figure it out ourselves when the compiler knows

Vector Example

```
#include <iostream>
#include <vector>
using std::cout, std::endl, std::vector;
int main() {
    vector<int> vec; // vector<double> works here too
    for(int i=0;i<5;++i){
        vec.push_back(i);
    }
    for(
        auto = vec.begin();
        it != vec.end();
        it++
    ){
        cout << *it << " ";
    }
    cout << endl << "Reverse!" << endl;
    return 0;
}
```


- You CAN use auto anywhere, but you shouldn't
- However, for complex types, auto is great

Vector Example

```
auto k = 0; // Less clear than int k = 0;
```

- You can use auto almost anywhere
 - For local var declaration
 - For return types of functions
- It makes the code generic
- But...
 - Not for argument types

Vector Example

```
template <class Container>
void rprint(Container& con){
    for(auto it = con.rbegin(); k!= con.rend(); ++k){
        cout << *k << " ";
    }
    cout << endl;
}

int main(){
    vector<int> vec;
    for(int i = 0; i<10; ++i){
        vec.push_back(i);
    }
    rprint(vec);
}
```

Same code... different type

Vector Example

```
template <class Container>
void rprint(Container& con){
    for(auto it = con.rbegin(); k!= con.rend(); ++k){
        cout << *k << " ";
    }
    cout << endl;
}

int main(){
    list<int> vec;
    for(int i = 0; i<10; ++i){
        vec.push_back(i);
    }
    rprint(vec);
}
```

Auto is helpful

- As long as you have a good sense of the types
 - For now, write out most of them so they stick
 - Be careful as you learn!
 - Once you are comfortable, you can replace the types with auto

Vectors

- You can also use references in your range loops

Vector Example

```
#include <iostream>
#include <vector>
#include <string>
using std::cout, std::endl, std::vector, std::string;
template <class T>
T sum(vector<T>& vec){
    T ttl{}; // NOTE: Can't set to zero, depends on type T.
    //for(T x : vec) ttl += x;
    for(const auto &x : vec) ttl += x;
    return ttl;
}
int main() {
    vector<int> vec;
    vec.push_back(string("Hello"));
    vec.push_back(string("World"));
    for(int x: vec) cout << x << " ";
    cout << endl;      cout << sum(vec) << endl;
```

In-Class Coding Problem: Reverse Words with STL

Problem Statement

Write a C++ program that:

- 1 Takes the sentence:
`C++ is so much fun`
- 2 Splits it into words and stores them in a `std::vector<std::string>`.
- 3 Print the words in reverse order using a reverse iterator on the vector.
- 4 You may need `iostream`, `string`, `vector`, and `sstream` headers. `istringstream` may also be helpful.

Reverse Words with STL — Solution

Solution

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

int main() {
    std::string sentence = "C++ is so much fun";
    vector<string> words;
    istringstream iss(sentence);
    string word;
    while (iss >> word)
        words.push_back(word);
    for (auto it = words.rbegin(); it != words.rend(); ++it)
        cout << *it << " ";
    cout << endl;
    return 0;
}
```


- Algorithms are another important part of the STL
- Algorithms are also a heavy user of templates
- The STL provides algorithms
 - Works on elements in containers
 - Doesn't operate on the container themselves, mainly the elements within the containers
 - Iterates over the container
 - Highly reusable; Same algo: many containers
- Relies on iterators, templates, and lambdas.

Find algorithm

Find Example

```
template <class T> // NOTE: Sorted sequential containers
bool find(std::vector<T>& x, const T& v, std::function<int(
    double, double)> cmp){
    int low = 0; int up = x.size() - 1;
    while(low <= up){
        const int mid = (low + up) / 2;
        int c = cmp(x[mid], v);
        if (c == 0){return true;}
        else if (c > 0){up = mid - 1;}
        else{low = mid + 1;}
    }
    return false;
}
```

Find algorithm

- There exists a container in the STL called find as well
- Works with any containers, not just vectors
- Also doesn't require the container to be sorted!
 - Not logarithmic time search... linear search
 - Remember, $\log_2 n$ for binary search, since for every doubling, only one extra operation

Find algorithm

- Current implementation
 - Operates on the entire container
 - Restricted to a pure equality test
 - Assumes that the container is sorted

- What should the STL algo look like?

Find Example

```
int main(){
    std::vector<int> x = {1, 20, 5, 12, 34, 17}; // not
        sorted!
    ??? = my_find(???, 17);
    bool found = ???;
    std::cout << "found?" << found << std::endl;
    if(found){
        std::cout << "value=" << ??? << endl;
    }
}
```

Using find

- We get back an iterator!
 - Can dereference for value
 - if we are at the end (past container), not found
 - Remember: .end is PAST THE END!

Find Example

```
#include <vector>
#include <iostream>
int main(){
    std::vector<int> x = {1, 20, 5, 12, 34, 17}; // not
        sorted!
    auto at = my_find(x.begin(), x.end(), 17);
    bool found = (at != x.end());
    std::cout << "found?" << found << std::endl;
    if(found){
        std::cout << "value=" << *at << endl;
    }
}
```

Find Example

```
#include <vector>
#include <iostream>
template<class InputIterator, class T>
InputIterator myfind(InputIterator from, InputIterator to,
    const T& v){
    while(from != to){
        if (*from == v){return from;}
        ++from;
    }
    return from;
}

int main(){
    std::vector<int> x = {1, 20, 5, 12, 34, 17}; // not
        sorted!
    auto at = myfind(x.begin(), x.end(), 17);
    bool found = (at != x.end());
    std::cout << "found?" << found << std::endl;
    if(found){std::cout << "value=" << *at << std::endl;}
```

Problems to solve

- We have now solved all of these problems:
- Where to search within the container?
 - The whole thing?
 - Less?
- What should we get back as a result
 - Depends on what we wish to do?
 - What is most flexible?
- What if the search failed?
 - We didn't find it, how do we tell?

We have the API for find!

- Use iterators
 - For arguments
 - For results
- Use compile-time composition (templates)
- Use a constant reference for the value being sought

Implementation

- Uses a linear scan
- Uses equality operator - T must implement!
 - If you use this on space station class without equality, it will error!

Find Example

```
#include <vector>
#include <iostream>
template<class InputIterator, class T>
InputIterator myfind(InputIterator from, InputIterator to,
    const T& v){
    while(from != to){
        if (*from == v){return from;}
        ++from;
    }
    return from;
}
```

- So far...
 - No sorting requirements
 - Capable of handling arbitrary ranges
 - Iterator state conveys success!
- Establish success by testing for iterator against the end iterator
 - Remember - this is past the end of the container!
- One can use the iterator to read from or write to the container!
 - We know both the value, and where it is, so we can overwrite if need be

What about the equality test?

- We wish to separate the condition for finding
 - No longer a built in equality test
 - Let the programmer choose
- Let's generalize our find function to have an arbitrary equality test.

Go for lambda

- STL has a similar implementation called `find_if`
- `myfind_if` takes a function that takes in a `T` and returns a `bool` (called a predicate)

Find Example

```
#include <vector>
#include <iostream>
int main(){
    std::vector<int> x = {1, 20, 5, 12, 34, 17}; // not
        sorted!
    auto at = myfind_if(x.begin(), x.end(), [](int v) ->
        bool {return v == 17;});
    bool found = (at != x.end());
    std::cout << "found?" << found << std::endl;
    if(found){std::cout << "value=" << *at << std::endl;}
}
```

Go for lambda

- How does the C++ compiler determine the type of predicate?
 - We know that `*from` is of type `T`, so `T` is going in
 - We know that it evaluates to a boolean since we are checking in an if block
 - We can also see that predicate is a function
 - So predicate is a function that takes in type `T` and returns a boolean value, and the C++ compiler deduces all of that for us.
 - No need to try and deduce the type ourselves!

Find Example

```
template<class InputIterator, class Predicate> // or
typename Predicate
InputIterator myfind(InputIterator from, InputIterator to,
    Predicate p){
    while(from != to){
        if (p(*from)){return from;}
        ++from;
    }
    return from;
}
```

Containers are interchangeable

- list (linked list) instead of vector (make sure to include)
- nothing else changes!

Find Example

```
#include <vector>
#include <list>
#include <iostream>
int main(){
    std::list<int> x = {1, 20, 5, 12, 34, 17}; // not sorted
    !
    auto at = myfind_if(x.begin(), x.end(), [](int v) ->
        bool {return v == 17;});
    bool found = (at != x.end());
    std::cout << "found?" << found << std::endl;
    if(found){std::cout << "value=" << *at << std::endl;}
}
```

So far...

- Remember, still no sorting assumption
- Capable to handle arbitrary ranges
- Iterator state conveys success!
- Variant to handle external condition by passing lambda

Summary

- Iterators are great for range and for results
 - Very similar to pointers
- Templates can be used for composition
 - We can use predicates and C++ is doing a lot of the type inference for us
- Lambdas are very useful for generality
- You can check the docs for further details
 - <http://en.cppreference.com/w/cpp/algorithm>

Adapters

- Adapters are essential to make the best out of the containers and algorithms that we have
- To start, let's look at a function to copy one container into another
- Here `back_inserter` is an adapter!

Adapter Example

```
#include <vector>
#include <iostream>
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b;
    mycopy(a.begin(), a.end(), std::back_inserter(b));
    for(int v : b){
        std::cout << v << " ";
    }
    std::cout << std::endl;
}
```

Adapters

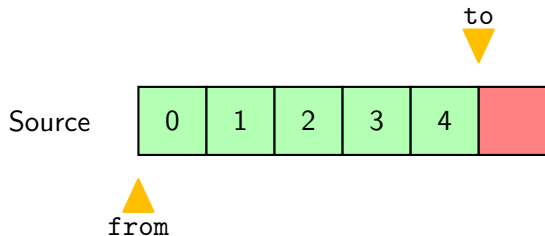
- Here is the implementation

Adapter Example

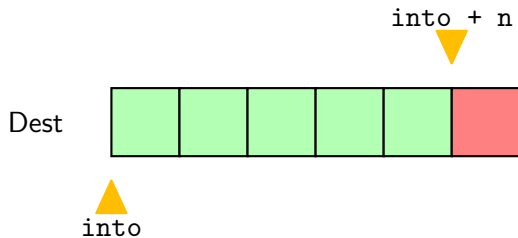
```
#include <vector>
#include <iostream>
#include <algorithm>
#include <iterator>

// typename=class. Input to read, output to write
template <typename InputIterator, typename OutputIterator>
OutputIterator mycopy(InputIterator from, InputIterator to,
    OutputIterator into){
    while(from != to){
        *into = *from;
        ++from;
        ++into;
    }
    return into;
}
```

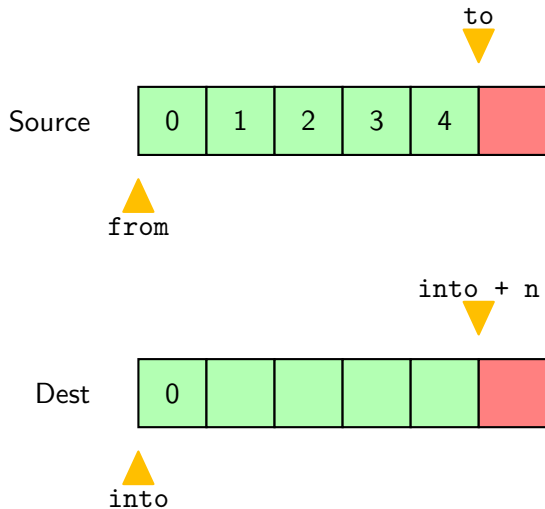
Adapters



Before running:



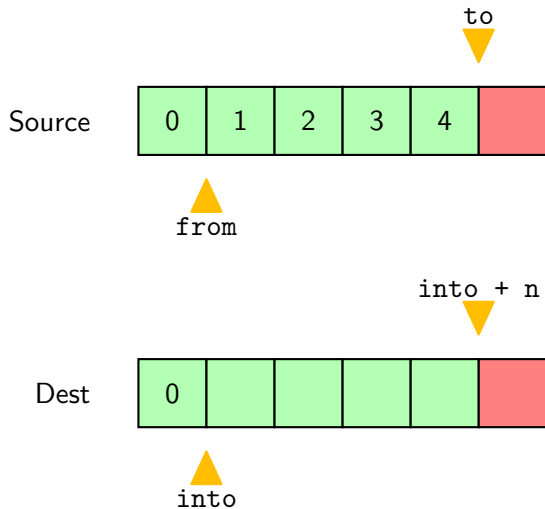
Adapters



After running:

```
*into = *from;
```

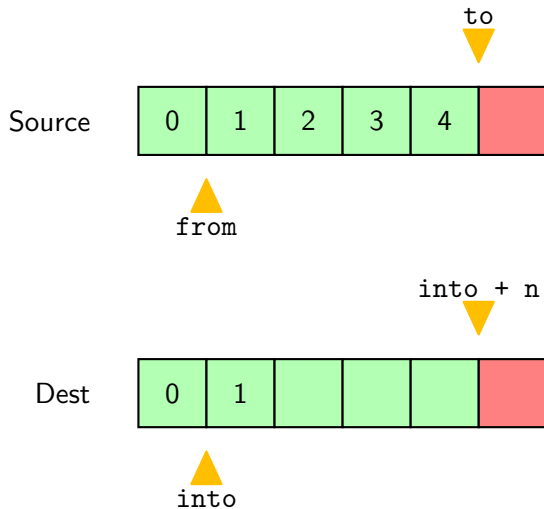
Adapters



After running:

```
++from; ++into;
```

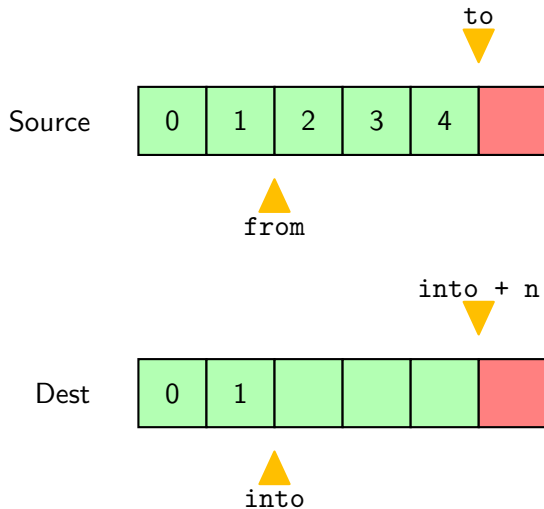
Adapters



After running:

```
*into = *from;
```

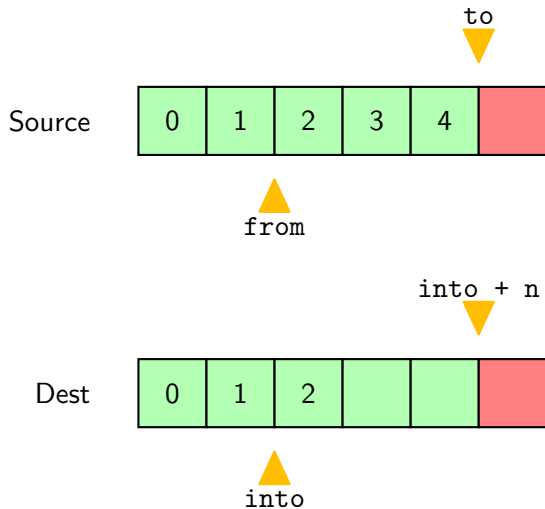
Adapters



After running:

```
++from; ++into;
```

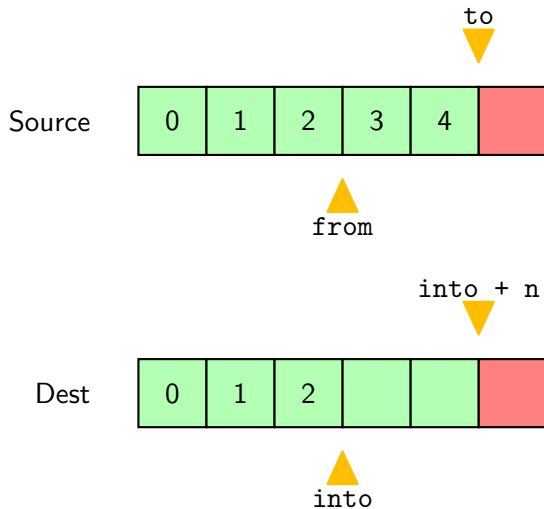

Adapters



After running:

```
*into = *from;
```

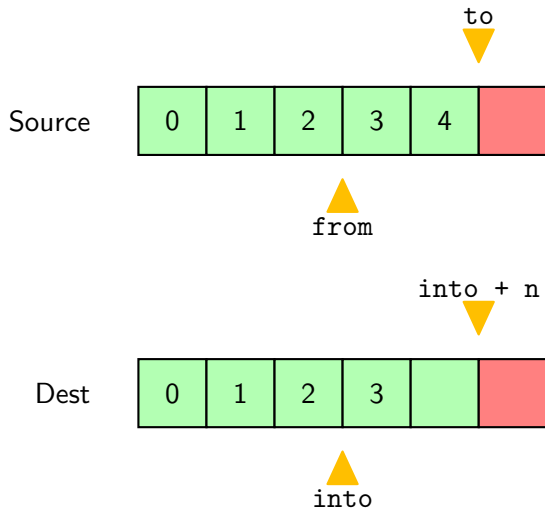
Adapters



After running:

```
++from; ++into;
```

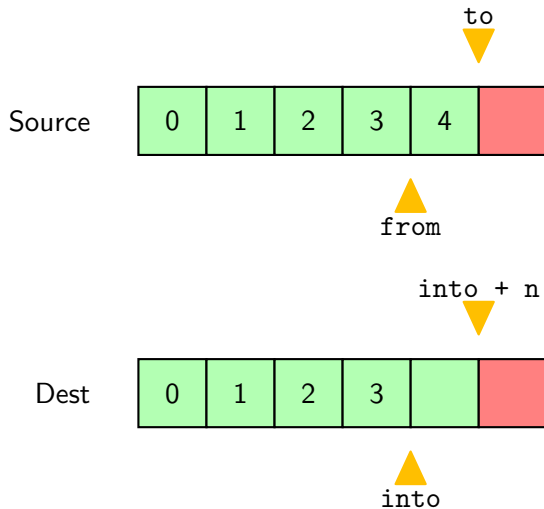
Adapters



After running:

```
*into = *from;
```

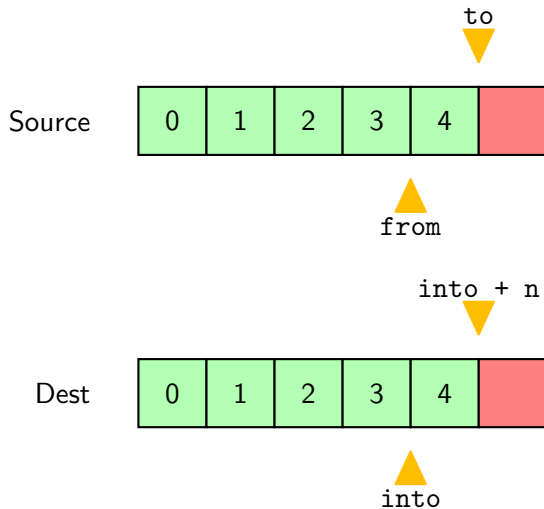
Adapters



After running:

```
++from; ++into;
```

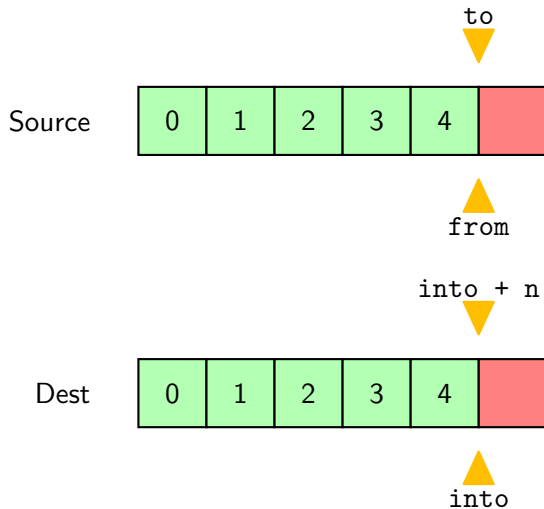
Adapters



After running:

```
*into = *from;
```

Adapters



After running:

```
++from; ++into;
```

- Here in this case the into iterator is actually our adapter!

Purpose?

- Compose more primitive algorithms
- For example:
 - We need a concatenate algorithm
 - Given two containers A and B
 - Inserts at the end of B everything that is in A
 - similar to extend in python

- How to call our function from main

Adapter Example

```
int main(){
    std::vector<int> a = {0, 1, 2, 3};
    std::vector<int> b = {4, 5, 6};
    insertInto(b, a.begin(), a.end());
    for(int v:b){
        std::cout << v << " ";
    }
    std::cout << std::endl;
}
```

Quite Direct

- Implementation for our insertInto

Adapter Example

```
template <class Container, class InputIterator>
void insertInto(Container& b, InputIterator from,
    InputIterator to){
    while(from != to){
        b.push_back(*from);
        ++from;
    }
}
```

aka

```
template <class Container, class InputIterator>
void insertInto(Container& b, InputIterator from,
    InputIterator to){
    while(from != to)
        b.push_back(*from++);
}
```

What about our copy function?

- Remember from previous slides that we used a copy function that inserted from one array into another using the iterator
- Why do we need a new insertInto function?
 - We already have a copy function that copies to an iterator and advances the iterator
 - Maybe we can just use that but copy to the end?
- We don't want to implement new code for something our previous function might already do since it's mostly copy paste
 - If we have a bug in one place we'll be copy pasting it all over

Copy Algorithm

- Quite Direct
- Uses input iterators to read from source (from, to)
- And output iterators to write to output (into)

Implementation

Adapter Example

```
#include <vector>
#include <iostream>
#include <algorithm>
#include <iterator>
template <typename InputIterator, typename OutputIterator>
OutputIterator mycopy(InputIterator from, InputIterator to,
    OutputIterator into){
    while(from != to)
        *into++ = *from++;
    return into;
}
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b = {5, 6, 7, 8}
    mycopy(a.begin(), a.end(), b.end());
    for(int v : b){
        std::cout << v << " ";
    } std::cout << std::endl;
```

Error!

- Errors out when running, what could be going on?
- `b.end()` is pointing to an element that doesn't exist!
 - We can't write to a block of memory that `b` doesn't own!
 - `b.end()` points to a place that doesn't exist!
- `b.end` is an output iterator but it can only output into existing locations
- Iterators act only on the elements of the container, and can't create new memory blocks
 - How can we do this? Maybe we can have a smarter iterator that creates a new memory cell before writing to it if such a cell does not yet exist...

- For this, we can use adapters
- Adapters look like a duck, quack like a duck, but isn't quite a duck.
 - They look like an output iterator, and act like an output iterator, but aren't quite an output iterator.
 - It can write to a memory cell that already exists
 - It can advance
 - It can do one thing iterators can't - create new memory cells!

- We can have a special OutputIterator that
 - Inserts at the end of the container, adding an element on the fly
 - Responds differently to the * operator by first creating the cell then writing
 - And presents itself as an OutputIterator to copy
 - Message++

Implementation

Adapter Example

```
#include <vector>
#include <iostream>
#include <algorithm>
#include <iterator>
template <typename InputIterator, typename OutputIterator>
OutputIterator mycopy(InputIterator from, InputIterator to,
    OutputIterator into){
    while(from != to)
        *into++ = *from++;
    return into;
}
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b = {5, 6, 7, 8}
    mycopy(a.begin(), a.end(), std::back_inserter(b);    //
        Adapter
    for(int v : b){
        std::cout << v << " ";
    }
```

- `back_inserter` takes a generic container and creates an end iterator on steroids
- Acts like an output iterator but with all these extra features!
 - We can now create new cells that we want to write to
- Now our code compiles and runs well

Templates and Type Inference

- Template Instantiation used loosely
- We need to understand what's really going on

Templates and Functions

- Consider this max function
- The way we use max is very different from how we instantiate a container. But both are templates!
- But when we use max, we can simply drop the <>... how does the compiler know?

Adapter Example

```
#include <iostream>
template <typename T>
T max(T a, T b){
    return a > b ? a : b;
}
int main(){
    int x = 3;
    int y = max(3, 5);
    std::cout << "max=" << y << std::endl;
}
```

Templates and Functions

- This is valid syntax! But you don't need the <>...
- Why is it optional in some cases but not others?
 - Type inference!

Adapter Example

```
#include <iostream>
template <typename T>
T max(T a, T b){
    return a > b ? a: b;
}
int main(){
    int x = 3;
    int y = max<int>(3, 5);
    double z = max<double>(3, 5.25);
    std::cout << "max=" << y << std::endl;
    std::cout << "max=" << z << std::endl;
}
```

Original Copy

```
template <typename InputIterator, typename OutputIterator>
OutputIterator mycopy(InputIterator from, InputIterator to,
    OutputIterator into){
    while(from != to)
        *into++ = *from++;
    return into;
}
```

Calling Copy

Original Calling Copy

```
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b = {5, 6, 7, 8}
    mycopy(a.begin(), a.end(), std::back_inserter(b)); //
        Adapter
    for(int v : b){
        std::cout << v << " ";
    } std::cout << std::endl;
}
```

Calling Copy

- This is insane!
- Who can remember this??
- The type of the two inputs and output iterators passed in as arguments are already too much!

Calling Copy - With Types!

```
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b = {5, 6, 7, 8}
    mycopy<std::vector<int>::iterator, std::
        back_inserter<std::vector<int>>>(a.begin(), a
        .end(), std::back_inserter(b));
    for(int v : b){
        std::cout << v << " ";
    } std::cout << std::endl;
}
```


Calling Copy

- Much easier!
- The compiler knows the types of the iterators, why would we need to figure it out?
 - This is something the compiler does to make our lives better!
- Type inference works not only with auto, but also with templates!

Original Calling Copy

```
int main(){
    std::vector<int> a = {0, 1, 2, 3, 4};
    vector<int> b = {5, 6, 7, 8}
    mycopy(a.begin(), a.end(), std::back_inserter(b)); //
        Adapter
    for(int v : b){
        std::cout << v << " ";
    } std::cout << std::endl;
}
```

Templates... but with Constraints

- So far our templates accept *any* type
- But what if the type doesn't support what we need?
 - Example: `a > b`
 - Example: `a + b`
 - Example: `*it`
- The compiler only complains deep inside the template
- Can we express our requirements clearly up front?

Enter: Concepts (C++20)

- Concepts constrain template parameters
- They describe what a type must be able to do
- They are checked at compile time
- They produce clearer error messages

Basic Form

```
template<typename T>  
concept Name = /* requirement */;
```

First Real Concept (Step by Step)

Comparable Concept

```
#include <concepts>

template<typename T>
concept Comparable =
    requires(T a, T b) {
        { a > b }
        -> std::convertible_to<bool>;
    };
```

- Let's break this down carefully...

Breaking Down the Syntax

- **template<typename T>**
 - We are parameterizing on T
- **concept Comparable =**
 - Defines a compile-time boolean rule
- **requires(T a, T b)**
 - Introduces a requirements expression
 - a and b are imaginary placeholders
 - No runtime objects are created

Breaking Down the Syntax (continued)

- `{ a > b }`
 - The expression must compile
- `-> std::convertible_to<bool>`
 - The result must satisfy a concept
 - Not just compile, the value must be usable as `bool`
- If all requirements succeed, then the concept evaluates true

Why Do We Need the Arrow Syntax?

- Without arrow syntax:

Weak Requirement

```
requires(T a, T b) {  
    a > b;  
}
```

- Only checks that the expression compiles
- Does NOT constrain the return type

Compound Requirement

```
{ expression } -> Concept;
```

- Expression must compile
- The expression's type must satisfy the Concept
- Reads as:
 - "The result of this expression models this concept"

Why Not Just Require a Type?

- Concepts are not just types
- They are boolean predicates on types
- We want flexibility:
 - Exactly this type?
 - Convertible to this type?
 - Derived from this type?
- Arrow syntax allows arbitrary concept checks

The Four Kinds of Requirements

Inside a `requires` block we can write:

- **Simple Requirement**

- `a + b;`
- Only checks it compiles

- **Compound Requirement**

- `{ a + b } -> std::same_as<int>;`

- **Type Requirement**

- `typename T::value_type;`

- **Nested Requirement**

- `requires sizeof(T) > 4;`

Common Return-Type Concepts

Most common concepts used after the arrow:

- `std::convertible_to<T>`
 - Implicitly convertible to T
- `std::same_as<T>`
 - Exactly the same type
- `std::derived_from<T>`
 - Inherits from T
- `std::integral`
 - Any integer type
- `std::floating_point`
 - Floating point types

More Standard Library Concepts

- `std::signed_integral`
- `std::unsigned_integral`
- `std::copyable`
- `std::movable`
- `std::default_initializable`
- `std::equality_comparable`
- `std::totally_ordered`
- These are building blocks for precise constraints

Constrained max

```
template<Comparable T>
T max(T a, T b){
    return a > b ? a : b;
}
```

- Now max only works for Comparable types
- Constraint failure is clean and immediate

Requires Clause

```
template<typename T>  
requires Comparable<T>  
T max(T a, T b){  
    return a > b ? a : b;  
}
```

- Same logic
- Different placement of constraint

End of Day 19

C++ Programming with Class(es)

- Let's dive deeper into C++ classes
- With this topic you'll know most of what you need to be effective at C++

- Classes and objects
- Privacy
- Attributes
- Constructors
- Destructors
- Members

- Recall that there are two families of object oriented languages
- Class based
 - C++
 - C#
 - Java
- Object based
 - Smalltalk
 - Objective C
- We'll be focusing on the Class based style

- A class is a TYPE
- It describes a set of objects that
 - Share some properties
 - Meet some requirements
- Adding properties or requirements
 - Constrains the type
 - Fewer objects have the properties or requirements
 - Ex: Many students. Fewer CSE3150 students. Etc.
 - This is subclassing

Objects?

- We call them instances of a class
 - Simply members of the set of objects denoted by the class
- When a program starts
 - It has a lot of classes
 - But often no instances
 - You can declare classes at the start of runtime with static... but don't
 - Instances are generally created at runtime

Student

- Let's define a class, Student!

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
struct Student { // In c name @ end, in C++ @ start
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
};
```

student.h Example in C

```
typedef struct { // No need to do this anymore
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
} A
```

What is going on?

- Reuse the struct concept of C
 - Attrs of the structure are attrs of the class
 - A struct fits the definition of a class
 - There is a catch though!

- Reuse the "struct" concept of C
 - Attrs of struct are attrs of the class
- A struct fits the definition of a class
 - There is a catch though

student.cpp

```
#include "student.h"
#include <memory>
#include <iostream>
int main(){
    Student s; // Stack
    //std::shared_ptr<Student> sp(new Student); //heap
    // automatically destroys object when all shared
    // pointers are out of scope
    std::shared_ptr<Student> sp = std::make_shared<Student>
        >(); //heap
    s.name_ = "Joe";
    s.mt_ = 80;
    s.final_ = 90;
    s.homeworks_.push_back(65);
    s.homeworks_.push_back(42);
    s.homeworks_.push_back(96);
}
```


student.cpp

```
#include "student.h"
#include <memory>
#include <iostream>
int main(){
    Student s; // Stack
    auto sp = std::make_shared<Student>();
    //...
    *sp = s; // Copies stack obj onto heap
    std::cout << "s.name_" << s.name_ << std::endl;
    std::cout << "sp->name_" << sp->name_ << std::endl;
    std::cout << "&s.name_" << &s.name_ << std::endl;
    std::cout << "&sp->name_" << &sp->name_ << std::endl;
    s.name_ = "Bert";
    std::cout << "s.name_" << s.name_ << std::endl;
    std::cout << "sp->name_" << sp->name_ << std::endl;
}
```

- The structure in our previous example only bundled the attributes
 - It does not enforce access mechanisms
 - It does not provide high level operations
 - It does not provide a contract
- This is essentially structured programming

- We must combine both state, and behavior, and have strong cohesion between both
- Must couple state and behavior
 - We must create abstractions for the students
- Things need to be related to be bundled by classes
- Classes also have protection for state... not accessing, reading, writing private variables
- And there should be a contract for such behavior... separation between interface and implementation

- Handle
 - State and Behavior
 - Lifetime
 - Privacy
 - Contracts

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    // Unlike Python, Java, you write the code for funcs
    elsewhere
    void read(std::istream& is);
    void print(std::ostream& os);
};
```

student.cpp Example in C++

```
#include "student.h"

void Student::read(std::istream& is){
    is >> name_ >> mt_ >> final_; // whitespace splits
    // stream into name, midterm final
    // Attrs don't need self!
    int nbH = 0;
    is >> nbH;
    for(int i = 0; i < nbH; ++i){
        int v; is >> v; homeworks_.push_back(v);
    }
}

void Student::print(std::ostream& os){
    os << "Name:" << name_ << "[" << mt_ << "," << final_ <<
        "]" << std::endl;
    for(int v : homeworks_){
        os << "\t" << v << std::endl;
    }
}
```

- You can overload methods. Several definitions with:
 - Same name
 - Different number of arguments
 - Different types of arguments
- C++ disambiguates the call and selects the right method
- overloading is not overriding!

- Birth
 - With constructors
- Transfers
 - With copy operators
- Death
 - With destructors

Birth: Constructor

- Simple Idea
 - Go to a memory pool (heap or stack) and grab memory
 - Either grab mem from stack (automatic)
 - Or use the keyword new on the heap
 - Then initialize
 - Executes a method when creating an object
 - Can overload constructors (multiple ways to initialize)
- Multiple kinds of constructors
 - Default
 - Customer
 - Copy
 - Move

Default Constructor

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    // If you forget to initialize a var, undefined!
    // Except for objects, which get the default constructor
    // called before entering the constructor. This is the
    // default constructor:
    Student() { mt_ = final_ = 0; }
    void read(std::istream& is);
    void print(std::ostream& os);
};
```

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    Student() { mt_ = final_ = 0; } // Default
    // Overloaded creates members, then reassigns values
    Student(const std::string& s) {name_=s;mt_=final_=0;}
    // Better: use initializer list (direct initialization,
        no reassignment)
    // Must be consistent with the ordering above!
    Student(const std::string& s):name_(s),mt_(0),final_(0)
    {}
    void read(std::istream& is);
    void write(std::ostream& os) const;
```

Copy Constructor

main.cpp Example in C++

```
//How you call a copy constructor
Student x;
// Code that messes with and creates student x
Student y(x); // Creating a student from a student invokes
               the copy constructor
```

Copy Constructor

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    Student() { mt_ = final_ = 0; } // Default
    Student(const std::string& s):name_(s),mt_(0),final_(0)
        {} // Custom
    Student(const Student& s2){ // Makes a copy constructor
        name_ = s2.name_; mt_ = s2.mt_; final_ = s2.final_;
        for(double d: s2.homeworks_){homeworks_.push_back(d)}
    }
    void read(std::istream& is);
    void write(std::ostream& os) const;
```

Copy Constructor

- If you omit the copy constructor, the compiler provides one for you
 - Shallow copy
- Shallow copies just change pointers, not values (think Python)
- Deep copies - compiler doesn't do. You must implement!

Copy Constructor, take 2

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    Student() { mt_ = final_ = 0; } // Default
    Student(const std::string& s):name_(s),mt_(0),final_(0)
        {} // Custom
    Student(const Student& s2):
        name_(s2.name_),
        mt_(s2.mt_),
        final_(s2.final_),
        homeworks_(s2.homeworks_) // vectors have a copy
        constructor
}
```

Move Constructor

- Purpose: transfer ownership of resources instead of copying
 - Avoids expensive deep copies
 - Leaves source object in a valid but unspecified state
- Syntax: takes an rvalue reference ($T\&\&$)
- Used automatically when returning large objects from functions

Move Constructor Example

student.h Example in C++

```
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;

    // Move constructor
    Student(Student&& other)
        : name_(std::move(other.name_)),
          mt_(other.mt_),
          final_(other.final_),
          homeworks_(std::move(other.homeworks_)) {}

    // Reset source object (optional)
    // other.mt_ = other.final_ = 0;
};
```

Move Constructor in Action

main.cpp Example in C++

```
#include "student.h"
#include <iostream>

Student make_student() {
    Student s;
    s.name_ = "Alice";
    s.homeworks_ = {90, 85, 100};
    return s; // Move constructor used
}

int main() {
    Student s1 = make_student();
    std::cout << s1.name_ << std::endl;
}
```

Copy vs Move

Feature	Copy Constructor	Move Constructor
Ownership	Duplicate resource	Transfer ownership
Cost	Potentially expensive	Cheap (pointer swap)
When used?	lvalue	rvalue / temporary
Example	Student y(x);	Student y(std::move(x));

Lvalues: Think "Named Object"

- Lvalue = object with a **name and address**
- Sticks around after the current line of code
- You can take its address with &

```
int x = 42;           // x is an lvalue
x = 100;              // can assign to it
int* p = &x;          // OK: has an address
```

Rvalues: Think "Temporary"

- Rvalue = temporary object / literal
- No persistent address, vanishes after the statement
- Cannot take its address directly

```
10          // literal, rvalue
x + 2       // result is temporary, rvalue
std::string("hi") // temporary string

// int& bad = 10; // ERROR: can't bind lvalue ref to rvalue
```

- **Lvalue:** House with an address
 - You can return to it any time
- **Rvalue:** Hotel guest
 - Exists briefly, then disappears
 - Best option: "take their stuff" before they leave (move!)

T& vs T&&

- T& = reference to lvalue (named object)
- T&& = reference to rvalue (temporary)

```
int x = 42;  
int& ref1 = x;      // OK: binds to lvalue  
// int& ref2 = 5;    // ERROR  
  
int&& rref1 = 5;     // OK: binds to rvalue  
int&& rref2 = x+3;   // OK: binds to temporary
```

When to Write a Move Constructor

- If your class manages resources directly (raw `new`, file handles, sockets)
- For classes that contain only STL containers (`string`, `vector`, etc.):
 - Compiler-generated move constructor is usually sufficient
- Rule of 5: if you write one of
 - Destructor
 - Copy constructor
 - Copy assignment operator
 - Move constructor
 - Move assignment operator

you often need to write the others. Without a move constructor, the compiler often falls back to the copy constructor.

Transfer Operators

- Simple idea
 - Execute a method by invoking the assignment operator
 - Ex: `x = y; int z = 5;`
- When assigning an object, it's a little more complicated
 - You have the ability to redefine what happens when you transfer objects
 - You can have multiple transfer operators, such as copy and move (similar to what we've seen in constructors)

Refresh your memory - Student constructors

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    Student() { mt_ = final_ = 0; } // Default
    Student(const std::string& s):name_(s),mt_(0),final_(0){} // Custom
    Student(const Student& s2): name_(s2.name_), mt_(s2.mt_), final_(s2.final_),
        homeworks_(s2.homeworks_) {}// vectors have a copy constructor
    Student(Student&& other) // Move constructor
        : name_(std::move(other.name_)),
          mt_(other.mt_),
          final_(other.final_),
          homeworks_(std::move(other.homeworks_)) {}
    void read(std::istream& is);
    void print(std::ostream& os);
};
```

Adding our assignment operator

student.h Example in C++

```
#pragma once
#include <string>
#include <vector>
#include <istream>
#include <ostream>
struct Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
    Student() { mt_ = final_ = 0; } // Default
    Student(const std::string& s):name_(s),mt_(0),final_(0){} // Custom
    Student(const Student& s2): name_(s2.name_), mt_(s2.mt_), final_(s2.final_),
        homeworks_(s2.homeworks_) {}// vectors have a copy constructor
    Student(Student&& other) // Move constructor
        : name_(std::move(other.name_)),
          mt_(other.mt_),
          final_(other.final_),
          homeworks_(std::move(other.homeworks_)) {}
    Student& operator=(const Student& s){ // Redefining operator =. Returns ref.
        name_ = s.name_;
        mt_ = s.mt_;
        final_ = s.final_;
        homeworks_ = s.homeworks_;
        return *this;
    }
    void read(std::istream& is);
    void print(std::ostream& os);
};
```

Why assignment operator returns a reference

- This is perfectly legal syntax in C++
- The assignment of student to z returns a reference to z
- The reference of z is then assigned to y
- The reference of y is then assigned to x

student.h Example in C++

```
int x = y = z = Student();
```

Why assignment operator takes a const reference

- When we do assignment we want a reference for performance reasons (we don't want to copy it)
- When we assign something, we don't want to unexpectedly be modifying the object!

student.h Example in C++

```
int x = y = z = Student();
```

Adding our assignment operator

- Why return `*this`?
- The keyword `this` is a pointer to our instance student
- We dereference the pointer to get back the student
- The C++ compiler looks at the return type and binds `*this` to a reference
- For strings and vectors, assignment operators are also called

student.h Example in C++

```
struct Student {  
    ...  
    Student& operator=(const Student& s){ // Redefining  
        operator =. Returns ref.  
        name_ = s.name_;  
        mt_ = s.mt_;  
        final_ = s.final_;  
        homeworks_ = s.homeworks_;  
        return *this;  
    }  
};
```

Adding our assignment operator

- What's the runtime?
- Not $O(1)$!
- $O(n + m)$
 - The size of the name string and the size of the homeworks vector

student.h Example in C++

```
struct Student {  
    ...  
    Student& operator=(const Student& s){  
        name_ = s.name_;  
        mt_ = s.mt_;  
        final_ = s.final_;  
        homeworks_ = s.homeworks_;  
        return *this;  
    }  
};
```

Adding our assignment operator

- Let's refine this a bit further to prevent $x = x$;

bad assignment

```
Student& foo(Student& y){  
    if (condition_here){return y;}  
    else {complicated stuff;}}  
x = foo(x); // x = x;
```

student.h Example in C++

```
struct Student {  
    Student& operator=(const Student& s){  
        if (this == &s){return *this;}  
        name_ = s.name_;  
        mt_ = s.mt_;  
        final_ = s.final_;  
        homeworks_ = s.homeworks_;  
        return *this;  
    }  
};
```


Adding our assignment operator

- So what happens when $x = x$?
- Every member of student is copied... to itself!
- For example, the name attribute is an object of type string
- To copy our name into another field name, we:
 - Free the memory that name is occupying on the heap
 - Move the pointer to the new memory on the heap for name on the copy
 - But if you are copying yourself to yourself... you've just deallocated your own memory and then moved the pointer to a block of deallocated memory!
 - Doesn't work; which is why we need our safeguards

Adding our assignment operator

- What does the compiler do by default?
 - Exactly what we wrote here
- So when do we need to do this?
 - Same as before - when we want deep copies instead of shallow copies.
 - But if you're using normal types like strings, vectors, these come with proper deep copy assignment operators
- `x = x` also handled by string and vector, no need for concern. Only when you are using unsafe types like raw pointers, manually managed memory, etc
 - Smart pointers fix this too!

student.h Example in C++

```
struct Student {  
    Student& operator=(const Student& s){  
        name_ = s.name_;  
        mt_ = s.mt_;  
        final_ = s.final_;  
        homeworks_ = s.homeworks_;  
        return *this;  
    }  
};
```

Adding our assignment operator

- We can also override the assignment operator for move
- This is for what's called an rvalue, a short lived value in C++

student.h Example in C++

```
struct Student {  
    Student& operator=(Student&& s){  
        name_ = std::move(s.name_);  
        mt_ = s.mt_;  
        final_ = s.final_;  
        homeworks_ = std::move(s.homeworks_);  
        return *this;  
    }  
};
```

Our final lifetime of classes: Destructors

- We use destructors whenever we want to release memory held by the instance
 - Memory
 - Files
 - Network connections (sockets)
- Easier to use shared pointer!
 - No need to manually delete or reclaim
- Similar to dunder del in Python; Invoked when the object dies

Destructor

- There's no overloading the destructor, only one per class
- Always starts with the tilde
- Destructors are called implicitly and they are recursive in nature (ex: Student will call destructor on each member like string and vector)
 - Often implicitly called when exiting the stack, or when delete is called to free the memory

student.h Example in C++

```
struct Student {  
    ~Student() {  
        std::cout << "Destroy!" << this << std::endl;  
    }  
};
```

How might these all be called?

- Files located on huskyct

Sidenote: Rust

- This is fairly complicated!
- Very important in general to stick to smart pointers and STL classes... then the compiler implementation works fine
- Copy constructors are particularly tricky...
 - Rust, a type safe version of C++, actually fixes this by removing the copy constructors from C. Only moves are allowed by default (much safer).
 - While there are some extensions, Rust is mostly a subset of C++ that is a lot safer, written a lot more recently
 - Very, very popular
 - Lots of libraries, are being rewritten in Rust now under the hood
 - Rust borrow checker (compiler) actually makes sure your code is memory safe, type safe, etc. A LOT more safeguards than C++.
 - It's also backwards compatible with C++, you can write C++ code in Rust.
 - Wait, so then what's the downsides?
 - Aside from longer compile times, there really isn't many, beyond lack of classes, which some consider a plus
 - If you have some free time check it out!

- We wish to hide valuable bits!
 - Protect from intruders
- Problem: A C style struct exposes everything by default
 - But we can change that
- Three strategies:
 - Public
 - Protected
 - Private

Making the state private

- Everything after the word `private` is private!
- Note: We must then move our implementation outside of our class, since the members are not yet declared.
- Also will want some getters and setters (ex: `getName`, `setName`).

student.h Example in C++

```
struct Student {
    Student(); // default constructor
    Student(const std::string& s); // string constructor
    Student(const std::string& s, double mt, double final); // extended constructor
    Student(const Student& other); // copy constructor
    Student(Student&& other); // move constructor
    Student& operator=(const Student& other); // copy assignment
    Student& operator=(Student&& other); // move assignment
    ~Student();
    void read(std::istream& is);
    void print(std::ostream& os) const;
private:
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
};
```

Making the state private

- We could use the private keyword first, and keep our methods in our struct
- But that is a bit strange, since you can omit public by default, and you'd need to add the public keyword later
- You can have private and public listed as many times as you want though

student.h Example in C++

```
struct Student {  
private:  
    std::string name_;  
    double mt_, final_;  
    std::vector<double> homeworks_;  
public:  
    Student(); // default constructor  
    Student(const std::string& s); // string constructor  
    Student(const std::string& s, double mt, double final); // extended constructor  
    Student(const Student& other); // copy constructor  
    Student(Student&& other); // move constructor  
    Student& operator=(const Student& other); // copy assignment  
    Student& operator=(Student&& other); // move assignment  
    ~Student();  
    void read(std::istream& is);  
    void print(std::ostream& os) const;  
};
```

Making the state private

- You can also make functions like the assignment operator private!
 - Prevents assignment to your object

Why is public the default?

- Not a great default
- BUT... backwards compatible with C
- Use different keywords to make everything private... like with classes

Making the state private

- By default in classes everything is private!
- Start with private variables, then public functions with the public keyword
- NOTE: functions defined in private ARE private. Only the Student class can call them.

student.h Example in C++

```
class Student {
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
public:
    Student(); // default constructor
    Student(const std::string& s); // string constructor
    Student(const std::string& s, double mt, double final); // extended constructor
    Student(const Student& other); // copy constructor
    Student(Student&& other); // move constructor
    Student& operator=(const Student& other); // copy assignment
    Student& operator=(Student&& other); // move assignment
    ~Student();
    void read(std::istream& is);
    void print(std::ostream& os);
};
```

Class vs Struct

- So the only difference between class and struct is the private default state?
 - Essentially... yes
- Generally:
 - Struct is used to keep track of data without behavior
 - Class is used to keep track of data and behavior.
- But this is just a convention.
- What about protected?
 - We'll come back to this with inheritance

- C++ supports the separation of
 - Contract (the what)
 - Implementation (the how)
- Two mechanisms
 - Header and implementation file
 - Abstract classes (yet to cover)

Abstract Classes

- Idea: A class with
 - State and behavior
 - No implementation or a partial implementation
- Corollary
 - One NEVER instantiates an abstract class
 - One ONLY subclasses an abstract class
- In Python this can be done with `collections.abc` (VERY common in industry)

Abstract Classes

- Virtual, must use for abstract class methods
- We set these equal to zero to make it clear: Not an implementation!
- a "concrete" subclass can introduce these methods

student.h Example in C++

```
class AStudent {  
public:  
    virtual void read(std::istream& is) = 0;  
    virtual void print(std::ostream& os) = 0;  
    virtual void setName(const std::string& n) = 0;  
};
```

Abstract Classes

- Here's the idea:
 - Use inheritance to claim that you support the contract
 - Provides and implementation in the subclass for pure methods
 - Pure methods are methods that equal zero and don't carry an implementation
- Our subclass can also add additional methods and extensions beyond our abstract class, similar to Python

Abstract classes

- We implement everything the methods in a subclass, plus additional
- Types must match between the two! Or else it's not overriding, but overloading

student.h Example in C++

```
class AStudent {
public:
    virtual void read(std::istream& is) = 0;
    virtual void print(std::ostream& os) = 0;
    virtual void setName(const std::string& n) = 0;
};

class Student: public AStudent { // We will revisit
    std::string name_;
    double mt_, final_;
    std::vector<double> homeworks_;
public:
    //... other constructors
    void read(std::istream& is);
    void print(std::ostream& os);
    void setName(std::string& n);
};
```

Abstract Classes

- Let's take a look at the implementation (files on huskyct)
- And an implementation using the interface from the abstract class

- C++ Lets you:
 - Define methods (prefix style)
 - Define operators (prefix, infix, and postfix!)
 - Similar to magic methods in Python, this can make our objects feel like normal types in C++ (like an int)

End of day 20

Complex Number Example

- For an example of how to do this, we'll look at complex numbers.
- Remember from math class:
 - $x^2 + 1 = 0$
 - $x^2 = -1$
 - What number squared gives us -1 ?
 - An imaginary number!
- Complex numbers are made of a real part and an imaginary part.
- Standard form: $a + bi$ (where $a, b \in \mathbb{R}$ and $i^2 = -1$)
- Complex numbers are nice because they add together very simply:
 - $(a + bi) + (c + di) = (a + c) + (b + d)i$

End of day 21

Course Project?

- How many people have started?

- One tool that may be useful to you guys as you complete the course project: agentic programming
- Many students have probably used AI like ChatGPT
- You ask a question, you get back an answer
- But there is a much more powerful way of utilizing programming for AI, with agentic programming

- With agentic programming, the AI runs in something called a "code harness"
- If you ask the AI a question, it will recursively iterate on itself using the code harness
- For example, we could ask AI to build us a website
 - The AI will write our website across many files
 - The AI will run the code
 - If there are bugs, the AI will attempt to fix them

- So is AI coming for your jobs? After all, what can AI not do then?
- Well, first off it's very expensive. Often more expensive than a human.
- Additionally, the larger the context, the larger the codebase, the worse AI performs
- Many, many papers have been written about how AI for large codebases over time is far worse than humans without AI
- Instead, you need humans to guide the AI and make good design decisions, and correct it when it hallucinates

- For example, you have the opportunity to use AI for some extra credit with the course project to build a website where you deploy your code using WASM
- You can use AI for the entire course project if you wish
 - Fair warning though... last year people thought AI could complete the entire course project
 - Some people used it for the latter half, and AI destroyed their code so much they had to start over
 - Others tried to AI their way through the whole thing and never got it working
- When iteration is easy, AI is great. But...
- AI fails on larger projects
- AI fails when testing is difficult

- So today we'll demoing Claude Code throughout class just to demonstrate how useful it is and hopefully to inspire you guys to try it out
- I've built a lot of stuff with Claude Code, like clientide.com, Furuness.com, bgpsimulator.com, while relying on the AI agent to various degrees.
- Keep in mind, many companies do not even look at candidates that do not list either Claude Code or Codex on their resume. They get thrown away.
- So let's build a mock website today for CSE3150!
 - We'll revisit this as we code our C++ today

- What should we make our prompt for our CSE3150 website?

- Four pillars to OO programming
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Delegation
 - Hopefully everything except for this is a repeat from CSE2050

Encapsulation

- The famous "has a" relationship
 - A car has a wheel
 - A wheel has a tire
 - A car has four seats
 - A queue has a linked list
 - A linked list has a head node and a tail node
 - A binary tree has a root node
- Purpose
 - Bundle things together as a cohesive package
- Note:
 - We already used it!
 - Struct and class are used to offer this feature

Encapsulation

- The purpose is to bundle related entities to form a highly cohesive new entity
- Attributes can be whatever we want at all
 - Scalar (int, double, etc)
 - Arrays
 - Objects
 - etc

- Language Abstractions
 - Struct
 - Class
- Both have the same mechanics
- The only difference is the defaults for privacy

Let's build a Point class

point.h

```
#pragma once
#include <ostream>
class Point {
    double x_; double y_;
public:
    Point() { x_ = y_ = 0; }
    Point(double x, double y) { x_ = x; y_ = y; }
    Point(const Point& p) : x_(p.x_), y_(p.y_) {}
    double getX() const { return x_; }
    double getY() const { return y_; }
    void set(double x, double y) { x_ = x; y_ = y; }
    Point& operator=(const Point& p) {
        x_ = p.x_; y_ = p.y_; return *this;
    }
    friend Point operator+(const Point& p1, const Point& p2);
    friend Point operator*(const Point& p1, double s);
    friend std::ostream& operator<<(std::ostream& os, const
        Point& p);
```

What is our Point class doing?

- Let's review, for each of these functions, what is going on?
- What are all the different type of constructors called here?
- Which ones are using initializer lists and which ones are not?
- What does friend mean here?
- What operators have we overloaded?

Point Implementation

point.cpp

```
#include "Point.h"
#include <ostream>

Point operator+(const Point& p1, const Point& p2) {
    return Point(p1.x_ + p2.x_, p1.y_ + p2.y_);
}

Point operator*(const Point& p, double s) {
    return Point(p.x_ * s, p.y_ * s);
}

std::ostream& operator<<(std::ostream& os, const Point& p) {
    os << "(" << p.x_ << ", " << p.y_ << ")";
    return os;
}
```

Point Demo

main.cpp

```
#include <iostream>
#include "Point.h"
int main() {
    Point a(1.0, 2.0);
    Point b(3.0, 4.0);
    Point c = a + b;           // add two points
    Point d = c * 2.0;         // scale by 2
    std::cout << "a = " << a << "\n";
    std::cout << "b = " << b << "\n";
    std::cout << "c = " << c << "\n";
    std::cout << "d = " << d << "\n";
    return 0;
}
```


Rectangle

rectangle.h

```
#include <memory>
#include <ostream>
#include "Point.h"
class Rectangle {
    std::shared_ptr<Point> corner_;
    double w_, h_;
public:
    Rectangle() : corner_(new Point), w_(0), h_(0) {}
    Rectangle(double x, double y, double w, double h)
        : corner_(new Point(x, y)), w_(w), h_(h) {}
    std::shared_ptr<Point> getCorner() const {return corner_;}

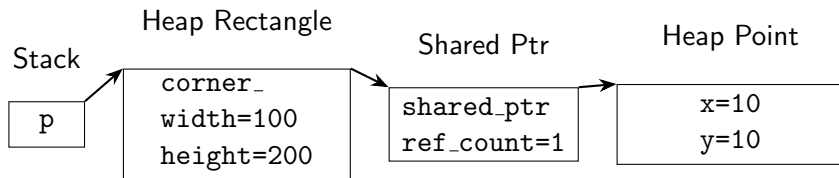
    double getWidth() const { return w_; }
    double getHeight() const { return h_; }
    friend std::ostream& operator<<(std::ostream& os, const
        Rectangle& r) {
        return os << "rect(" << *r.corner_
            << "," << r.w_ << "," << r.h_ << ")";
    }
};
```


Here we have our point object allocated on the heap

- What is going on in this rectangle class?
 - Why do we not need a destructor here, even though the point is on the heap?
- This is the common model in languages like Java and Python
 - EVERYTHING lives on the heap in Python
- In C++ we get to choose
- Let's check out why...

Rectangle Object Model

- `Rectangle* p = new Rectangle(10, 10, 100, 200);`



What is the problem with this model?

- Let's take a look at some code that will reveal some of the flaws in these design decisions

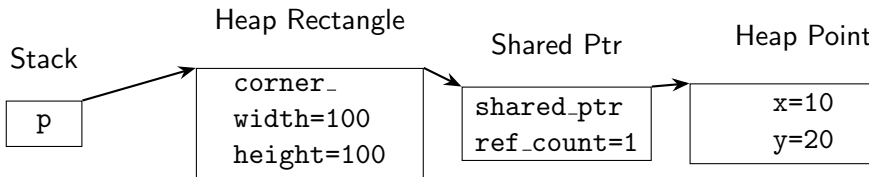
Rectangle Demo

main.cpp

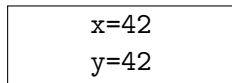
```
#include "rect.h"
#include <iostream>
int main()
{
    Rectangle* p = new Rectangle(10, 20, 100, 100);
    std::cout << "Before: " << *p << std::endl;
    Point p1(42, 42);
    *p->getCorner() = p1;
    std::cout << "After: " << *p << std::endl;
    return 0;
}
```

Step 2: Create p1

- `Point p1(42, 42);`

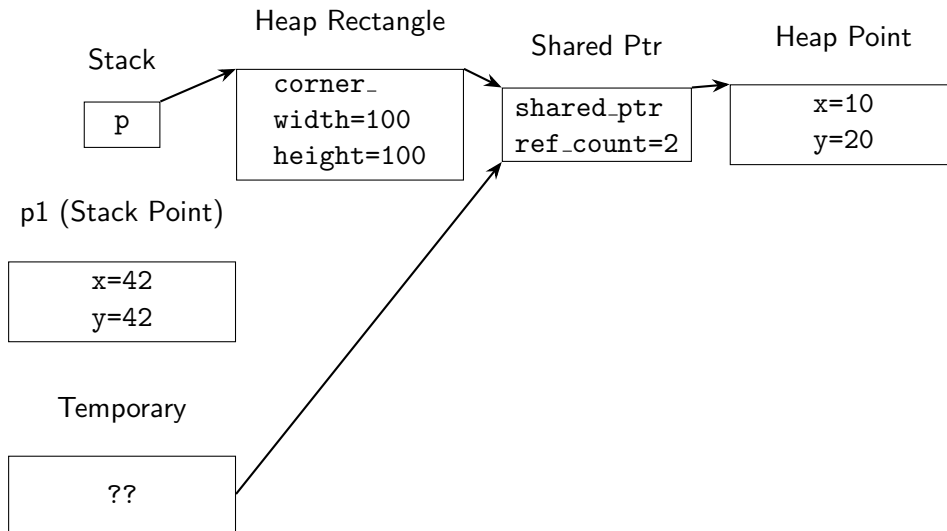


p1 (Stack Point)



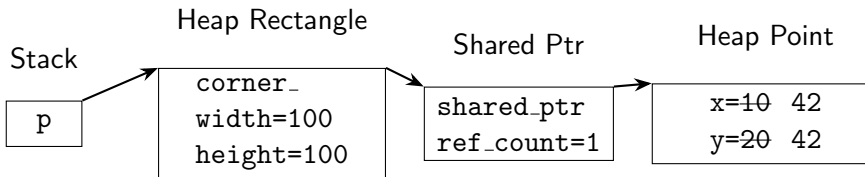
Step 3: Temporary Pointer

- `*p->getCorner()`

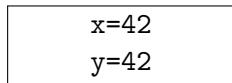


Step 4: Assign p1

• = p1;



p1 (Stack Point)



What is the problem with this model?

- Now when we print our rectangle, it's not at 10, 20??
- This doesn't make sense... rectangles shouldn't be moving around
- The problem with encapsulation is that it's so easy to break
 - Implementation should be separate from contract
 - Just like queues can use vectors or linked lists but it operates the same way,
 - Here we shouldn't be letting a user manipulate the "guts" of the rectangle
- We want to imagine in those diagrams that there is a giant box around our object
 - Like a castle with a moat, nothing should get in!
- So how can we fix this?

The fix

- Simply allocate the point in the class on the stack!
- Now if getCorner is called, it returns a copy of the point, rather than an object on the heap
- This is a much cleaner solution! Everything is colocated in memory (right next to each other)

corner_	x_	10
	y_	20
width	100	
height	200	

Rectangle (stack-allocated Point)

rectangle.h

```
class Rectangle {
    Point _corner;
    double _w, _h;
public:
    Rectangle() : _w(0), _h(0) {}
    Rectangle(double x, double y, double w, double h)
        : _corner(x, y), _w(w), _h(h) {}
    Point getCorner() const { return _corner; }
    double getWidth() const { return _w; }
    double getHeight() const { return _h; }
    friend std::ostream& operator<<(std::ostream& os,
                                    const Rectangle& r) {
        return os << "rect(" << r._corner << ", "
                   << r._w << ", " << r._h << ")";
    }
};
```

End of day 22

Moving on... Inheritance

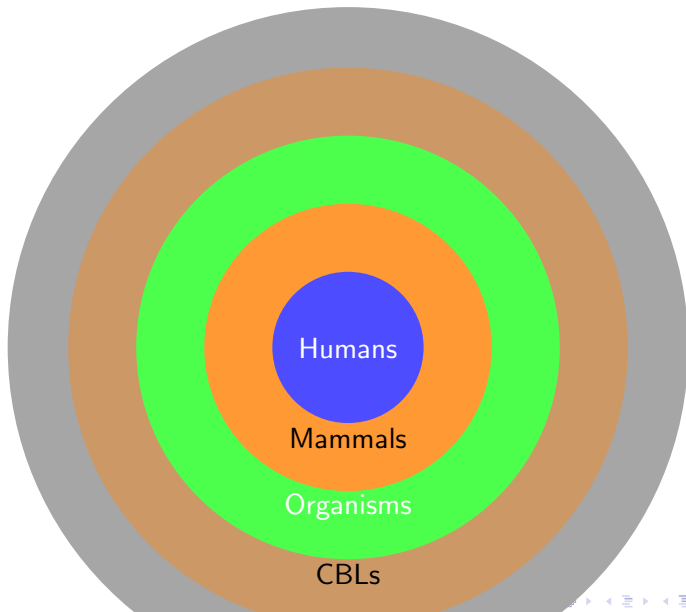
- What it promotes
 - Code reuse
 - Code specialization
- Related concepts
 - Sub typing
 - Sub classing
 - Abstract classes
 - etc

Purpose

- From a pragmatic standpoint
 - Code reuse
- From a theory standpoint
 - Support type refinements
 - aka subtyping through subclassing

- Consider concepts such as
 - Humans, Mammals, Organisms, Carbon Based Lifeforms (CBLs), Chemical Compounds (CCs)
- They are related!

Venn Diagram



- With every ring in our concentric circle we refine the type!
- Every object is being more and more restricted
 - And there are less and less objects as you go

- With each specializations
 - Requirements grow
 - The set of entities that meet those requirements shrinks
- Specialization is akin to
 - Making subsets
 - Making subtypes

Requirements

- These can take different forms
- Mainly:
 - Attributes
 - Specific behaviors
- Ex:
 - Car
 - Do you have an attribute for four wheels
 - Do you have a behavior of electric charging or ICE

What about in C++?

- The language supports subclassing
- subtypes ARE subclasses
 - They are identical
- Sometimes it's for code, sometimes it's type refinement
 - You'll often use this for subtypes rather than code
 - Code reuse is often just a side effect

- Let A, B be two types
 - We write $A <: B$ to state that A is a subtype of B
- Example:
 - $A = \text{Car}$
 - $B = \text{Vehicle}$
 - $A <: B$ means Car is a subtype of Vehicle

Subsumption

- Whenever $A \leq B$
 - Anytime a B is expected, one can provide an A
- In other words:
 - A is a subtype
 - aka a subset of B
 - Hence every A is also B

ex.h

```
foo(Vehicle& v){};  
Car c = ...  
foo(c); // Legal!
```


Mammal and Human Classes

vehicle.h

```
#include <iostream>

class Vehicle {
public:
    Vehicle() {}
    void print() {
        std::cout << "I (" << this << ")'m a vehicle!" <<
            std::endl;
    }
};

class Car : public Vehicle {
public:
    Car() {}
    void print() {
        std::cout << "I (" << this << ")'m a car!" << std::
            endl;
    }
};
```

Main Function

main.cpp

```
int main() {  
    Vehicle v;  
    Car c;  
    v.print();  
    c.print();  
    foo(v);  
    foo(c);  
    return 0;  
}
```

Subsumption

- Here Car is inheriting from Vehicle; This is the syntax
- The public here means something different than the attrs/methods public
- Here it means we want to let the compiler know that Car is a subclass of Vehicle
 - This lets cars be passed into functions requiring vehicles

ex.h

```
class Car : public Vehicle {
```

Subsumption

- You can omit public
 - Then subsumption is not applied
 - You can't pass cars into things that use vehicles
- You have both subtyping and subsumption
 - subsumption USES subtyping to verify the validity of the types in specific statements
- Almost all of the time, you will use public when subtyping
 - It's not the default though, so you must remember

ex.h

```
class Car : Vehicle {
```

- Both calls to foo are fine so long as subsumption is present
- Even though foo expects a vehicle, we can pass a car
 - Inheritance is an "is a" relationship
 - A car "is a" vehicle
- Since a car "is a" vehicle calls to foo using cars is legal
 - The object identity is preserved (see the pointer!)
 - The compiler won't change the types at all due to subsumption
- There is a :public annotation in front of the inheritance stanza
 - This is related to the pragmatic view of inheritance

Running this code

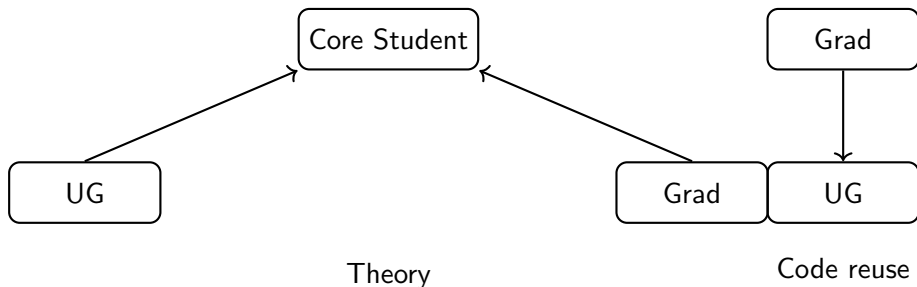
- Why do we get I'm a vehicle??
 - We passed in a car... but it called the vehicles print instead
 - This is VERY different from how languages like Python, Java work
- This is called polymorphism
 - In C++, you can define your classes without polymorphism
 - This is what we've done thus far
 - As we defined our classes, when we pass car into functions for vehicle...
 - It will call the vehicle implementation methods

Diving Deeper: The Pragmatic View

- Given two classes A and B where
 - A and B share lots of common code
 - A refines and specializes some of B
 - Adding state
 - Adding or changing behaviors
- Then A should inherit the common code of B
- You can have both A and B inheriting from the same parent
 - But this isn't great or intuitive
 - C++ provides other ways such as through delegation... we'll get into this a bit later

Two Views of Inheritance

- Imo theory version is much better, cleaner
- Undergrads can do some things that grads don't need, not a proper superclass (imo)



Inheritance and Privacy

- When inheriting you can alter the privacy of what you inherit
 - Public
 - Protected
 - Private
- For example, if UG is a superclass of Grad and Grad inherits privately, then only Grad can call UGs methods, not the outside world

- Semantics
 - What you inherit keeps it's previous status
 - Public remains public
 - Protected remains protected
 - Private remains private, Subclasses can not access the parents private members.
 - Whatever you override
 - Uses the privacy setting of the overridden function

- The common case
 - But it is NOT the default when working with a class
 - It is however, the default when working with a struct
- When A publicly inherits from B
 - A
 - A's subclasses
 - And every function knows that A inherits from B

Protected Inheritance

- Semantics
 - What you inherit CHANGES it's previous status
 - Public becomes protected
 - Protected remains protected
 - Private remains private. Subclasses can not access the parents private members.
 - Whatever you override
 - Uses the privacy setting of the overridden function

vehicle.h

```
class Car : protected Vehicle {
```

Protected Inheritance

- This is not the default
 - You must ask for it with class A: protected B
- When A in a protected way inherits from B
 - A and it's subclasses know that they inherit from B
 - Nobody else knows

- Semantics
 - What you inherit changes it's previous status
 - Everything becomes private (public, protected, private)
 - Whatever you override
 - Uses the privacy setting of the overridden function

Private Inheritance

- THE DEFAULT for classes
 - When class A inherits in a private way from B
 - A knows that it inherits from B
 - As subclasses are clueless
 - Everything else is clueless
 - The inheritance fact is completely hidden
- This is often used when you are in the mindset of "code reuse"
 - But doesn't really stick with the nice theoretical overview of OO
 - But C++ supports both

vehicle.h

```
Car c;  
Vehicle& vr = c;  // ILLEGAL IF PRIVATE!
```

AI Demo (Final)

Inheritance vs Overriding

- When class A inherits from class B
 - It has all attrs of B
 - It has all methods of B
 - It can add attributes or methods
 - But it can also:
 - Upgrade or refine some methods
 - That's called overriding. NOT overloading!

Inheritance vs Overriding

- When class A inherits from class B
 - It has all attrs of B
 - It has all methods of B
 - It can add attributes or methods
 - But it can also:
 - Upgrade or refine some methods
 - That's called overriding. NOT overloading!
- Next lecture we'll get into how this works

Example: Grading policy

- Consider two classes
 - Undergrad
 - Grad
- Graduates are like undergrads
 - Except for grading
 - Grads need a 95 to get an A, undergrad needs only an 80

Grad vs Undergrad

Example

```
#include <ostream>
#include <string>
#include <iostream>
class UGrad {
protected:
    std::string name_;
    double grade_;
public:
    UGrad(const std::string& n, double g): name_(n), grade_(
        g) {}
    const char letterGrade() const;
    friend std::ostream& operator<<(std::ostream& os, const
        UGrad& ug){
        return os << ug.name_ <<< "(UG)=" << ug.grade_ << "(
            " << ug.letterGrade();
        }
}
```

Grad vs Undergrad

Example

```
class Grad:public UGrad {
public:
    Grad(const std::string& n, double g): UGrad(n, g) {}
    const char letterGrade() const;
    friend std::ostream& operator<<(std::ostream& os, const
        Grad& grad){
        return os << grad.name_ <<< "(G)=" << grad.grade_ <<
            "(" << grad.letterGrade();
    }
}
```

Grad vs Undergrad

Example

```
const char UGrad::letterGrade() const{
    if (80 < grade_ && grade_ <= 100){return 'A';}
    else {return 'F';}
}

const char Grad::letterGrade() const{
    if (95 < grade_ && grade_ <= 100){return 'A';}
    else {return 'F';}
}
```


Grad vs Undergrad

Example

```
int main(){
    UGrad s1("Bernard", 61);
    Grad s2("Bob", 81);
    UGrad& gr = s2;
    std::cout << s1 << std::endl;
    std::cout << s2 << std::endl;
    std::cout << gr << std::endl;
}
```

Example: Grading policy

- Running our code, we get Bernards and Bobs grades no problem
- But when printing `gr`, we get that Bob is an undergrad and Bob gets graded on a curve!
 - What's going on here?
 - This brings us back to our concept that polymorphism is NOT the default in C++
 - This is because polymorphism is EXPENSIVE. There is a cost to doing this.
 - `gr` is a reference to an undergrad
 - This behavior is different from Java/Python
- You can still have polymorphism, with one special keyword...

- You have inheritance and lack polymorphism, the solution is dynamic binding
- We have compile time polymorphism
 - Method overloading (this is NOT method overriding!)
 - Template functions and classes
- Now let's talk about RUNTIME polymorphism
 - This is dynamic binding of overridden (NOT overloading) methods

- Provide the ability to respond to messages based on the dynamic type rather than the compile time type
- Meanwhile in Java and Python, this is the default
- In C++ you get to choose
 - Polymorphism carries a cost, which is why C++ gives you a choice

The keyword... finally

- Remember the `virtual` keyword for our abstract classes?
- That is what it does: it switches the method from
 - Static binding (compile-time types)
 - To dynamic binding (run-time types)

Grad vs Undergrad

Example

```
class UGrad {
protected:
    std::string name_;
    double grade_;
public:
    UGrad(const std::string& n, double g): name_(n), grade_(
        g) {}
    virtual char letterGrade() const;
    virtual std::string kind() const { return "UGrad"; }
    friend std::ostream& operator<<(std::ostream& os, const
        UGrad& ug){
        return os << ug.name_ << " (" << ug.kind() << ") = "
            << ug.grade_ << " (" << ug.letterGrade()
            << ")";
    }
};
```

Grad vs Undergrad

Example

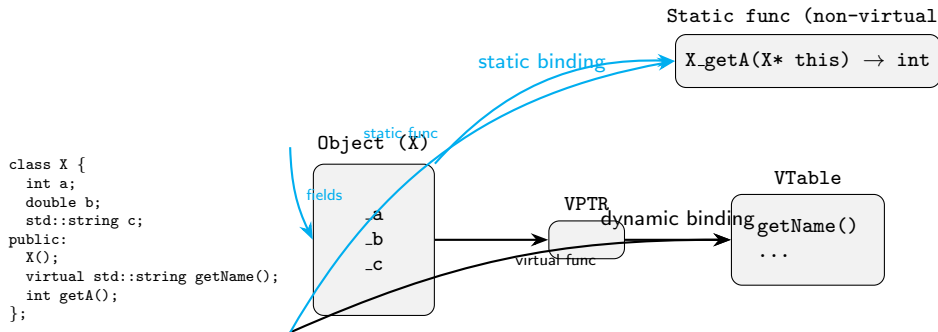
```
class Grad : public UGrad {  
public:  
    Grad(const std::string& n, double g): UGrad(n, g) {}  
    // NOTE: when a function is declared virtual once, it  
    // stays  
    // that way for the entire inheritance chain  
    char letterGrade() const;  
    std::string kind() const { return "Grad"; }  
};
```

Tricky Inheritance

- Very convenient to use inheritance, but a few caveats:
 - Space usage and memory layout
 - Overload and overrides
 - Deallocation
 - Multiple inheritance
 - Diamond inheritance

- An object is
 - A VPTR aka a pointer to a table containing pointers to dynamically bound methods
 - Collection of fields for attributes

Space & Memory



Static binding: compiler resolves call directly to function address

Dynamic binding: uses vtable lookup at runtime via VPTR

Space and Memory

- An object is
 - A VPTR aka a pointer to a table containing pointers to dynamically bound methods
 - Collection of fields for attributes
- Keep in mind, objects don't get a VPTR (a pointer to the virtual table) if there are no virtual methods!
- It adds space!
- Also keep in mind, space alignment is the same in C++ as it is for C
 - Hopefully you guys remember this from CSE3100
 - int 4 bytes, double 8 bytes, string 16 bytes, vptr 8 bytes for a 64 bit machine
 - There will be some padding in your object to get that double onto the right block
 - Note: 32 bit machines change the vptr size to 4 bytes! only 32 bit addresses!

End of day 23

Overload and Override

Example

```
#include <iostream>
class B{
public:
    virtual void f(short) {std::cout << "in B" << std::endl
        ;}
};
class C: public B{
public:
    virtual void f(int) {std::cout << "in C" << std::endl;}
};
int main(){
    B* aPtr = new C;
    aPtr->f(1);
    delete aPtr;
    return 0;
}
```

Overloading vs overriding

- Here function `f` is overloaded, not overridden
- We can change `f` to not be overloaded but overridden and it will do what we expect
- When it's overloaded, `C` has two functions in it's vtable.
 - `f` that takes a short
 - `f` that takes an int
- You can even add the same parameters, but append a `const` to the function type (saying that you will not modify the object). Boom - overloading!
- How to prevent this? override!

Overload and Override

Example

```
#include <iostream>
class B{
public:
    virtual void f(short) {std::cout << "in B" << std::endl
        ;}
};
class C: public B{
public:
    // NOTE: virtual not mandatory!
    virtual void f(int) override {std::cout << "in C" << std
        ::endl;}
};
int main(){
    B* aPtr = new C;
    aPtr->f(1);
    delete aPtr;
    return 0;
}
```

- You can add the final keyword to functions
- This says, NO MORE OVERRIDING! This is the last one.

Example

```
#include <iostream>
class B{
public:
    virtual void f(short) {std::cout << "in B" << std::endl
        ;}
};
class C: public B{
public:
    void f(short) override final {std::cout << "in C" << std
        ::endl;}
};
class D: public C{
public:
    void f(int) override {std::cout << "in D" << std::endl;}
};
int main(){
    B* aPtr = new D; aPtr->f(1); delete aPtr;
    return 0;
}
```

- When would you want to use this?
- It's more for the compiler than it is for the developers
- The compiler can see this, and if your code is only one or two subclasses deep, it can completely optimize away all the overhead of inheritance.
- Under the hood, function calls will occur as if no inheritance was occurring
- You can also add this to an entire class as a keyword for inheritance.

Example

```
int main(){
    UGrad s1("Bernie", 78);
    UGrad* s2 = new Grad("Billy", 67);
    std::cout << s1 << std::endl;
    std::cout << s2 << std::endl;
    delete s2;
    return 0;
}
```

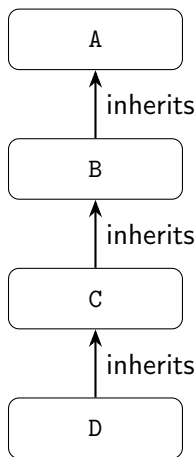
Destructors

- We are deleting s2, which is an undergrad, but...
 - The actual object is a grad!
 - We don't want to free just an undergrad worth of space, we want to free an entire grad students worth of space!
 - We want our deallocation and destruction to be polymorphic.
 - This ties in to homework 5

Example

```
class UGrad{  
protected:  
...  
public:  
    virtual ~UGrad();  
}
```

Virtual Destructors and Destruction Chain

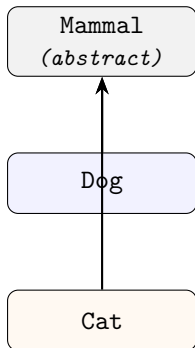


UML Diagram:

A → B → C → D

- When you call `delete` on a pointer to A, and the actual object is of type D, C++ ensures all destructors run correctly.
- If A's destructor is **virtual**, then:
 - The destructor of D is invoked first.
 - After D frees its members, C's destructor runs.
 - Then B's destructor.
 - Finally, A's destructor.
- Each class only cleans up its own members, the chain ensures the entire object is safely destroyed.
- Without a **virtual** destructor in A, only A's destructor runs, leading to resource leaks!

In-Class Exercise: Abstract Mammal Hierarchy



UML: Mammal → Dog, Cat

- Create an abstract base class Mammal:
 - Store the mammal's name
 - Declare a pure virtual function `speak()`
 - Define a virtual destructor that prints when called
- Create two concrete subclasses:
 - Dog: adds breed, says "Woof!"
 - Cat: says "Meow!"
- In `main()`, create an array of `Mammal*` and call `speak()` on each.
- Observe polymorphic behavior and destructor chaining.

- This final week we'll be looking at Lambdas
- Lambdas exist in most programming languages
- They are very convenient if you know how to use them right

- We want to understand lambdas
- We'll start in C and figure out how C does something similar, and how lambdas evolved from this
- In C, the lambda equivalent is a function pointer (a pointer to a function) that you can pass around
- Let's take a look at the find algorithm and re implement this in a C style

Example

```
#include <iostream>
#include <vector>
bool find(std::vector<int> x, int v){
    int low = 0;
    int up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        if (v == x[mid]) return true;
        else if (v < x[mid]) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}

int main() {
    std::vector<int> x = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    bool found_it = find(x, 5);
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
```

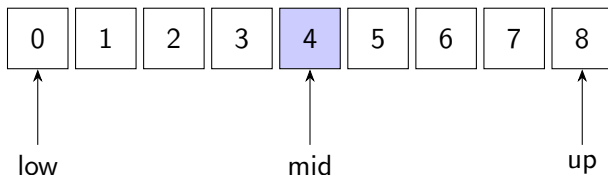
Binary search idea

- Sorted array: $\{0,1,2,3,4,5,6,7,8\}$
- We are searching for the value 5
- We keep track of three indices: low, up, and mid

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

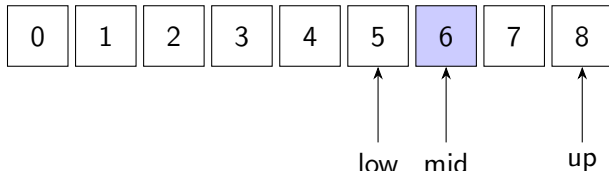
Binary search: initial state

- $\text{low} = 0, \text{up} = 8$
- $\text{mid} = (0 + 8) / 2 = 4$
- Compare $x[\text{mid}] = 4$ with $v = 5$



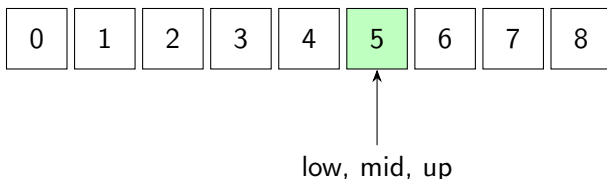
Binary search: move low

- We found $x[\text{mid}] = 4$ and $v = 5$
- Since $v > x[\text{mid}]$, we discard the left half
- New range: $\text{low} = \text{mid} + 1 = 5$, $\text{up} = 8$
- $\text{mid} = (5 + 8) / 2 = 6$



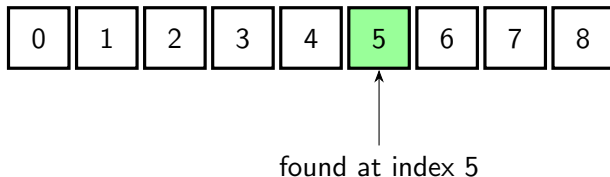
Binary search: move up

- Now $x[mid] = 6$ and $v = 5$
- Since $v < x[mid]$, we discard the right part
- New range: $low = 5$, $up = mid - 1 = 5$
- $mid = (5 + 5) / 2 = 5$



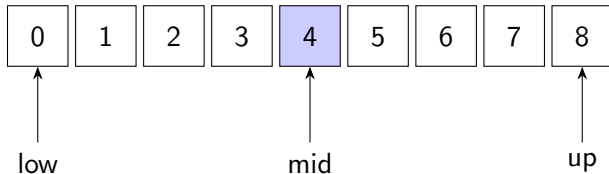
Binary search: found

- `low = up = mid = 5`
- `x[mid] == v`, so we return true



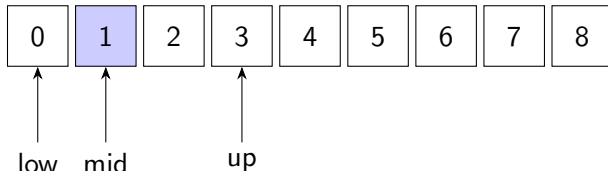
Binary search for 0: initial state

- Sorted array: $\{0,1,2,3,4,5,6,7,8\}$
- We are searching for the value 0
- $\text{low} = 0, \text{up} = 8$
- $\text{mid} = (0 + 8) / 2 = 4$



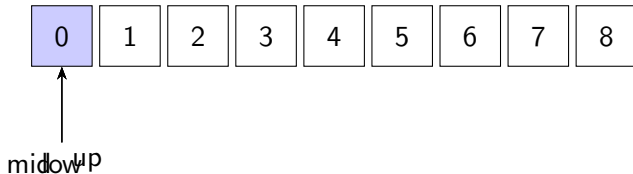
Binary search for 0: move up

- $x[\text{mid}] = 4, v = 0$
- Since $v < x[\text{mid}]$, we discard the right side
- New range: $\text{low} = 0, \text{up} = \text{mid} - 1 = 3$
- $\text{mid} = (0 + 3) / 2 = 1$



Binary search for 0: move up again

- Now $x[\text{mid}] = 1$, $v = 0$
- Since $v < x[\text{mid}]$, we discard the right side again
- New range: $\text{low} = 0$, $\text{up} = \text{mid} - 1 = 0$
- $\text{mid} = (0 + 0) / 2 = 0$



Binary search for 0: found

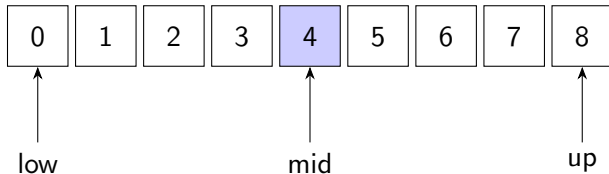
- `low = up = mid = 0`
- `x[mid] == v`, so we return true



found at index 0

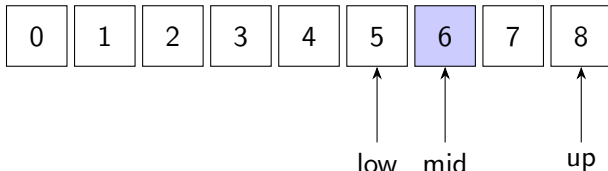
Binary search for 9: initial state

- Sorted array: $\{0,1,2,3,4,5,6,7,8\}$
- We are searching for the value 9 (not in the array)
- $\text{low} = 0, \text{up} = 8$
- $\text{mid} = (0 + 8) / 2 = 4$



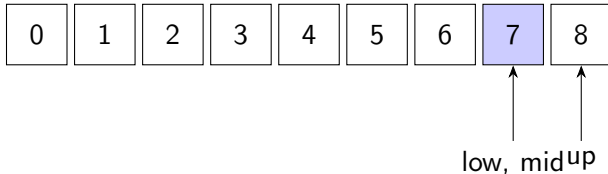
Binary search for 9: move low

- $x[\text{mid}] = 4, v = 9$
- Since $v > x[\text{mid}]$, we discard the left side
- New range: $\text{low} = \text{mid} + 1 = 5, \text{up} = 8$
- $\text{mid} = (5 + 8) / 2 = 6$



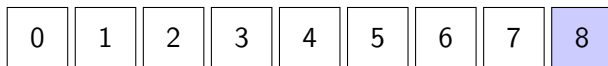
Binary search for 9: move low again

- Now $x[\text{mid}] = 6$, $v = 9$
- Since $v > x[\text{mid}]$, we discard the left side again
- New range: $\text{low} = \text{mid} + 1 = 7$, $\text{up} = 8$
- $\text{mid} = (7 + 8) / 2 = 7$



Binary search for 9: move low yet again

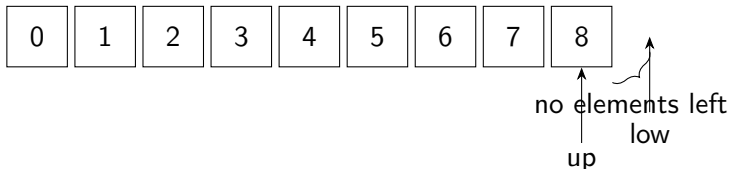
- Now $x[\text{mid}] = 7$, $v = 9$
- Since $v > x[\text{mid}]$, we discard the left side again
- New range: $\text{low} = \text{mid} + 1 = 8$, $\text{up} = 8$
- $\text{mid} = (8 + 8) / 2 = 8$



↑
low, mid, up

Binary search for 9: not found

- Now $x[\text{mid}] = 8, v = 9$
- Since $v > x[\text{mid}]$, we set $\text{low} = \text{mid} + 1 = 9$
- Now $\text{low} = 9, \text{up} = 8$ so $\text{low} > \text{up}$
- Loop stops and we return false



- We want to understand lambdas
- We'll start in C and figure out how C does something similar, and how lambdas evolved from this
- In C, the lambda equivalent is a function pointer (a pointer to a function) that you can pass around
- Let's take a look at the find algorithm and re implement this in a C style

- Remember our function here
- This works only with a specific vector, a vector of int
- We are also making use of less than or equal to specifically for integers
- What if we wanted our own custom objects like spaceships? What would define our equality then? Maybe we'd want to sometimes check if the heights were equal, and sometimes check if the type of fuel was equal.
 - We still want our function to work the same!

Example

```
bool find(std::vector<int> x, int v){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        if (v == x[mid]) return true;
        else if (v < x[mid]) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- Step 1 towards fixing this: Generics with templates
- What's the issue with this?

Example

```
template<class T>
bool find(std::vector<T> x, T v){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        if (v == x[mid]) return true;
        else if (v < x[mid]) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- Step 2 towards fixing this: No more pass by value! Immutability!
- Don't forget that `v` is not an `int`, it's a `T`! The object could be huge, like an image or a spaceship!
- Does this work with doubles?

Example

```
template<class T>
bool find(const std::vector<T>& x, const T& v){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        if (v == x[mid]) return true;
        else if (v < x[mid]) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- [illegible]

- Let's implement a hardcoded custom comparator function.

Example

```
#include <math.h>
const double eps = 0.01;
int compare (double x, double y) {
    double v = fabs(x - y);
    if (v < eps) return 0;
    else if (x < y) return -1;
    else return 1;
}

template<class T>
bool find(const std::vector<T>& x, const T& v){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = compare(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
}
```

- We can run this compared to 1.6666666 and pass
- If we reduce the specificity to 1.6, it doesn't find the element! We aren't specific enough according to our epsilon!
- Still, we don't want to hardcode our comparison. We want to take an arbitrary comparison, so that we can compare any values of any types!

- Let's add a comparator function pointer to our arguments
- Remember the spiral rule!

Example

```
template<class T>
bool find(const std::vector<T>& x, const T& v, int (*cmp)(
    double, double)){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}

int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    bool found_it = find(x, 1.6, compare);
}
```

- Part 5: We must also make our comparator generic!
- The only reason it worked before is because we **only** used doubles, but that's not generic.
- Still, something is wrong. What is it?

Example

```
template<class T>
bool find(const std::vector<T>& x, const T& v, int (*cmp)(T,
T)){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```


- Part 6: Pass by value bug again!

Example

```
#include <math.h>

int compare (const double& x, const double& y) {
    double v = fabs(x - y);
    if (v < eps) return 0;
    else if (x < y) return -1;
    else return 1;
}

template<class T>
bool find(const std::vector<T>& x, const T& v, int (*cmp)(
    const T&, const T&){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = compare(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
}
```

- Part 7: Alternative... pass by template??

Example

```
template<class T, int (*cmp)(const T&, const T&)>
bool find(const std::vector<T>& x, const T& v){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = compare(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }    return false;
}

int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    bool found_it = find<double, compare>(x, 1.6);
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
```

- We have now moved our compare function from the function call find, to the instantiation of find
- Remember, first you have template instantiation (which creates the function with the angle brackets), and only afterwards do you have the function call itself (with the parens)
- Everything else is as before
- The difference...
 - Instantiation of templates contains either types (with typename or class), OR values!
 - This is the first time we're passing a value to a template
- What are the advantages of doing it this way?

- Now the compare function is compiled into our find function at compile time
- Before, compare was being passed in at runtime (this is how C does it)
-
- What are the advantages of doing it this way?
 - The compiler can much better optimize your comparison function
 - Your comparison function can potentially even be inlined and unrolled by the compiler
 - I.e. no function call overhead!
 - It would be as if you'd hardcoded your comparison function into your find function (that level of speed)
- For getting the best performance, you want to do it in this way

- C Function pointers are cheap closures in functional languages
 - There is no state!
- New C++ closures DO have state!
- C++ functions can get what is defined by the environment at the time you define the function
 - The variable names that are bound to those values
 - Any name defined before the closure
- Just like how our function can access our epsilon, or with templates our cmp function

- C++ Closures have lexical scoping
- You capture the value of the name at definition time, just like the epsilon
- C++ closures still refer to and capture variables from the environment
- But what do you capture?
 - Do we capture the entire environment? Or just specific variables?
- What about how we capture it?
 - By copy?
 - By reference? Should our function change if the object we capture is mutated outside of our function?
 - In C++ we have to worry about this!

- Function pointers are a poor mans version of closures though and don't have this C++ functionality
- They have no state!
- They capture nothing at all!
- They only capture global variables, that's it, nothing else
- Onto lambdas and closures!

- Lambdas are true closures for C++
- Lambdas let us specify:
 - What to capture
 - How to capture it

function pointers are simple C style closures

- We want to understand the relationship between lambdas and their environment

- Lambdas are closures in C++
- Closures are built from a block of code
- Closures capture their environment (more details later)
- Closures can have arguments and behave like functions
- You can pass them around and store them! Very convenient and flexible.

- Let's take the simplest example, the compare case!
- Note we use auto, and use it like a function!

Example

```
int main()
{
    auto cmp = [](int a, int b) -> int {
        if (a == b) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    std::cout << "cmp(1,2) = " << cmp(1,2) << std::endl;
    return 0;
}
```

- At the time of defining the function, the lambda can capture environment variables
- Lambdas also have a type, we don't want to write this, it's a bit cumbersome. We can use auto instead.

- So what we are we capturing?
- That's the double bracket!
 - This specifies what to capture
 - And how to capture it
- An empty list means we don't have to capture anything
- It's the same as the function pointer in this case!

- Empty brackets: Capture nothing `[]`
- Brackets with an equals sign: Capture everything by value `[=]`
- Brackets with an ampersand: Capture everything by reference `[&]`
- `[capturelist]` Only captures the named entities, list separated by commas
 - We can proceed by an ampersand to show that we want to capture by reference rather than by value
 - This is the finest level of capture control
 - Generally you want to do it this way so you aren't copying an entire environment

- Let's go back to our example and run it

Example

```
#include <iostream>
int main()
{
    auto cmp = [](int a, int b) -> int {
        if (a == b) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    std::cout << "cmp(1,2) = " << cmp(1,2) << std::endl;
    return 0;
}
```

- Let's modify this to work with an epsilon

Example

```
#include <iostream>
#include <math.h>
int main()
{
    auto cmp = [](double a, double b) -> int {
        if (std::fabs(a-b) <= 0.01) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    std::cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << std
        ::endl;
    return 0;
}
```

- Let's modify this to work with an epsilon outside of the lambda
- Can't capture this without specifying how to capture this!

Example

```
#include <iostream>
#include <math.h>
int main()
{
    const double epsilon = 0.01;
    auto cmp = [](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    std::cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << std
        ::endl;
    return 0;
}
```


- Let's modify this to work with an epsilon outside of the lambda
- Instead we can capture everything by value!

Example

```
#include <iostream>
#include <math.h>
int main()
{
    const double epsilon = 0.01;
    auto cmp = [=](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    std::cout << "cmp(1,1.00001) = " << cmp(1,1.0001) << std
        ::endl;
    return 0;
}
```

- Let's modify this to work with an epsilon outside of the lambda
- We can also capture by reference, then the function will change when the epsilon changes!

Example

```
#include <iostream>
#include <math.h>
using std::cout, std::endl;
int main(){
    double epsilon = 0.01;
    auto cmp = [&](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    epsilon = 0.000000000001;
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    return 0;
}
```

- Let's modify this to work with an epsilon outside of the lambda
- Changing this to capture by value avoids the changing epsilon

Example

```
#include <iostream>
#include <math.h>
using std::cout, std::endl;
int main(){
    double epsilon = 0.01;
    auto cmp = [=](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    epsilon = 0.000000000001;
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    return 0;
}
```

- Let's modify this to work with an epsilon outside of the lambda
- Ideally we would specify the capture list as epsilon (defaults to by value)

Example

```
#include <iostream>
#include <math.h>
using std::cout, std::endl;
int main(){
    double epsilon = 0.01;
    auto cmp = [epsilon](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    epsilon = 0.000000000001;
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    return 0;
}
```

- Let's modify this to work with an epsilon outside of the lambda
- We can change epsilon to be by reference

Example

```
#include <iostream>
#include <math.h>
using std::cout, std::endl;
int main(){
    double epsilon = 0.01;
    auto cmp = [&epsilon](double a, double b) -> int {
        if (std::fabs(a-b) <= epsilon) return 0;
        else if (a < b) return -1;
        else return 1;
    };
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    epsilon = 0.000000000001;
    cout << "cmp(1,1.0001) = " << cmp(1,1.0001) << endl;
    return 0;
}
```

- With lambdas we don't need to declare a true function
- We can declare an anonymous function, such as passing a function in for comparison (rather than adding it to the global namespace)
- It can contain the ability to modify state
 - Either by value
 - Or by reference
- You can declare lambdas in your code really close to wherever you want to use them, rather than defining functions somewhere else (ex: Defining a comparison function for sort right next to where you call sort)
- Like function pointers, you can store them

- Back to our find function!

Example

```
#include <iostream>
#include <vector>
#include <math.h>
template<class T, class Cmp>
bool find(const std::vector<T>& x, const T& v, Cmp cmp){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- Using our lambda now. Notice how it is right next to our find call now.

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    bool found_it = find(x, 1.6666, [](double a, double b)
        -> int {
        if (std::fabs(a-b) < 0.01) return 0;
        else if (a > b) return 1;
        else return -1;
    });
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```


- Adding back our specific capture

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    const double eps = 0.01;
    bool found_it = find(x, 1.6666, [eps](double a, double b
        ) -> int {
        if (std::fabs(a-b) < eps) return 0;
        else if (a > b) return 1;
        else return -1;
    });
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```

- Type of a closure? It depends!
 - Could be a simple pointer to a function
 - Could be an anonymous structure
- Therefore we don't know
 - But we can use `auto` to specify the closure type!

- Example: We could have a normal function pointer:
 - A simple function pointer that capture nothing:
`int (*foo)(int a, int b);`
 - Remember the spiral rule!
 - When we want to capture something: `[a, b](int c) -> bool(...)`
- But what does this look like?

- When we capture variables, C++ stores this in an anonymous struct!
- Something like this:

Example

```
// NOTE: this is just pseudocode
// question mark since it's anonymous, we don't specify the
// name
struct ? {
    int a; // capturing a
    int b; // capturing b
    ptr to a func // pointer to a function
}
```

- Trying to deduce this crazy type... no need
- Just use `auto` and the compiler will infer it for you

- But we can't use auto for function arguments, so how do we pass a lambda into a function?
- There are a few ways, but one way in C++ is to use `std::function<T>`

Example

```
#include <functional> // <-- Don't forget this!
template<class T>
bool find(const std::vector<T>& x, const T& v, std::function
    <int(double, double)> cmp){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }    return false;
}
```

- Can we make this better by setting this to take two Ts?

Example

```
#include <functional> // <-- Don't forget this!
template<class T>
bool find(const std::vector<T>& x, const T& v, std::function
    <int(T, T)> cmp){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- It turns out no! The compiler can not correctly deduce those types, because of the weird struct-like nature of lambdas

- Instead we can add the parameter inside of our main function

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    auto cmp = [](double a, double b) -> int {
        if (std::fabs(a-b) < 0.01) return 0;
        else if (a > b) return 1;
        else return -1;
    };
    bool found_it = find(x, 1.6666, std::function<int(double
        , double)>(cmp));
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```

- Of course, we can one line this entire thing

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    bool found_it = find(x, 1.6666, std::function<int(double
        , double)>([](double a, double b) -> int {
        if (std::fabs(a-b) < 0.01) return 0;
        else if (a > b) return 1;
        else return -1;
    }));
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```


- Instead we can simplify with a template!

Example

```
template<class T, class Cmp>
bool find(const std::vector<T>& x, const T& v, Cmp cmp){
    int low = 0, up = x.size() - 1;
    while(low <= up) {
        int mid = (low + up) / 2; // int division
        const int c = cmp(v, x[mid]);
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- Is this as efficient as we can make this though?

- We can pass the lambda in at compile time!

Example

```
#include <iostream>
#include <vector>
#include <cmath>
template<class T, auto Cmp>
bool find(const std::vector<T>& x, const T& v) {
    int low = 0, up = x.size() - 1;
    while (low <= up) {
        int mid = (low + up) / 2;
        int c = Cmp(v, x[mid]);    // compile-time lambda
                                   object
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- Here we use auto for the template, introduced in C++20
- Our lambda has no type that we can specify here since the compiler generates it as an unnamed struct
- So we must specify auto, to show that we are passing in the actual value (like we did before), rather than an unnamed type

- Just like before, we can pass in our lambda!

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    auto cmp =
        [](double a, double b) {
            if (std::fabs(a - b) < 0.01) return 0;
            else if (a > b) return 1;
            else return -1;
        };
    bool found_it = find<double>(x, 1.6666);
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```

- Can we capture?

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    double epsilon = 0.01;
    auto cmp =
        [epsilon](double a, double b) {
            if (std::fabs(a - b) < epsilon) return 0;
            else if (a > b) return 1;
            else return -1;
        };
    bool found_it = find<double, cmp>(x, 1.6666);
    std::cout << "found? " << (found_it ? "yup" : "no") <<
        std::endl;
}
```

- No! Epsilon is evaluated at runtime, not compile time!
- However, we can force C++ to evaluate this simple expression at compile time by using `constexpr`
 - This lets C++ know that this line should be evaluated at compile time, rather than at runtime
 - Very useful when you want every ounce of speed. Used in gaming, finance (high frequency trading), embedded systems, etc
 - Introduced in C++20, so make sure to pass that into your compilation with `-std=c++20`

Example

```
int main() {  
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,  
        7, 8};  
    constexpr double epsilon = 0.01;  
    constexpr auto cmp =  
        [epsilon](double a, double b) {  
            if (std::fabs(a - b) < epsilon) return 0;  
            else if (a > b) return 1;  
        }  
}
```

- What about partials?
- What if you have a function of two arguments and wish to specialize it down to a single argument?
 - In other words, what if we wanted to make one of the parameters constant?
- C++ can do that!

- In our find function, our use of `v` is very limited
 - We only read from it on one line
 - We never write to it
- Why not just embed `v` into `compare`? Then we should be able to reduce the total number of parameters into our find function.

Example

```
template<class T, auto Cmp>
bool find(const std::vector<T>& x, const T& v) {
    int low = 0, up = x.size() - 1;
    while (low <= up) {
        int mid = (low + up) / 2;
        int c = Cmp(v, x[mid]);    // compile-time lambda
                                   // object
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```


- There are two ways to accomplish this
- One way is to use `std::bind`
- C++ allows you to "bind" or "hardcode" a value as a parameter of a function
- However... this is very, very old style C++ that should no longer be done

- References:

- C++ Core Guidelines (Stroustrup and Sutter): "Prefer lambdas to `std::bind`. `std::bind` is error-prone and hard to read; lambdas are safer and cleaner"
- Google C++ Style Guide: "Avoid `std::bind`. Use lambdas instead"
- LLVM Style Guide: "Lambdas are clearer and more explicit than `std::bind`"
- Herb Sutter (C++ Committee): "Obsolete in modern C++", "A mistake we can't remove, but we can stop teaching"
- For those reasons, I won't be teaching `std::bind`. At the time the course videos were made it was still prevalent, but ten years later or however long it has been, it's no longer relevant and there are essentially zero use cases for it.
- You can still watch the videos posted if you are interested.

- `std::bind` has:
 - implicit captures
 - weird type conversions
 - unintuitive error messages
 - were designed before lambdas were fully formed
 - rarely optimized as well as lambdas
 - placeholder based syntax is very awkward
 - Also can't do `constexpr` with `std::bind`, it wouldn't even work at all with our optimized example and we'd need to revert back to `std::function` and passing in at runtime. It's possible for the language to do this, but no one has implemented it because `std::bind` should not be used.

- There are two ways to accomplish this
- One way is to use `std::bind`
- C++ allows you to "bind" or "hardcode" a value as a parameter of a function
- However... this is very, very old style C++ that should no longer be done

- Back to our function!
- In our find function, our use of `v` is very limited
 - We only read from it on one line
 - We never write to it
- Why not just embed `v` into `compare`? Then we should be able to reduce the total number of parameters into our find function.

Example

```
template<class T, auto Cmp>
bool find(const std::vector<T>& x) {
    int low = 0, up = x.size() - 1;
    while (low <= up) {
        int mid = (low + up) / 2;
        int c = Cmp(x[mid]);    // compile-time lambda object
        if (c == 0) return true;
        else if (c < 0) up = mid - 1;
        else low = mid + 1;
    }
    return false;
}
```

- And the lambda operating as a partial

Example

```
int main() {
    std::vector<double> x = {0, 1, 5.0/3.0, 2, 3, 4, 5, 6,
        7, 8};
    constexpr double eps = 0.01;
    constexpr double target = 1.666666;
    constexpr auto cmp =
        [eps](double target, double value) {
            if (std::fabs(target - value) < eps) return 0;
            else if (target > value) return 1;
            else return -1;
        };
    constexpr auto partial_cmp =
        [target, cmp](double value){
            return cmp(target, value);
        };
    bool found_it = find<double, partial_cmp>(x);
    std::cout << "found? " << (found_it ? "yup" : "no") <<
```

Exercise 1: Basic Lambda

Write a lambda `dbl` that doubles an integer.

- Call it on 3, 10, and 41.
- (Optional) make it `constexpr`.

Starter

```
#include <iostream>

int main(){
    // TODO: auto dbl = ...
    std::cout << dbl(3) << std::endl;
    std::cout << dbl(10) << std::endl;
    std::cout << dbl(41) << std::endl;
}
```

Exercise 1: Answer

Solution

```
#include <iostream>

constexpr auto dbl = [](int x){ return x * 2; };

std::cout << dbl(3) << std::endl;
std::cout << dbl(10) << std::endl;
std::cout << dbl(41) << std::endl;
```


Exercise 2: Capture

Write a lambda `is_large` that tests whether a number is larger than a threshold.

- Capture threshold.
- Test 5, 10, and 20.

Starter

```
#include <iostream>

int main(){
    double threshold;
    std::cin >> threshold;

    // TODO: auto is_large = ...
    std::cout << is_large(5) << std::endl;
    std::cout << is_large(10) << std::endl;
    std::cout << is_large(20) << std::endl;
}
```

Exercise 2: Answer

Solution

```
#include <iostream>

auto is_large = [&](double x){ return x > threshold; };

std::cout << is_large(5) << std::endl;
std::cout << is_large(10) << std::endl;
std::cout << is_large(20) << std::endl;
```

Exercise 3: Custom Comparator

Write a lambda `abs_less` and use it in `min_index`.

- Compare integers by absolute value.

Starter

```
#include <iostream>
#include <vector>
#include <cstdlib> // std::abs

template<class T, class Cmp>
int min_index(const std::vector<T>& x, Cmp cmp){
    int best = 0;
    for(int i = 1; i < x.size(); i++){
        if (cmp(x[i], x[best])) best = i;
    }
    return best;
}

int main(){
    std::vector<int> v = {3, -7, 2, -1};
    // TODO: auto abs_less = ...
}
```

Exercise 3: Answer

Solution

```
#include <iostream>
#include <vector>
#include <cstdlib>

auto abs_less = [](int a, int b){
    return std::abs(a) < std::abs(b);
};

std::cout << min_index(v, abs_less) << std::endl;
```

Exercise 4: Sort With Lambda

Sort a `std::vector<std::pair<std::string,int>>` by the **second** value using `std::sort` and a lambda.

Starter

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

int main(){
    std::vector<std::pair<std::string,int>> v = {
        {"apple",5}, {"banana",2},
        {"cherry",9}, {"date",3}
    };
    // TODO: sort by second
}
```

Exercise 4: Answer

Solution

```
#include <algorithm>
#include <iostream>

std::sort(v.begin(), v.end(),
    [](const auto& a, const auto& b){
        return a.second < b.second;
    });

for(const auto& p : v)
    std::cout << p.first << " : "
               << p.second << std::endl;
```

End of day 24